

《计算机组成原理实验》

(第四次)

厦门大学信息学院软件工程系
2026年4月

目录

一、验证实验

1. MIPS汇编语言程序的运行
2. 在Logisim上运行MIPS程序
3. RISC-V汇编语言程序的运行
4. 在Logisim上运行RISC-V程序
5. Intel x86汇编语言程序的运行
6. ARMv7汇编语言程序的运行

二、设计实验

1. MIPS汇编语言程序设计（排序程序）
2. RISC-V汇编语言程序设计（排序程序）
3. Intel x86汇编语言程序设计（排序程序）

三、挑战性实验

- ARMv7汇编语言程序设计（排序程序）

一、验证实验

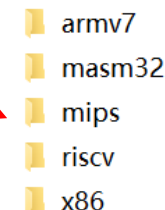
1、MIPS汇编语言程序的运行

- (1) 运行MIPS汇编仿真器

- 解压“**计算机组成原理实验——第4次实验（发给学生）.zip**”，将解压后的5个文件夹拷贝到D盘的根目录中（或其它盘的根目录中）。

- 查看电脑上是否已有JDK？

- 打开“命令提示符”，执行：
 - **java -version**



- 如果没有，下载并运行“jdk-8u121-windows-x64.exe”，安装JDK。

```
命令提示符
Microsoft Windows [版本 10.0.19044.1645]
(c) Microsoft Corporation. 保留所有权利。

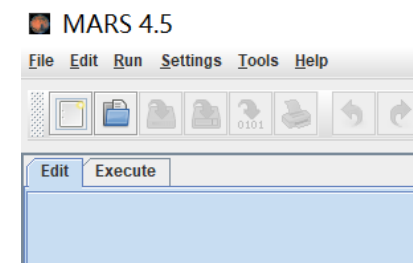
C:\Users\lily2002>java -version
java version "1.8.0_121"
Java(TM) SE Runtime Environment (build 1.8.0_121-b13)
Java HotSpot(TM) 64-Bit Server VM (build 25.121-b13, mixed mode)

C:\Users\lily2002>
```

- 运行MIPS汇编仿真器Mars4_5.jar。

- 打开“命令提示符”，执行：
 - **cd D:**
 - **cd \mips**
 - **java -jar Mars4_5.jar**

```
命令提示符 - java -jar Mars4_...
C:\Users\lily2002>cd \mips
C:\mips>java -jar Mars4_5.jar
四月 24, 2022 1:09:34 下午 java.util.pre
fs.WindowsPreferences <init>
WARNING: Could not open/create prefs root
node Software\JavaSoft\Prefs at root 0
x80000002. Windows RegCreateKeyEx(...) r
eturned error code 5.
```



Edit Execute

About Mars

MARS 4.5 Copyright 2003-2014

Pete Sanderson and Kenneth Vollmar

MARS is the Mips Assembler and Runtime Simulator.

Mars image courtesy of NASA/JPL.

Toolbar and menu icons are from:

- * Tango Desktop Project (tango.freedesktop.org),
- * glyFX (www.glyfx.com) Common Toolbar Set,
- * KDE-Look (www.kde-look.org) crystalline-blue-0.1,
- * Icon-King (www.icon-king.com) Nuvola 1.0.

Print feature adapted from HardcopyWriter class in David Flanagan's
Java Examples in a Nutshell 3rd Edition, O'Reilly, ISBN 0-596-00620-9.

确定

Mars Messages Run I/O

Clear

MARS 4.5

MIPS汇编仿真器

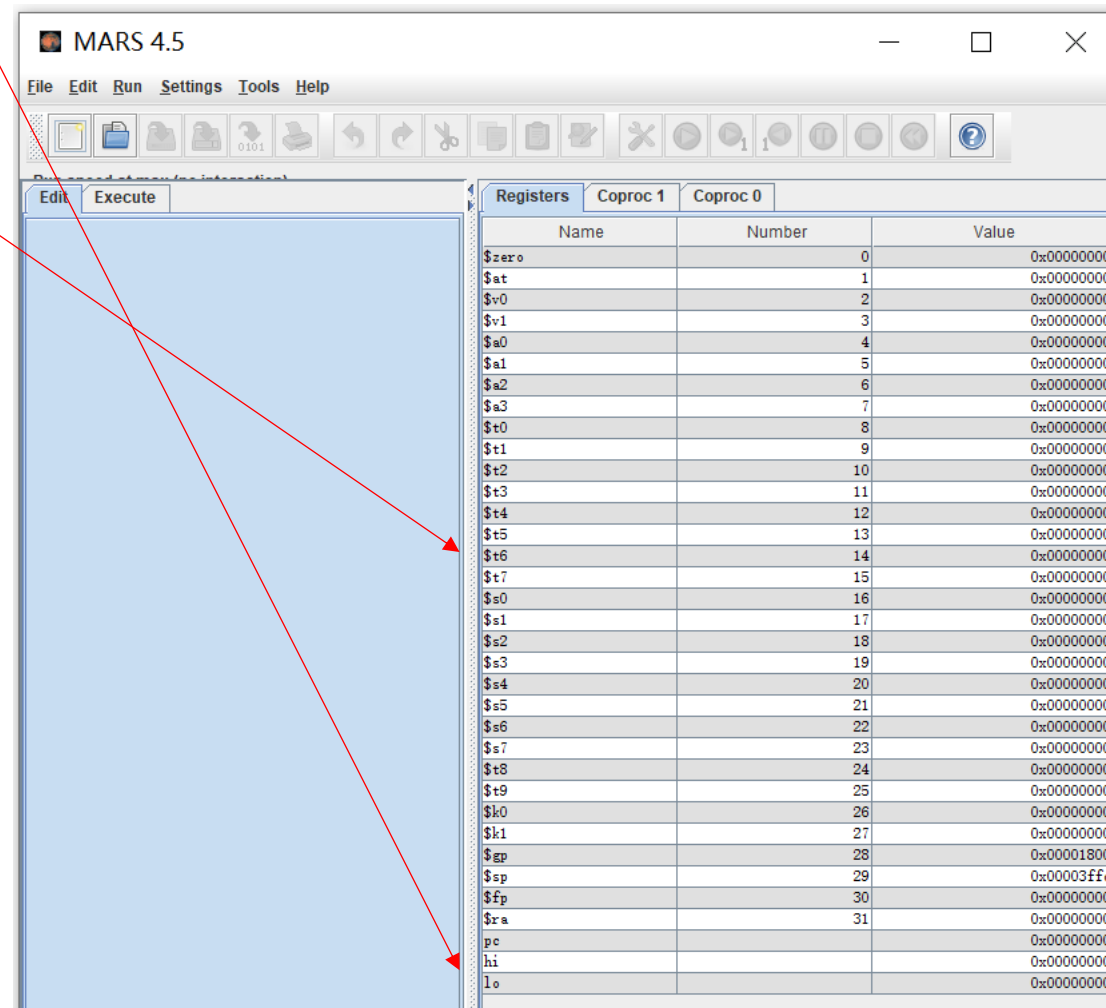
Registers Coproc 1 Coproc 0

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000000
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$t8	16	0x00000000
\$t9	17	0x00000000
\$s0	18	0x00000000
\$s1	19	0x00000000
\$s2	20	0x00000000
\$s3	21	0x00000000
\$s4	22	0x00000000
\$s5	23	0x00000000
\$s6	24	0x00000000
\$s7	25	0x00000000
\$s8	26	0x00000000
\$s9	27	0x00000000
\$gp	28	0x00001800
\$sp	29	0x00002ffc
\$fp	30	0x00000000
\$ra	31	0x00000000
pc		0x00000000
hi		0x00000000
lo		0x00000000

- MIPS处理器的32个寄存器，以及pc、hi、lo寄存器，均为32位寄存器。

表5.7 MIPS的通用寄存器

寄存器#	助记符	释义
\$0	\$zero	固定值为0 硬件置位
\$1	\$at	汇编器保留，临时变量
\$2~\$3	\$v0~\$v1	函数调用返回值
\$4~\$7	\$a0~\$a3	4个函数调用参数
\$8~\$15	\$t0~\$t7	暂存寄存器，被调用者按需保存
\$16~\$23	\$s0~\$s7	save寄存器，调用者按需保存
\$24~\$25	\$t8~\$t9	暂存寄存器，同上
\$26~\$27	\$k0~\$k1	操作系统保留，中断异常处理
\$28	\$gp	全局指针 (Global Pointer)
\$29	\$sp	堆栈指针 (Stack Pointer)
\$30	\$fp	帧指针 (Frame Pointer)
\$31	\$ra	函数返回地址 (Return Address)



- (2) 运行第一个MIPS汇编语言程序

- **helloworld_mips.asm** (位于\mips目录中) 为显示字符串“Hello, World!”的MIPS汇编语言程序。

helloworld_mips.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
#MIPS程序 helloworld_mips.asm 这个程序只能在MARS汇编器上运行

.data

out_string: .asciiz "\nHello, World!\n"

.text

main:

li \$v0,4

la \$a0,out_string

syscall

li \$v0,10

syscall

#数据段开始标志

#代码段开始标志

#4号系统调用, 显示字符串

#获取字符串的地址

#系统调用

#10号系统调用, 程序退出

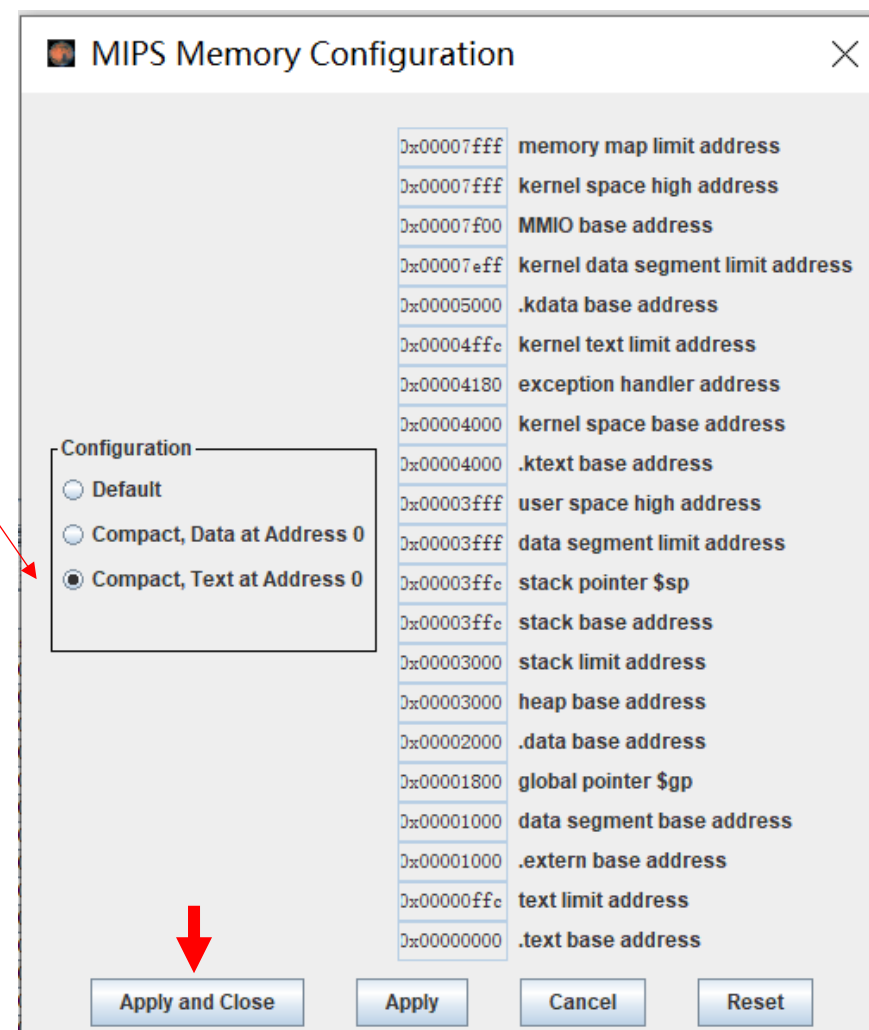
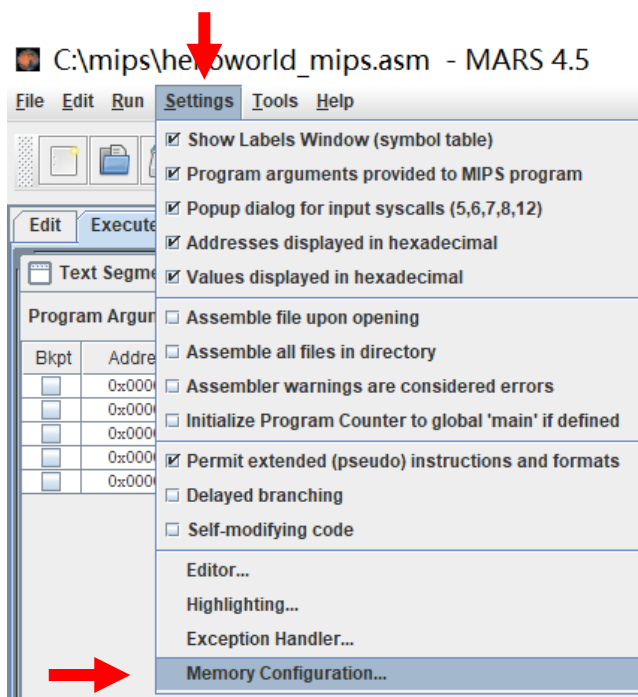
定义ASCII码字符串

立即数4 -> \$v0

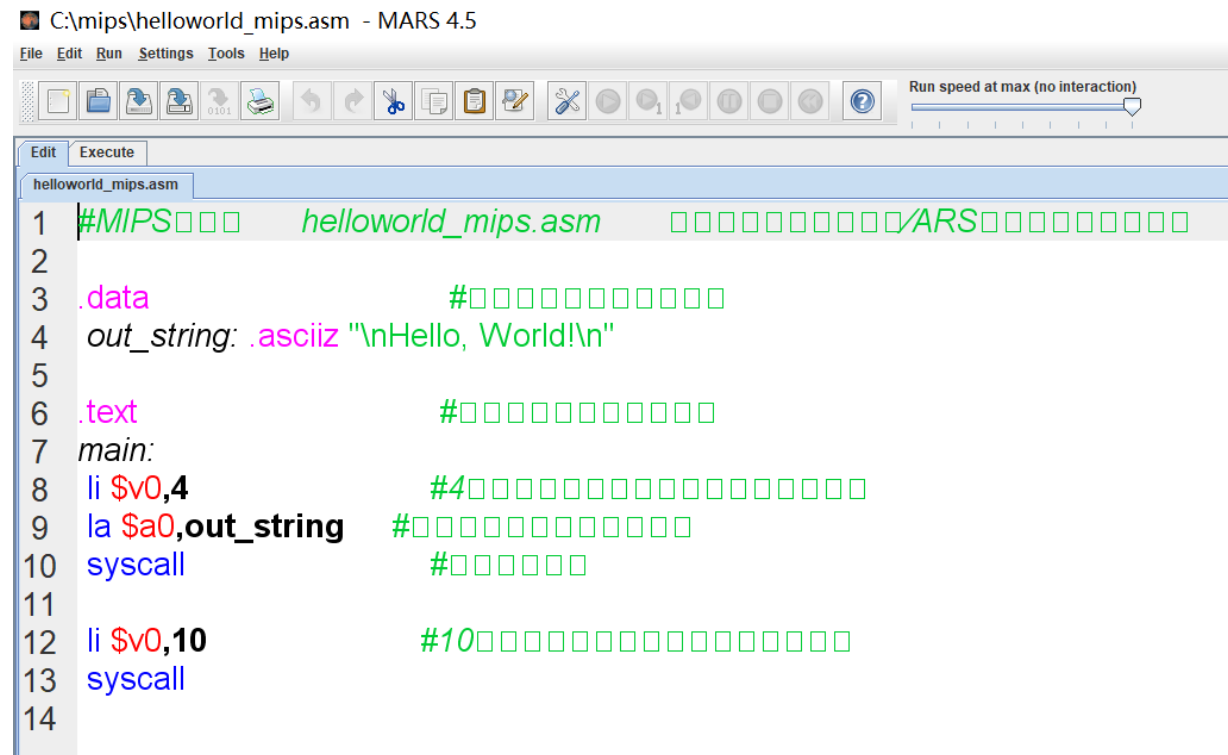
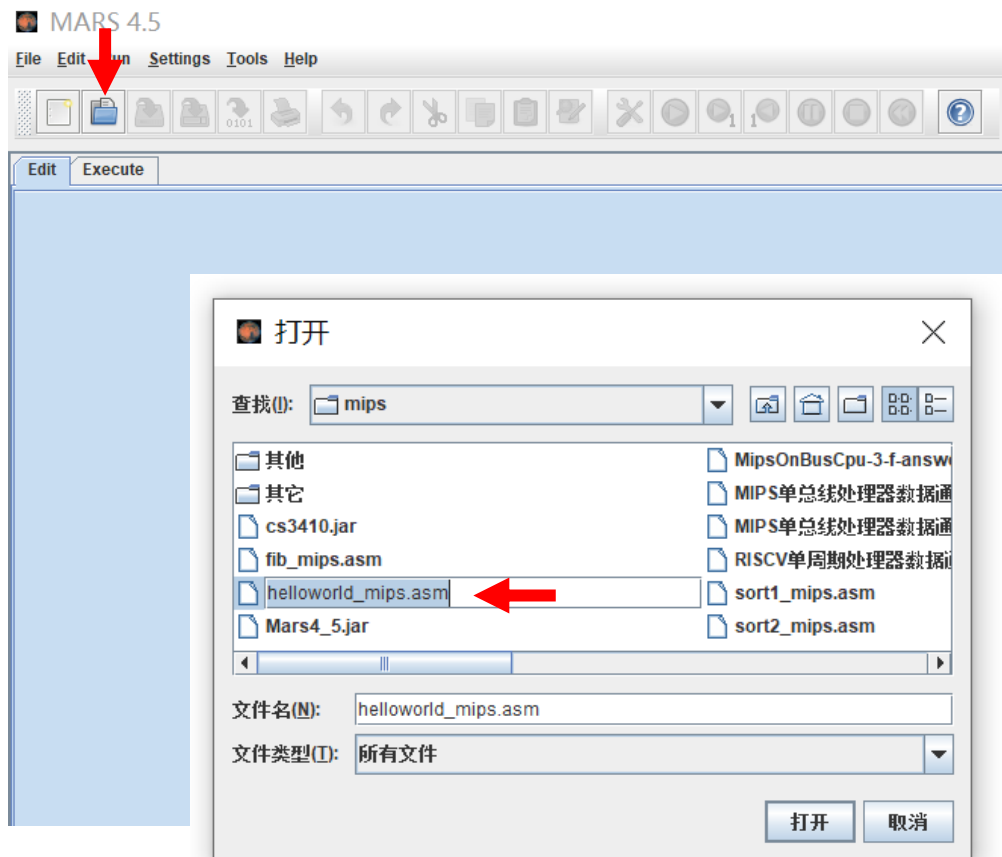
地址out_string -> \$a0

立即数10 -> \$v0

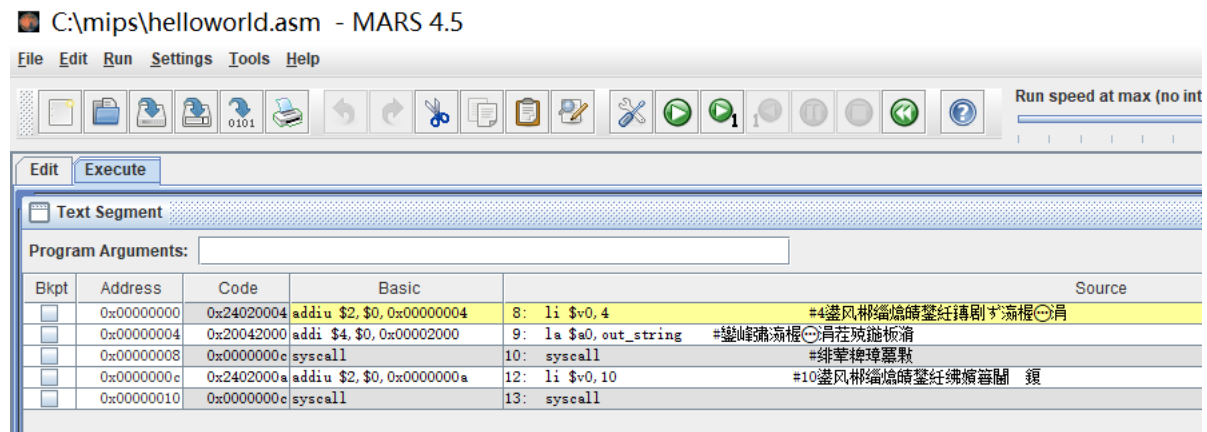
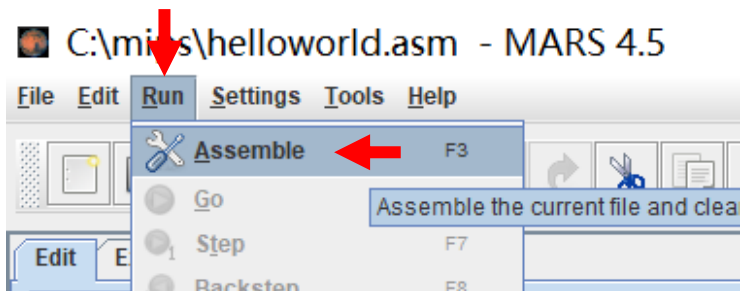
- 第一步：将MARS汇编仿真器的代码段的开始地址设置为0



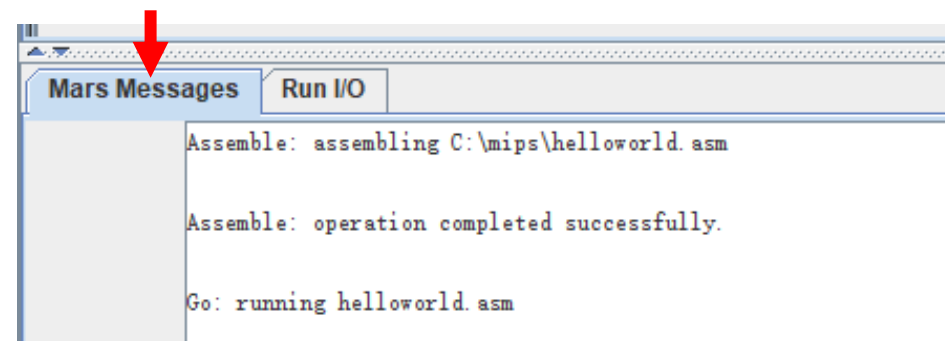
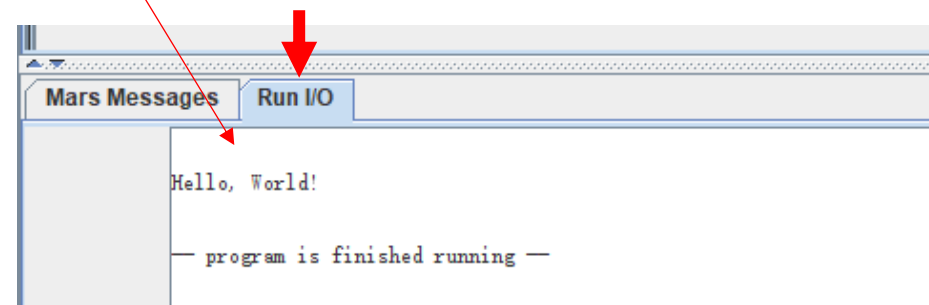
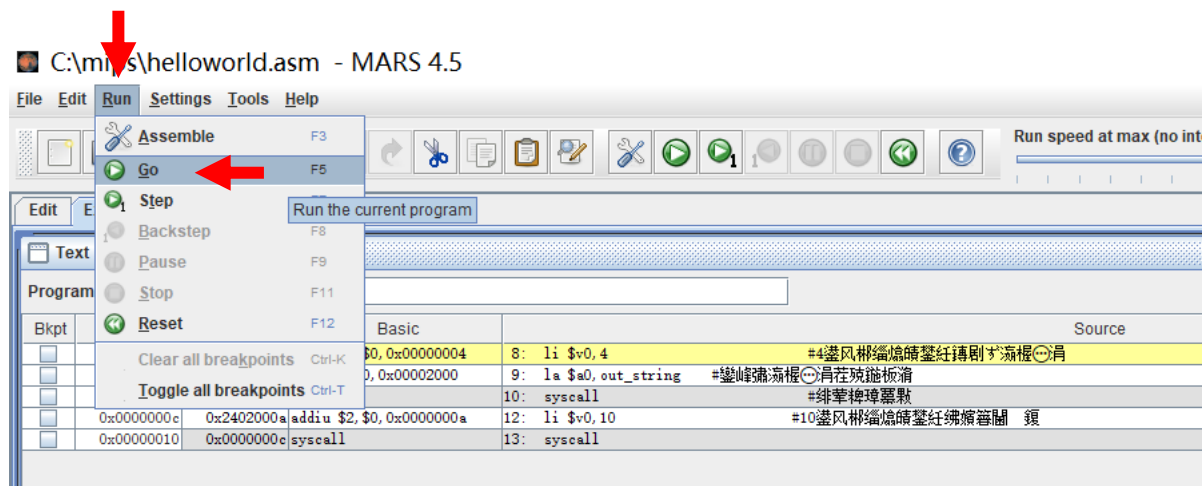
- 第二步：在MARS汇编仿真器中打开helloworld_mips.asm



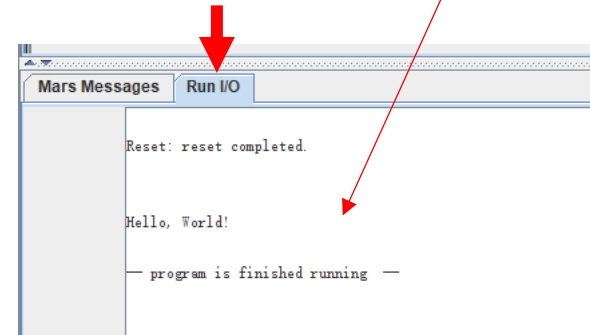
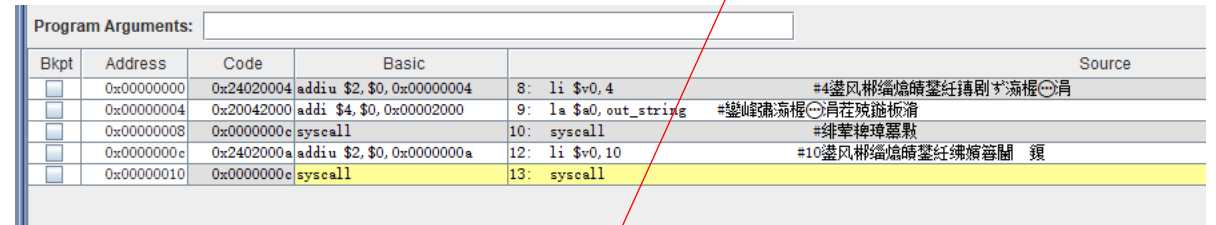
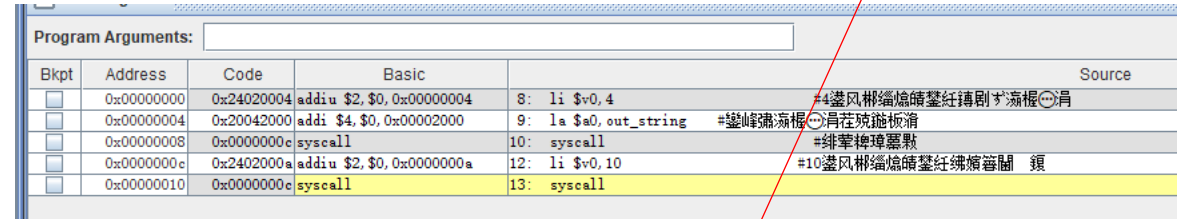
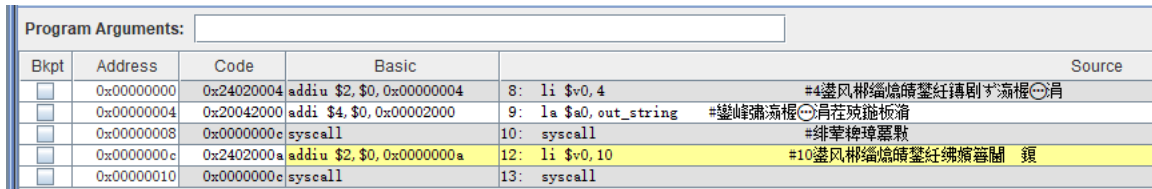
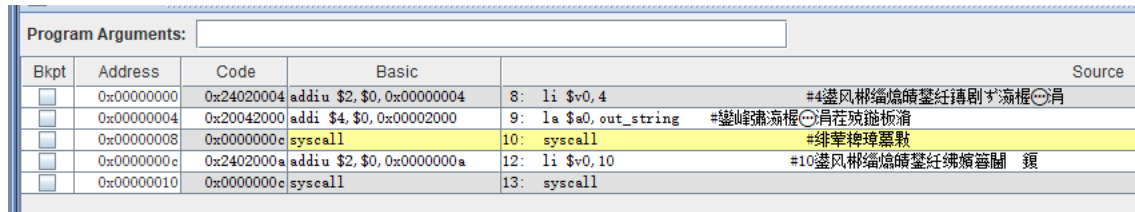
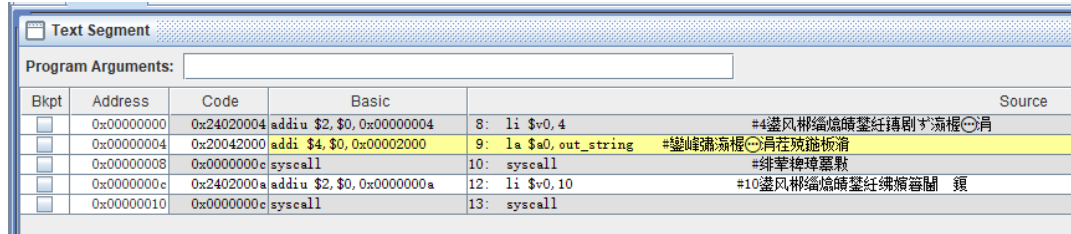
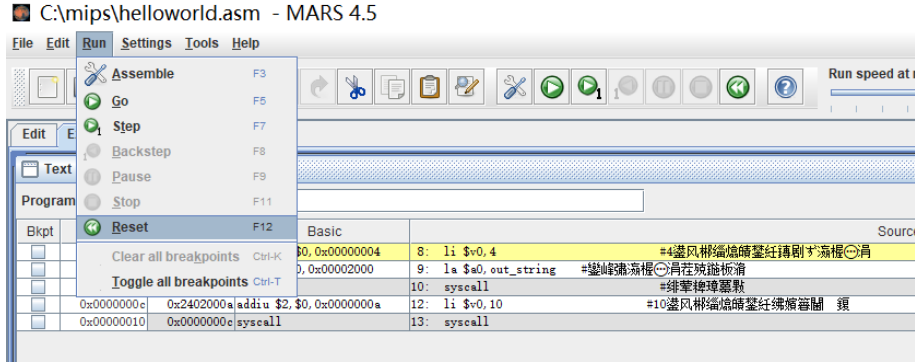
- 第三步：对helloworld_mips.asm源程序进行汇编（点击：**Assemble**）



- 第四步：运行汇编后的程序（点击：Go），在下面的界面中显示“Hello, World!”



- 第五步：也可以**单步**执行程序：首先点击：**Reset**，此时将第一条指令背景置黄色，表示从第一条指令开始执行；然后点击：**Step**，每点击一次Step，执行一条指令；直到所有指令执行完毕。



• helloworld_mips.asm源程序对应的MIPS指令和机器码：

机器码在存储器中的地址	指令的机器码	源程序对应的简化版MIPS指令	源代码
Address	Code	Basic	
0x00000000	0x24020004	addiu \$2, \$0, 0x00000004	8: li \$v0, 4
0x00000004	0x20042000	addi \$4, \$0, 0x00002000	9: la \$a0, out_string
0x00000008	0x0000000c	syscall	10: syscall
0x0000000c	0x2402000a	addiu \$2, \$0, 0x0000000a	12: li \$v0, 10
0x00000010	0x0000000c	syscall	13: syscall

0+4 -> \$2

4 -> \$v0

0+2000 -> \$4

2000 -> \$a0

0+0a -> \$2

10 -> \$v0

表5.7 MIPS的通用寄存器

寄存器#	助记符	释义
\$0	\$zero	固定值为0 硬件置位
\$1	\$at	汇编器保留, 临时变量
\$2~\$3	\$v0~\$v1	函数调用返回值
\$4~\$7	\$a0~\$a3	4个函数调用参数
\$8~\$15	\$t0~\$t7	暂存寄存器, 被调用者按需保存
\$16~\$23	\$s0~\$s7	save寄存器, 调用者按需保存
\$24~\$25	\$t8~\$t9	暂存寄存器, 同上
\$26~\$27	\$k0~\$k1	操作系统保留, 中断异常处理
\$28	\$gp	全局指针 (Global Pointer)
\$29	\$sp	堆栈指针 (Stack Pointer)
\$30	\$fp	帧指针 (Frame Pointer)
\$31	\$ra	函数返回地址 (Return Address)



```
源代码 (编写程序的原始文件)
)
.data
out_string: .asciiz "\nHello, World!\n"

.text
main:
    li $v0,4
    la $a0,out_string
    syscall

    li $v0,10
    syscall
```

- helloworld_mips.asm源程序数据段的内容:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)
0x00002000	0x6c65480a	0x202c6f6c	0x6c726f57	0x000a2164

0x0a, 0x48, 0x65, 0x6C, 0x6C, 0x6F, 0x2c, 0x20, 0x57, 0x6F, 0x72, 0x6C, 0x64, 0x21, 0x0a

\n H e l l o , 空格 W o r l d ! \n

```
.data
```

```
out_string: .asciiz "\nHello, World!\n"
```

• (3) 求累加和的MIPS汇编语言程序

• 求累加和程序如下 (**sum_mips.asm**, 位于\mips目录中) :

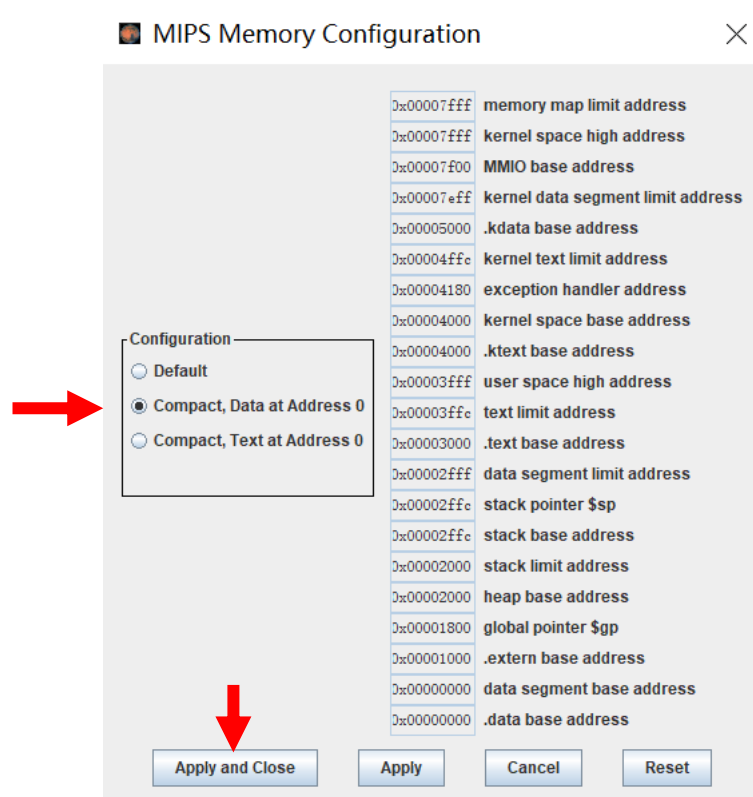
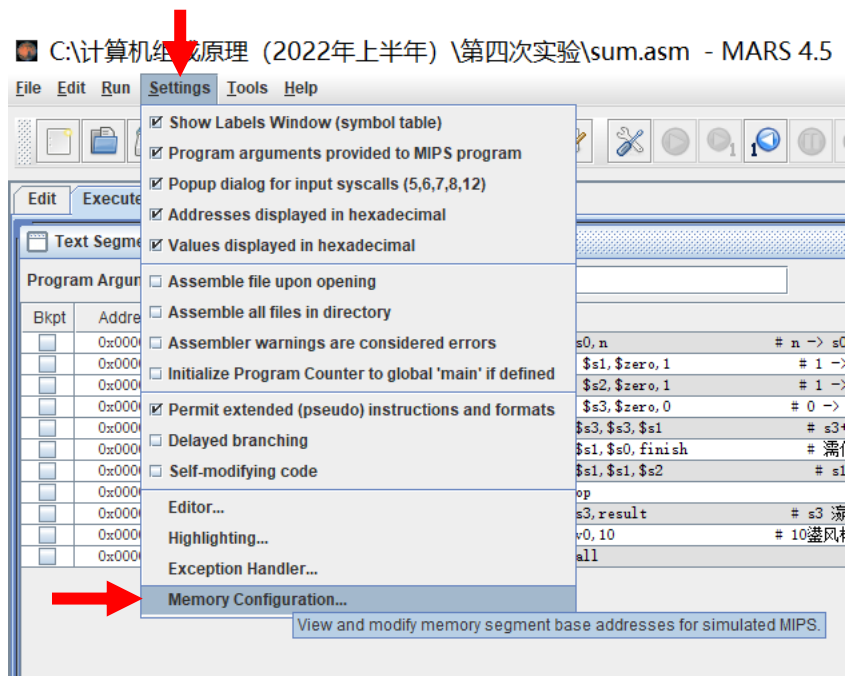
- $result = 1+2+\dots+n$
- $n=10=0ah$, $result=55=37h$
- $n=100=64h$, $result=5050=13bah$

```
sum_mips.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
#MIPS程序 求累加和                                sum_mips.asm
#求累加和: 1+2+.....+n, n的值为10 (可以改变), 累加和的结果存放到地址为0的数据存储器中

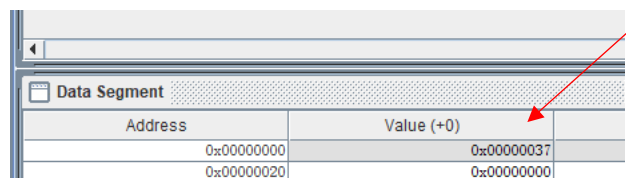
main:
    addi $s0,$zero,10          # n=10 -> s0
    addi $s1,$zero,1           # 1 -> s1
    addi $s2,$zero,1           # 1 -> s2
    addi $s3,$zero,0           # 0 -> s3
loop:
    add $s3,$s3,$s1             # s3+s1 -> s3
    beq $s1,$s0,finish          # 如果s1=s0, 则转finish
    add $s1,$s1,$s2             # s1+s2 -> s1
    j loop
finish:
    sw $s3,0($zero)             # s3 存到地址为0的存储单元中

    addi $v0,$zero,10           # 10号系统调用
    syscall                     # 程序退出
```

- 第一步：将MARS汇编仿真器的数据段的开始地址设置为0

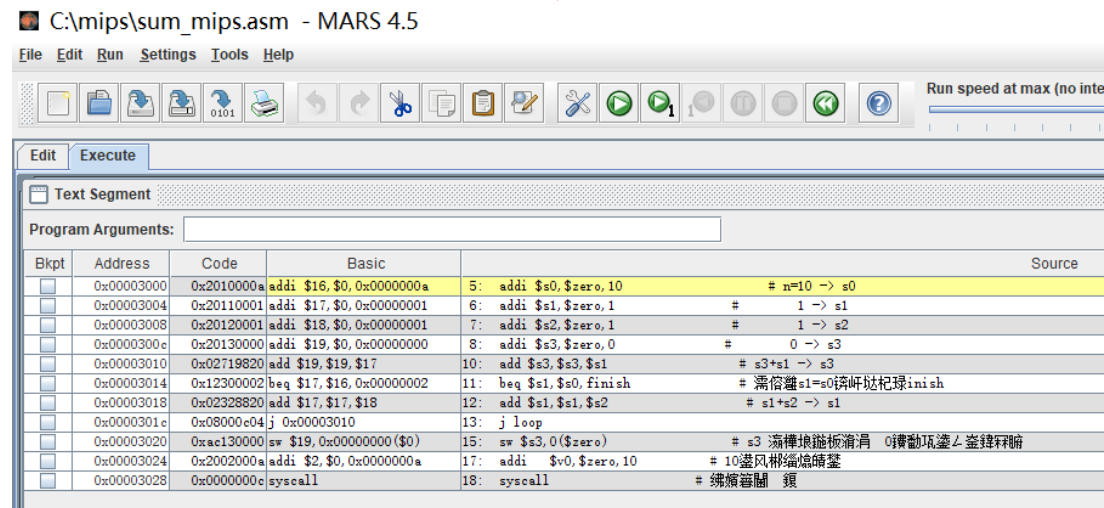


- 第二步：在MARS汇编仿真器中打开sum_mips.asm
- 第三步：对sum_mips.asm源程序进行汇编（点击： **Assemble**）
- 第四步：运行汇编后的程序（点击： **Go**）
- 第五步：观看数据段中的内容（地址=0）是否为37h=1+2+...+10=55？



Address	Value (+0)
0x00000000	0x00000037
0x00000020	0x00000000

result=55=37h



C:\mips\sum_mips.asm - MARS 4.5

File Edit Run Settings Tools Help

Run speed at max (no inte

Edit Execute

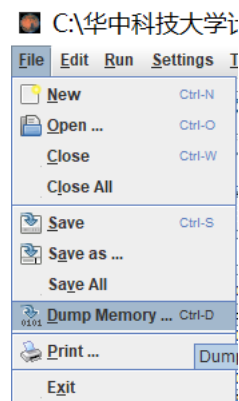
Text Segment

Program Arguments:

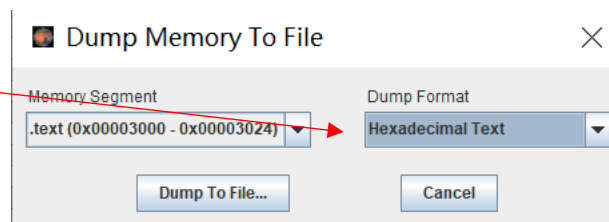
Bkpt	Address	Code	Basic	Source
	0x00003000	0x2010000a	addi \$16,\$0,0x0000000a	5: addi \$s0,\$zero,10 # n=10 -> s0
	0x00003004	0x20110001	addi \$17,\$0,0x00000001	6: addi \$s1,\$zero,1 # 1 -> s1
	0x00003008	0x20120001	addi \$18,\$0,0x00000001	7: addi \$s2,\$zero,1 # 1 -> s2
	0x0000300c	0x20130000	addi \$19,\$0,0x00000000	8: addi \$s3,\$zero,0 # 0 -> s3
	0x00003010	0x02719820	add \$19,\$19,\$17	10: add \$s3,\$s3,\$s1 # s3+s1 -> s3
	0x00003014	0x12300002	beq \$17,\$16,0x00000002	11: beq \$s1,\$s0,finish # 滿俗雖s1=s0時軒放紀球inish
	0x00003018	0x02328820	add \$17,\$17,\$18	12: add \$s1,\$s1,\$s2 # s1+s2 -> s1
	0x0000301c	0x08000e04	j 0x00003010	13: j loop
	0x00003020	0xac130000	sw \$19,0x00000000(\$0)	15: sw \$s3,0(\$zero) # s3 滿標浪拋板滑滑 0寶勳瓦漆△盜鍵釋
	0x00003024	0x2002000a	addi \$2,\$0,0x0000000a	17: addi \$v0,\$zero,10 # 10滿風彬編煌崎壁
	0x00003028	0x0000000c	syscall	18: syscall # 綁嬌簪闊 銀

- 第六步：导出汇编后的机器码：

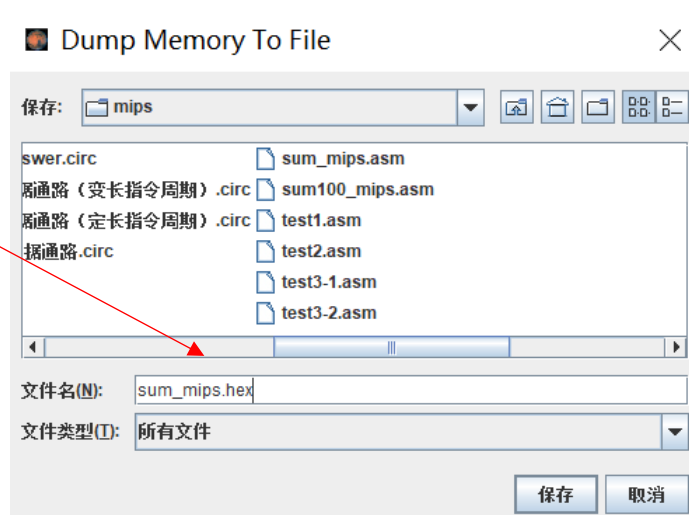
- “File” -> “Dump Memory”



- 保存为十六进制文本格式



- 保存到sum_mips.hex文件中



A screenshot of the 'sum_mips.hex' file content. It shows a list of hexadecimal values: 2010000a, 20110001, 20120001, 20130000, 02719820, 12300002, 02328820, 08000c04, ac130000, 2002000a, and 0000000c. A red arrow points from the 'sum_mips.asm' file in the previous dialog box to this list.

机器码

sum_mips.hex

文件(F) 编辑(E)

2010000a

20110001

20120001

20130000

02719820

12300002

02328820

08000c04

ac130000

2002000a

0000000c

C:\mips\sum_mips.asm - MARS 4.5

File Edit Run Settings Tools Help



Edit

Execute

Text Segment

Program Arguments:

机器码

Bkpt	Address	Code	Basic
☐	0x00003000	0x2010000a addi \$16,\$0,0x0000000a	5: addi \$s0,\$zero,10
☐	0x00003004	0x20110001 addi \$17,\$0,0x00000001	6: addi \$s1,\$zero,1
☐	0x00003008	0x20120001 addi \$18,\$0,0x00000001	7: addi \$s2,\$zero,1
☐	0x0000300c	0x20130000 addi \$19,\$0,0x00000000	8: addi \$s3,\$zero,0
☐	0x00003010	0x02719820 add \$19,\$19,\$17	10: add \$s3,\$s3,\$s1
☐	0x00003014	0x12300002 beq \$17,\$16,0x00000002	11: beq \$s1,\$s0,finish
☐	0x00003018	0x02328820 add \$17,\$17,\$18	12: add \$s1,\$s1,\$s2
☐	0x0000301c	0x08000c04 j 0x00003010	13: j loop
☐	0x00003020	0xac130000 sw \$19,0x00000000(\$0)	15: sw \$s3,0(\$zero)
☐	0x00003024	0x2002000a addi \$2,\$0,0x0000000a	17: addi \$v0,\$zero,10
☐	0x00003028	0x0000000c syscall	18: syscall

• (4) 计算费波那契数列的MIPS汇编语言程序

- 斐波那契数列 (Fibonacci sequence)，又称黄金分割数列，指的是这样一个数列：0、1、1、2、3、5、8、13、21、34、55、……；在数学上，斐波那契数列通常以递推的方法定义： $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$ ($n \geq 2$, $n \in \mathbb{N}^*$)。
- 计算n+1个费波那契数列的程序如下（**fib_mips.asm**，位于\mips目录中），n在程序中设置，计算好的费波那契数列存放在数据存储器地址为0、4、8、…、4*n的存储单元中。
- 请同学们在MARS 4.5汇编仿真器中运行该程序，并将该程序的机器码保存到**fib_mips.hex**文件中。

```
fib_mips.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
#MIPS程序 计算费波那契数列 fib_mips.asm
# n=10 (可以改变)，计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、……) 保存在地址为0、4、8、12、...的数据存储器中

main:
    addi $s2,$zero,10          # $s2=n=10
    addi $s1,$zero,0           # $s1=0
    sw $s1,0($zero)            # 0 -> (0)

    addi $s1,$zero,1           # $s1=1
    sw $s1,4($zero)            # 1 -> (4)

    sw $s1,8($zero)            # 1 -> (8)

    addi $s3,$zero,2           # $s3 = 2
    addi $s4,$zero,1           # $s4 = 1
    addi $s5,$zero,1           # $s5 = 1

loop:
    add $s6,$s4,$s5            # $s4+$s5 -> $s6
    addi $s4,$s5,0             # $s5 -> $s4
    addi $s5,$s6,0             # $s6 -> $s5
    addi $s3,$s3,1             # $s3+1 -> $s3
    add $s7,$s3,$s3            # $s3+$s3 -> $s7
    add $s0,$s7,$s7            # $s7+$s7 -> $s0
    sw $s6,0($s0)              # $s6 -> 0($s0)
    beq $s2,$s3,end            # if $s2 = $s3 goto end
    j loop                     # goto loop

end:
    addi $v0,$zero,10          # 10号系统调用
    syscall                    # 程序退出
```

```
*fib_mips.hex
文件(F) 编辑(E)
2012000a
20110000
ac110000
20110001
ac110004
ac110008
20130002
20140001
20150001
0295b020
22b40000
22d50000
22730001
0273b820
02f78020
ae160000
12530001
08000c09
2002000a
0000000c
```

运算结果

0x00000000	0x00000000	0x00000001	0x00000001	0x00000002	0x00000003	0x00000005	0x00000008	0x0000000d
0x00000020	0x00000015	0x00000022	0x00000037	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

fib_mips.asm - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

#MIPS程序 计算费波那契数列 fib_mips.asm

n=10 (可以改变), 计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、.....) 保存在地址为0、4、8、12、...的数据存储器中

main:

addi \$s2,\$zero,10 # \$s2=n=10

addi \$s1,\$zero,0 # \$s1=0
sw \$s1,0(\$zero) # 0 -> (0)

addi \$s1,\$zero,1 # \$s1=1
sw \$s1,4(\$zero) # 1 -> (4)

sw \$s1,8(\$zero) # 1 -> (8)

addi \$s3,\$zero,2 # \$s3 = 2
addi \$s4,\$zero,1 # \$s4 = 1
addi \$s5,\$zero,1 # \$s5 = 1

loop:

add \$s6,\$s4,\$s5 # \$s4+\$s5 -> \$s6
addi \$s4,\$s5,0 # \$s5 -> \$s4
addi \$s5,\$s6,0 # \$s6 -> \$s5
addi \$s3,\$s3,1 # \$s3+1 -> \$s3
add \$s7,\$s3,\$s3 # \$s3+\$s3 -> \$s7
add \$s0,\$s7,\$s7 # \$s7+\$s7 -> \$s0
sw \$s6,0(\$s0) # \$s6 -> 0(\$s0)
beq \$s2,\$s3,end # if \$s2 = \$s3 goto end
j loop # goto loop

end:

addi \$v0,\$zero,10 # 10号系统调用
syscall # 程序退出

- (5) 要求

- 请同学们修改“`sum_mips.asm`”程序，计算 $1+2+3+\cdots+100=5050=13bah$ （修改后的程序命名为：`sum100_mips.asm`），然后在MARS 4.5汇编仿真器中运行，观看运算结果是否正确？并将该程序的机器码保存到`sum100_mips.hex`文件中。

2、在Logisim上运行MIPS程序

- （1）Logisim实现的单周期MIPS处理器能够运行**24条指令**（核心指令8条+基础指令16条）

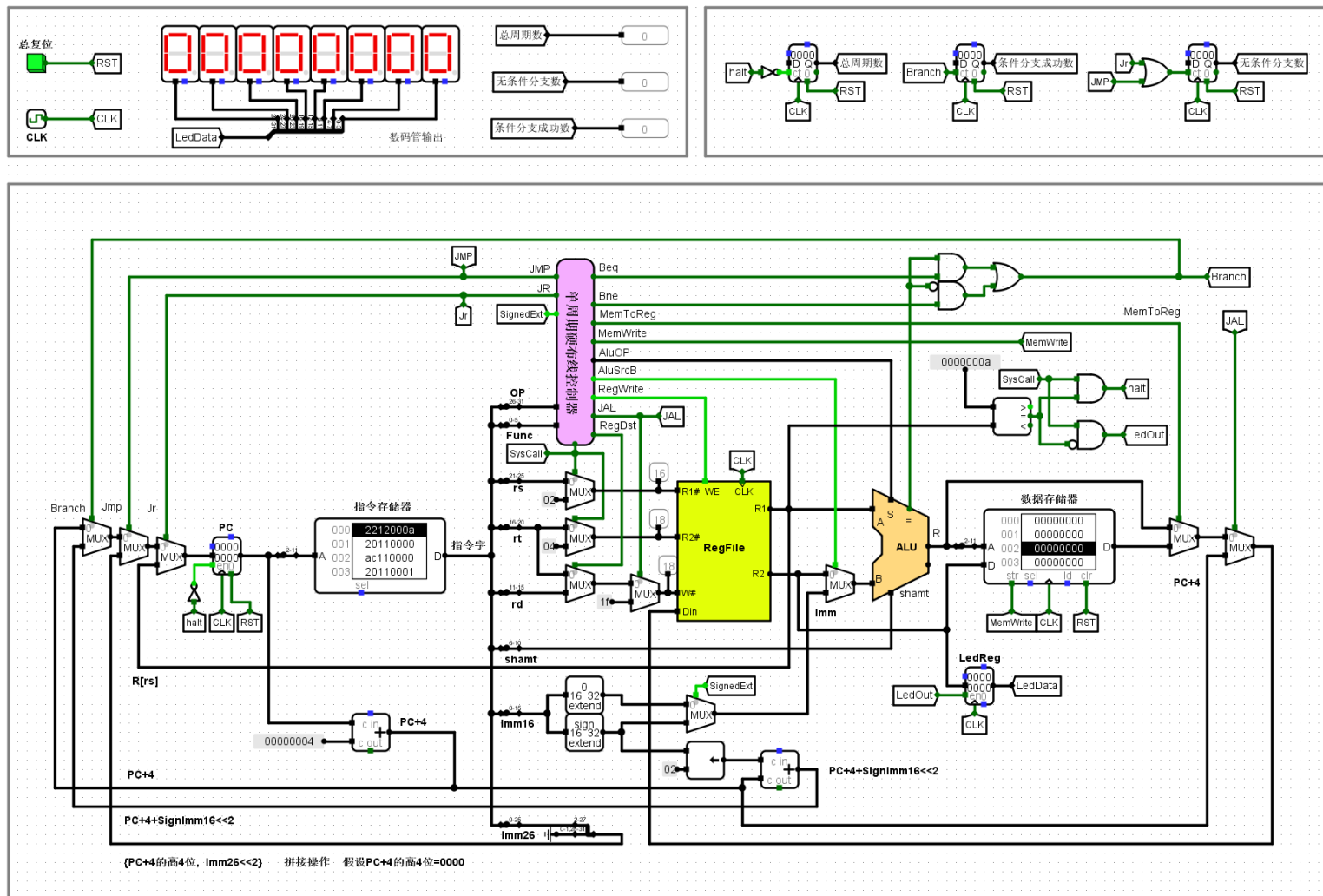
核心指令（8条）

序号	MIPS指令	RTL描述	功能说明
1	add \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] + R[rt]$	寄存器加法指令，不考虑溢出
2	slt rd,rs,rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于置1指令，有符号数比较
3	addi rt,rs,imm	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	立即数加法指令，不考虑溢出
4	lw rt,imm(rs)	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$	取数指令
5	sw rt,imm(rs)	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$	存数指令
6	beq rs,rt,imm	$\text{if}(R[rs] == R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(imm) \ll 2$	条件分支指令：如果rs == rt，则跳转
7	bne rs,rt,imm	$\text{if}(R[rs] != R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(imm) \ll 2$	条件分支指令：如果rs != rt，则跳转
8	syscall	系统调用指令，用于停机	系统调用指令，用于停机

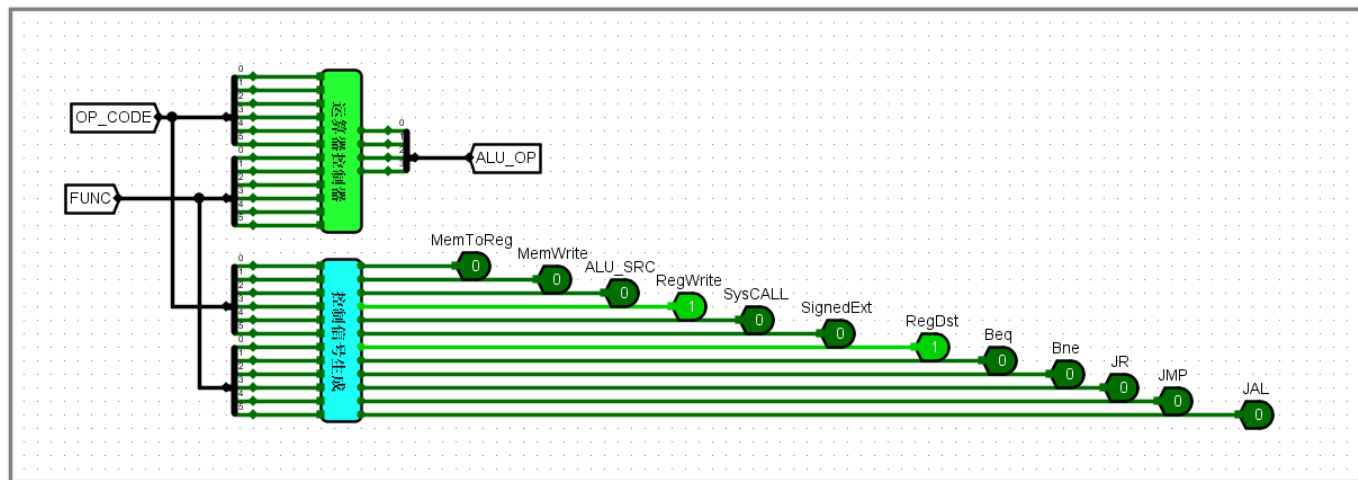
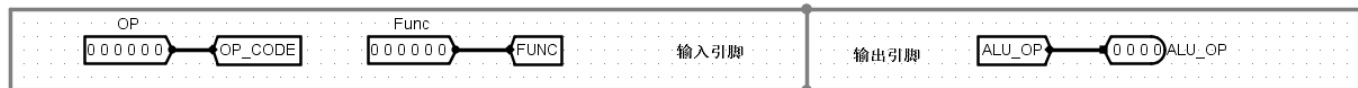
基础指令（16条）

序号	MIPS指令	RTL描述	功能说明
1	addu \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] + R[rt]$	无符号寄存器加法指令
2	addiu \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	无符号立即数加法指令
3	and \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] \& R[rt]$	逻辑与指令
4	andi \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] \& \{16\text{个}0, imm\}$	立即数逻辑与指令
5	sll \$rd,\$rt,imm	$R[rd] \leftarrow R[rt] \ll \text{shamt}$	逻辑左移
6	srl \$rd,\$rt,imm	$R[rd] \leftarrow R[rt] \gg \text{shamt}$	逻辑右移
7	sra \$rd,\$rt,imm	$R[rd] \leftarrow R[rt] \gg \text{shamt}$	算术右移
8	sub \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] - R[rt]$	寄存器减法指令
9	or \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] R[rt]$	逻辑或指令
10	ori \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] \{16\text{个}0, imm\}$	立即数逻辑或指令
11	nor \$rd,\$rs,\$rt	$R[rd] \leftarrow \neg(R[rs] R[rt])$	逻辑或非指令
12	sltu \$rd,\$rs,\$rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于置1指令，无符号数比较
13	slti \$rt,\$rs,imm	$R[rd] \leftarrow (R[rs] < \text{SignExt}(imm)) ? 1:0$	小于置1指令，立即数比较
14	j address	$PC \leftarrow \{(PC+4)_{31:28}, address<<2\}$	无条件分支指令
15	jal address	$R[31] \leftarrow PC+4 \quad PC \leftarrow \{(PC+4)_{31:28}, address<<2\}$	子程序调用指令
16	jr \$rs	$PC \leftarrow R[rs]$	寄存器跳转指令（jr \$ra 相当于主程序返回指令）

- (2) Logisim实现的单周期MIPS处理器 (**单周期MIPS处理器.circ**)

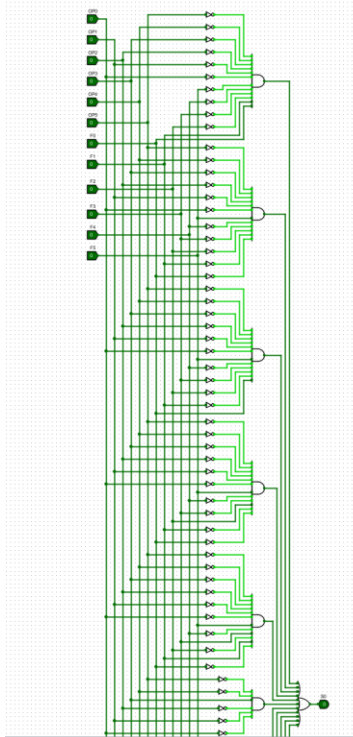


单周期 **MIPS** 处理器数据通路 (**24条指令**)

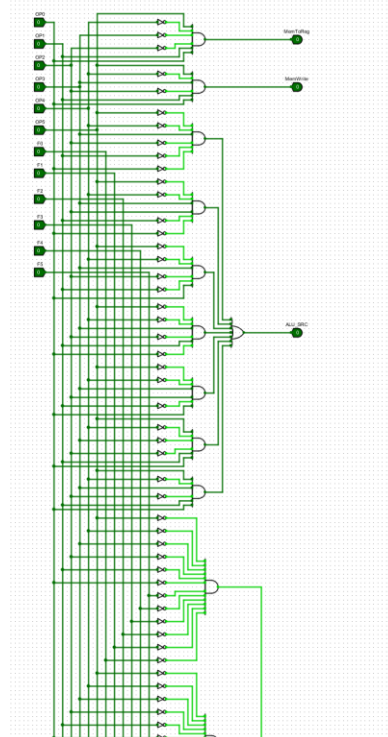


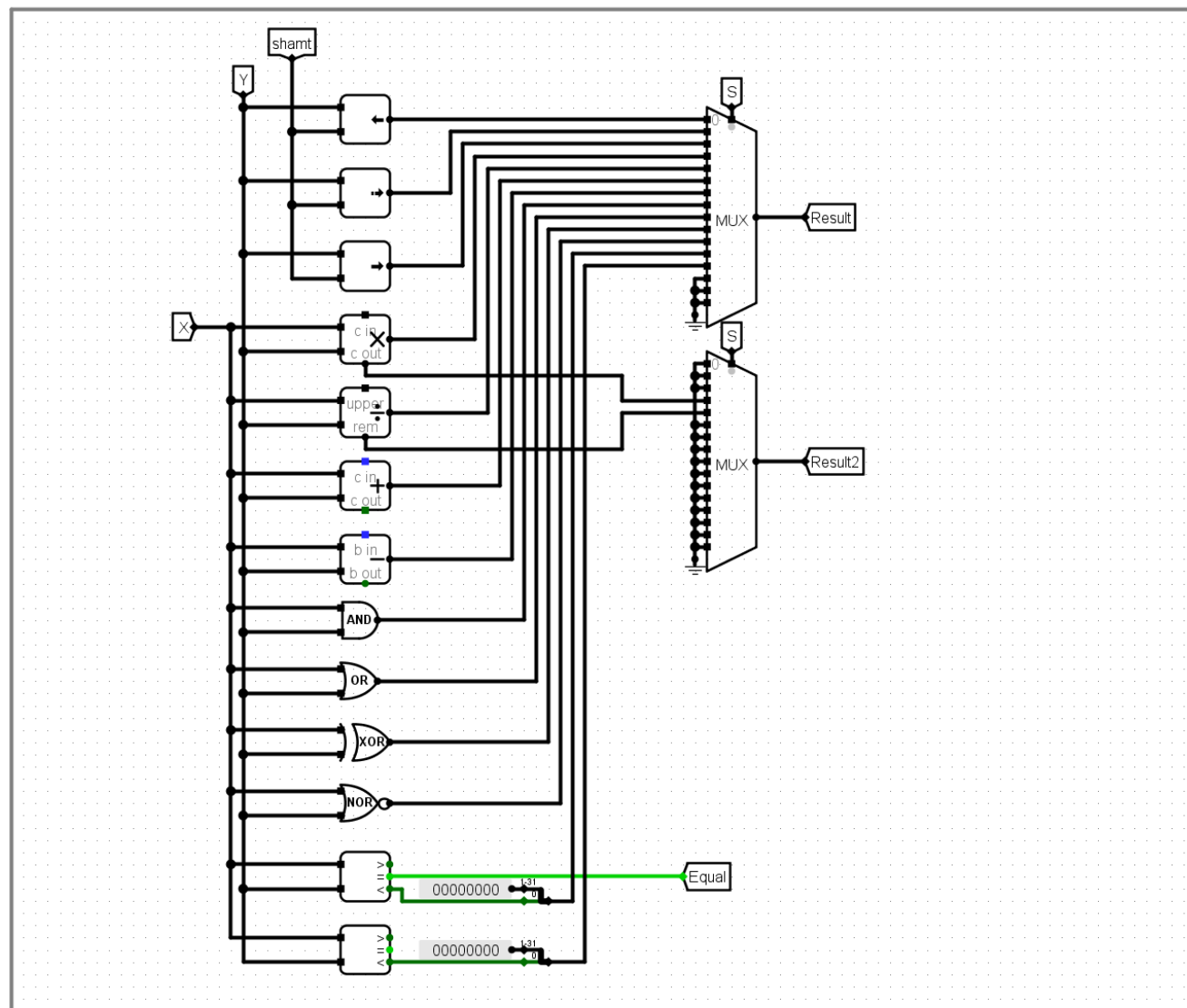
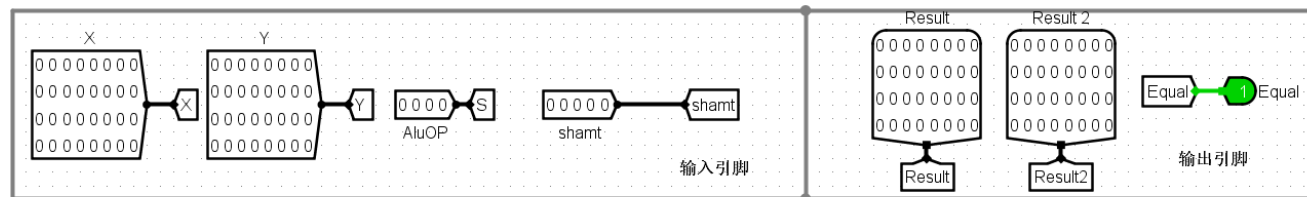
单周期硬布线控制器

运算器控制器

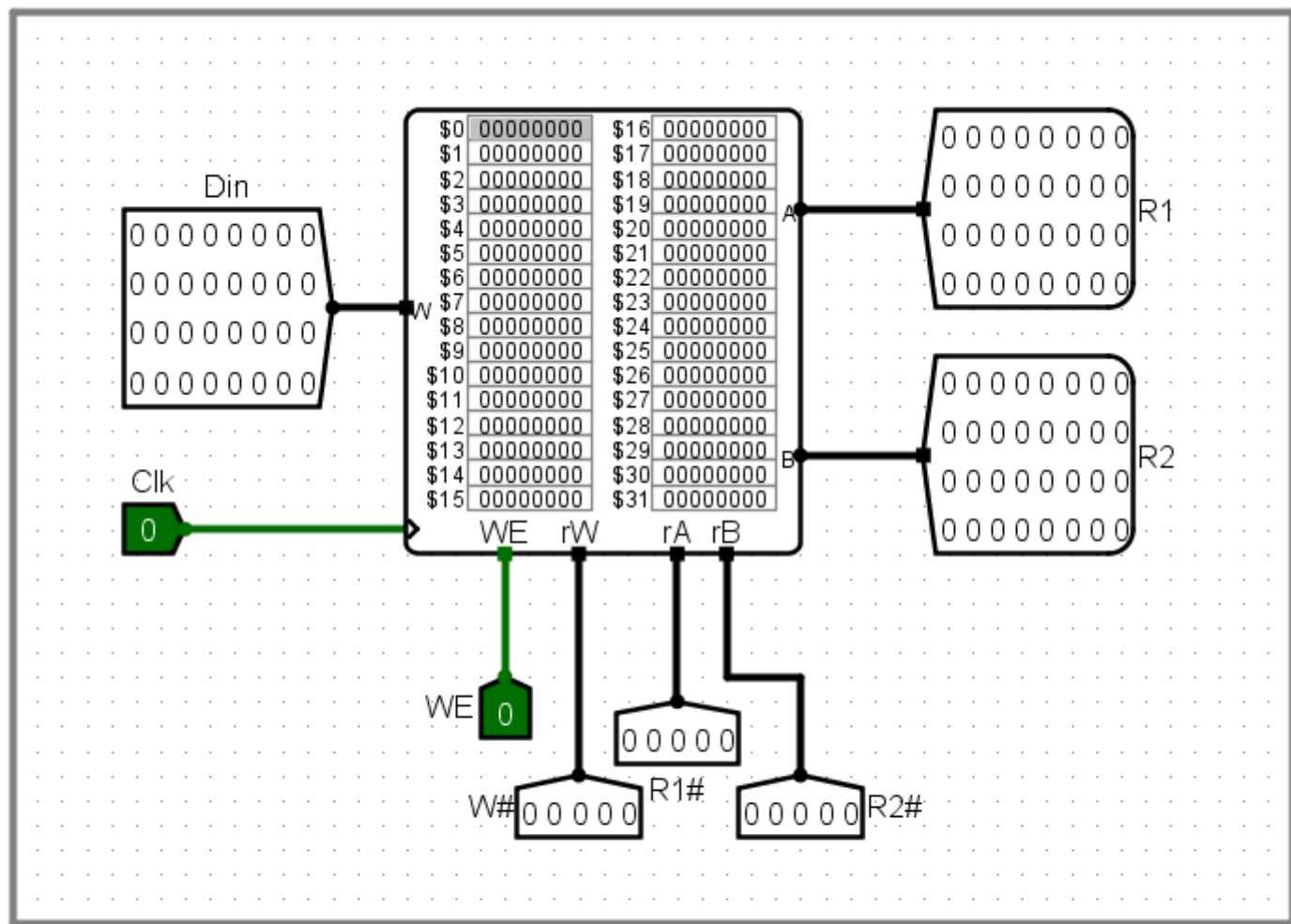


控制信号生成





算术逻辑单元ALU

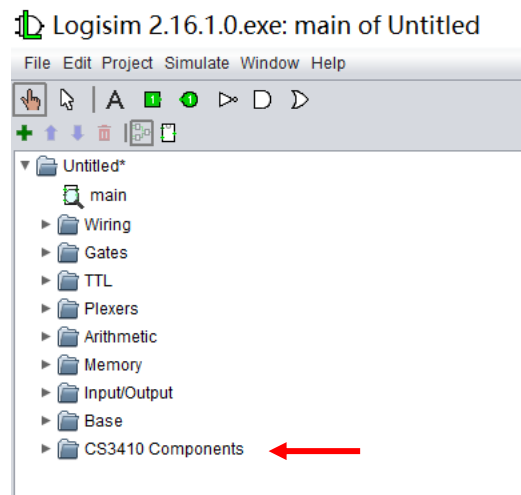
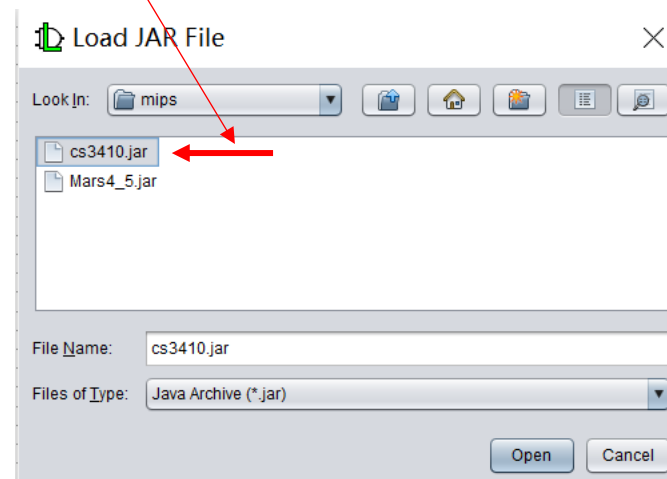
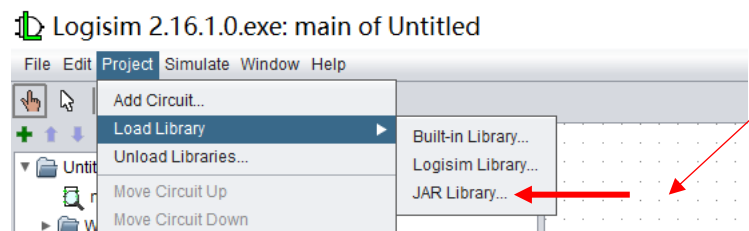


寄存器堆 **RegFile**

• (3) 在Logisim实现的单周期MIPS处理器上运行求累加和程序

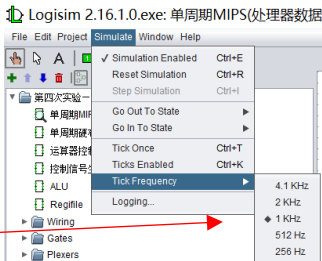
- 第一步：修改前面导出的机器码文件sum_mips.hex，增加：v2.0 raw
- 第二步：运行Logisim，在Logisim中增加寄存器文件库“cs3410.jar”（位于\mips目录中）
- 第三步：在Logisim中打开设计文件“单周期MIPS处理器.circ”（位于\mips目录中）

```
sum_mips.hex - 记事本
文件(F) 编辑(E) 格式(O)
v2.0 raw
2010000a
20110001
20120001
20130000
02719820
12300002
02328820
08000c04
ac130000
2002000a
0000000c
```



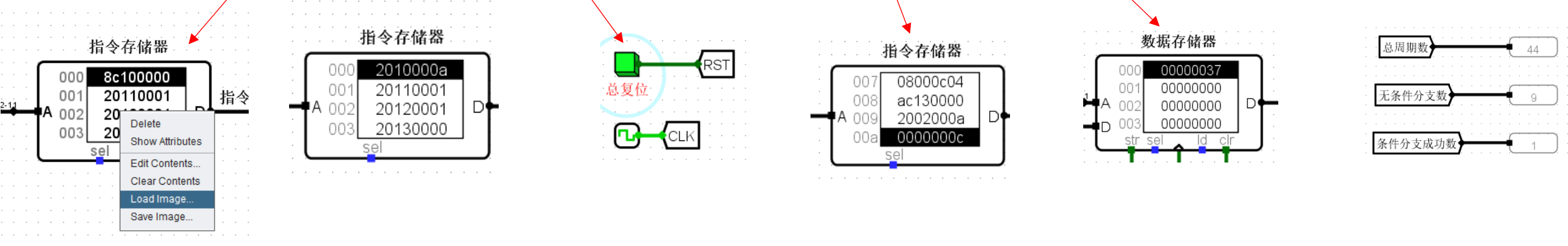
第四步：在单周期MIPS处理器数据通路的指令存储器中装入机器码文件sum_mips.hex

第五步：设置时钟频率=1KHz，按Ctrl+K启动时钟，程序开始执行



第六步：按总复位，程序重新执行；指令存储器中的内容停止滚动后，表示程序执行结束；再按Ctrl+K停止时钟

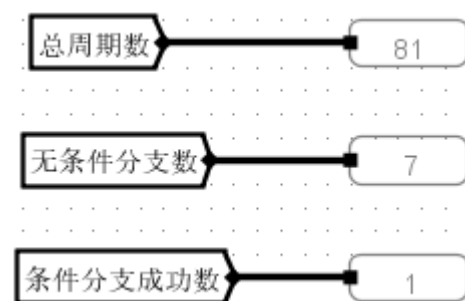
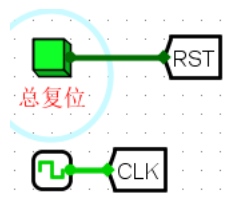
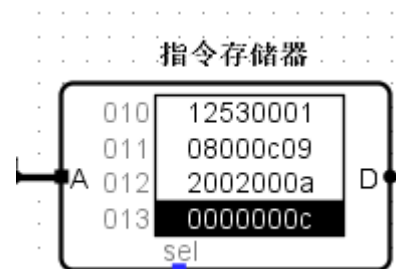
第七步：此时观察数据存储第0号单元的值（累加和，正确值为37h=55），以及程序运行的总周期数、无条件分支数、条件分支成功数



• (4) 在Logisim实现的单周期MIPS处理器上运行**计算费波那契数列**程序

- 第一步：修改前面导出的机器码文件fib_mips.hex，增加：**v2.0 raw**
- 第二步：在单周期MIPS处理器数据通路的指令存储器中装入机器码文件**fib_mips.hex**
- 第三步：设置时钟频率=1KHz，按**Ctrl+K**启动时钟，程序开始执行
- 第四步：按**总复位**，程序重新执行；指令存储器中的内容停止滚动后，表示程序执行结束；再按**Ctrl+K**停止时钟
- 第五步：此时观察数据存储器中的内容（0号单元开始的11个单元），以及程序运行的总周期数、无条件分支数、条件分支成功数

fib_mips.hex
文件(F) 编辑(E)
v2.0 raw
2012000a
20110000
ac110000
20110001
ac110004
ac110008
20130002
20140001
20150001
0295b020
22b40000
22d50000
22730001
0273b820
02f78020
ae160000
12530001
08000c09
2002000a
0000000c



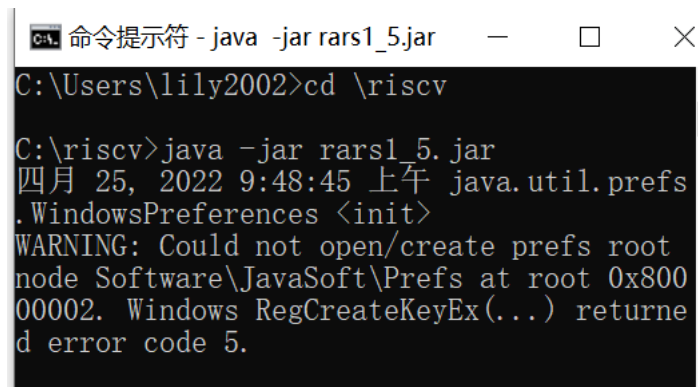
000 00000000 00000001 00000001 00000002 00000003 00000005 00000008 0000000d 00000015 00000022 00000037 00000000 00000000 00000000 00000000

- (5) 要求

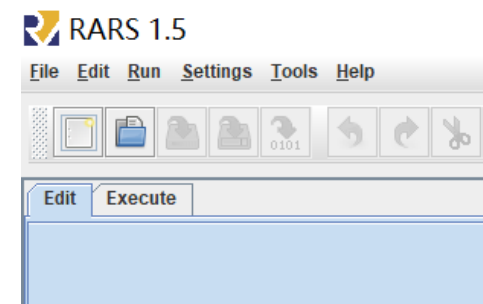
- 请同学们在Logisim实现的单周期MIPS处理器上，运行计算 $1+2+3+\cdots+100=5050=13bah$ 的机器语言程序（`sum100_mips.hex`），观看数据存储器0号单元的值是不是13bah？程序运行的总周期数、无条件分支数、条件分支成功数分别是多少？

3、RISC-V汇编语言程序的运行

- (1) 运行RISC-V汇编仿真器
 - 运行RISC-V汇编仿真器 **rars1_5.jar** (位于\risvcv目录中)
 - 打开“命令提示符”，执行：
 - **cd D:**
 - **cd \risvcv**
 - **java -jar rars1_5.jar**



```
命令提示符 - java -jar rars1_5.jar
C:\Users\lily2002>cd \risvcv
C:\risvcv>java -jar rars1_5.jar
四月 25, 2022 9:48:45 上午 java.util.prefs
.WindowsPreferences <init>
WARNING: Could not open/create prefs root
node Software\JavaSoft\Prefs at root 0x800
00002. Windows RegCreateKeyEx(...) returne
d error code 5.
```



Edit Execute

About Rars

RARS 1.5 Copyright 2003-2019

Pete Sanderson and Kenneth Vollmar

RARS is the RISC-V Assembler and Runtime Simulator.

Toolbar and menu icons are from:

- * Tango Desktop Project (tango.freedesktop.org),
- * glyFX (www.glyfx.com) Common Toolbar Set,
- * KDE-Look (www.kde-look.org) crystalline-blue-0.1,
- * Icon-King (www.icon-king.com) Nuvola 1.0.

确定

Messages

Run I/O

Clear

RARS 1.5

RISC-V汇编仿真器

Registers Floating Point Control and Status

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x0002ffe0
fp	3	0x00018000
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000
s2	10	0x00000000
s3	11	0x00000000
s4	12	0x00000000
s5	13	0x00000000
s6	14	0x00000000
s7	15	0x00000000
s8	16	0x00000000
s9	17	0x00000000
s10	18	0x00000000
s11	19	0x00000000
t3	20	0x00000000
t4	21	0x00000000
t5	22	0x00000000
t6	23	0x00000000
t7	24	0x00000000
t8	25	0x00000000
t9	26	0x00000000
s12	27	0x00000000
s13	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x00000000

- RISC-V的寄存器：32个寄存器，以及pc，均为32位

表5.12 RISC-V的通用寄存器

编号	助记符	英文全称	功能描述
x0	zero	zero	恒零值，可用0号寄存器参与的加法指令实现MOV指令
x1	ra	Return Address	返回地址
x2	sp	Stack Pointer	栈指针，指向栈顶
x3	gp	Global Pointer	全局指针
x4	tp	Thread Pointer	线程寄存器
x5 ~ x7	t0 ~ t2	Temporaries	临时变量，调用者保存寄存器
x8	s0/fp	Saved Register/Frame Pointer	通用寄存器，被调用者保存寄存器，在子程序中使用时必须先压栈保存原值，使用后应出栈恢复原值
x9	s1	Saved Registers	通用寄存器，被调用者保存寄存器
x10 ~ x11	a0 ~ a1	Arguments/Return values	用于存储子程序参数或返回值
x12 ~ x17	a2 ~ a7	Arguments	用于存储子程序参数
x18 ~ x27	s2 ~ s11	Saved Registers	通用寄存器，被调用者保存寄存器
x28 ~ x31	t3 ~ t6	Temporaries	临时变量

Registers			Floating Point			Control and Status		
Name			Number					
zero			0					
ra			1					
sp			2					
gp			3					
tp			4					
t0			5					
t1			6					
t2			7					
s0			8					
s1			9					
a0			10					
a1			11					
a2			12					
a3			13					
a4			14					
a5			15					
a6			16					
a7			17					
s2			18					
s3			19					
s4			20					
s5			21					
s6			22					
s7			23					
s8			24					
s9			25					
s10			26					
s11			27					
t3			28					
t4			29					
t5			30					
t6			31					
pc								

• RISC-V核心指令集RV32I（共47条指令）中最常用的9条指令：

- ① add rd,rs1,rs2 ; rs1+rs2->rd
- ② slt rd,rs1,rs2 ; 带符号数比较指令 if rs1<rs2 1->rd else 0->rd
- ③ sltu rd,rs1,rs2 ; 无符号数比较指令 if rs1<rs2 1->rd else 0->rd
- ④ ori rd,rs1,imm12 ; rs1 或 imm12 -> rd
- ⑤ lw rd rs1,imm12 ; M[rs1+imm12] -> rd
- ⑥ lui rd,imm20 ; imm20 -> rd
- ⑦ sw rs2,rs1,imm12 ; rs2 -> M[rs1+imm12]
- ⑧ beq rs1,rs2,imm12 ; if rs1=rs2 then goto imm12
- ⑨ jal rd,imm20 ; goto imm20 PC+4 -> R[rd]

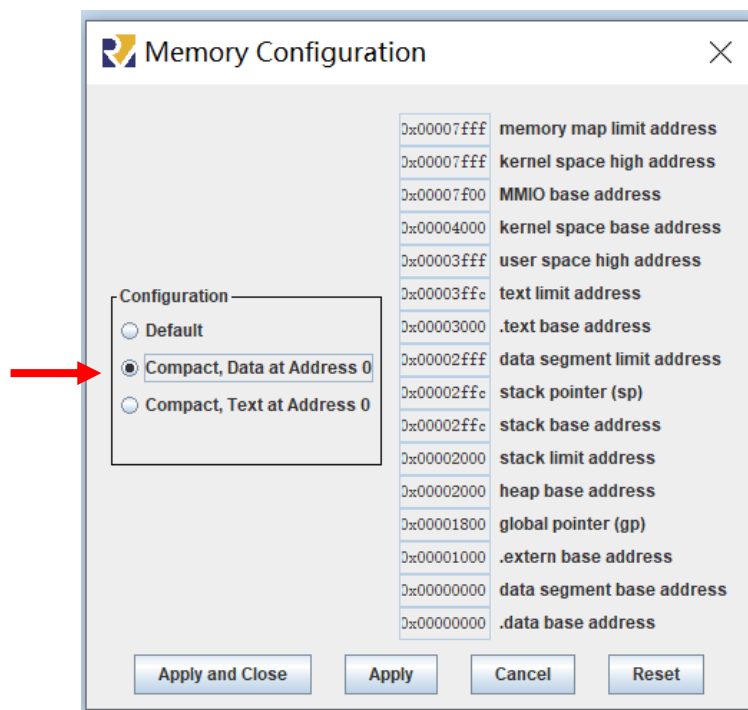
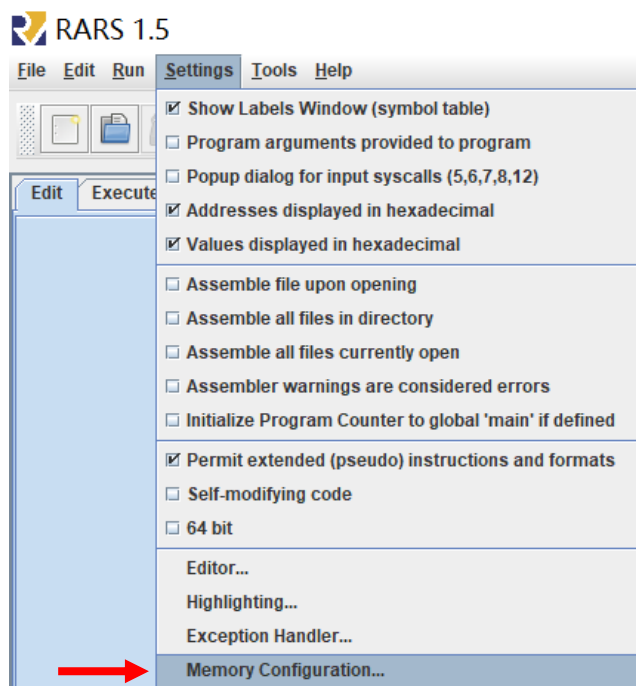
- (2) 运行RISC-V求累加和程序

- 利用前面的RISC-V的9条指令，编写一个计算 $1+2+\dots+n$ 的累加和程序，结果保存到数据存储器地址为0的存储单元中；该程序的汇编代码如下（`sum_riscv.asm`，位于\ riscv目录中）：

```
sum_riscv.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
# RISC-V计算累加和程序      sum_riscv.asm      result=1+2+...+n
# 结果result存放在地址为0的数据存储器中

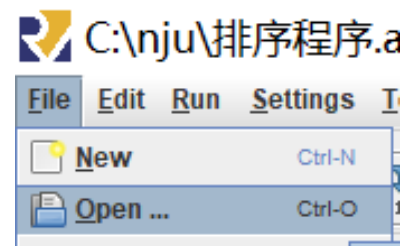
main:
    ori a3,zero,10           # a3=10=n
    ori a5, zero, 1          # a5=1
    ori a2, zero, 1          # a2=1
    ori a4, zero, 0          # a4=0
loop:
    add a4, a4, a5            # a4+a5 -> a4
    beq a5, a3, finish       # 判断a5=a3?      若a5=a3=n, 则跳出循环
    add a5, a5, a2            # a5+a2 -> a5
    jal zero, loop           # 无条件跳转到loop执行
finish:
    sw a4, 0(zero)           # a4 -> (0)      将累加结果result保存到存储器中
end:
    jal zero, end            # 无条件跳转到end执行
```

- 第一步：将RARS汇编仿真器的数据段的开始地址设置为0



- 第二步：在RARS中打开“sum_riscv.asm”

- File -> Open

A screenshot of the RARS 1.5 editor window. The title bar shows 'C:\riscv\sum_riscv.asm - RARS 1.5'. The menu bar includes File, Edit, Run, Settings, Tools, and Help. The toolbar contains various icons for file operations and execution. The editor window shows the assembly code for 'sum_riscv.asm'. The code is as follows:

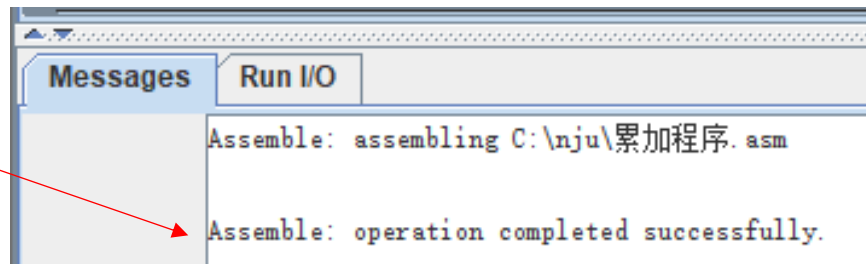
```
1 # RISC-V计算累加和程序          sum_riscv.asm          result=1+2+...+n
2 # 结果result存放在地址为0的数据存储器中
3
4 main:
5     ori a3, zero, 10                # a3=10=n
6     ori a5, zero, 1                # a5=1
7     ori a2, zero, 1                # a2=1
8     ori a4, zero, 0                # a4=0
9 loop:
10    add a4, a4, a5                  # a4+a5 -> a4
11    beq a5, a3, finish              # 判断a5=a3?          若a5=a3=n, 则跳出循环
12    add a5, a5, a2                  # a5+a2 -> a5
13    jal zero, loop                  # 无条件跳转到loop执行
14 finish:
15    sw a4, 0(zero)                  # a4 -> (0)          将累加结果result保存到存储器中
16 end:
17    jal zero, end                  # 无条件跳转到end执行
18
```

- 第三步：在RARS中汇编“sum_riscv.asm”

- Run -> **Assemble**



- 左下角窗口中显示汇编成功!

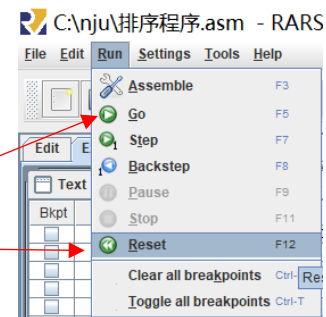


- 以下为汇编后的机器码:

Text Segment				
Bkpt	Address	Code	Basic	
☐	0x00003000	0x00a06693	ori x13,x0,0x0000000a	5: ori a3, zero, 10
☐	0x00003004	0x00106793	ori x15,x0,0x00000001	6: ori a5, zero, 1
☐	0x00003008	0x00106613	ori x12,x0,0x00000001	7: ori a2, zero, 1
☐	0x0000300c	0x00006713	ori x14,x0,0x00000000	8: ori a4, zero, 0
☐	0x00003010	0x00f70733	add x14,x14,x15	10: add a4, a4, a5
☐	0x00003014	0x00d78663	beq x15,x13,0x00000006	11: beq a5, a3, finish
☐	0x00003018	0x00c787b3	add x15,x15,x12	12: add a5, a5, a2
☐	0x0000301c	0xff5ff06f	jal x0,0xfffffffffa	13: jal zero, loop
☐	0x00003020	0x00e02023	sw x14,0x00000000(x0)	15: sw a4, 0(zero)
☐	0x00003024	0x0000006f	jal x0,0x00000000	17: jal zero, end

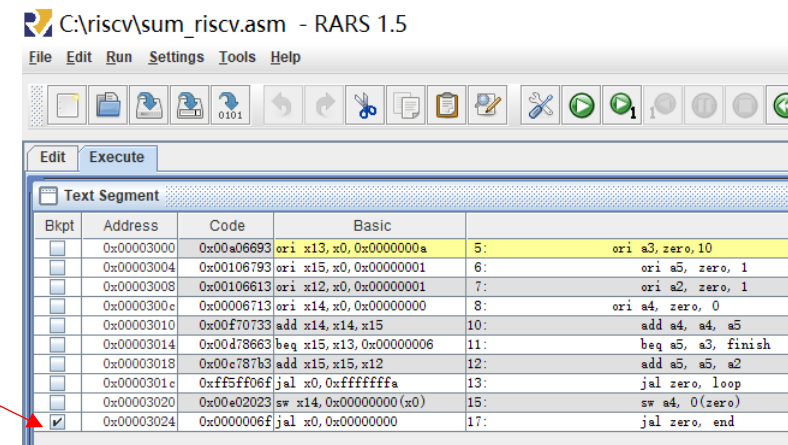
• 第四步：在RARS中运行“sum_riscv.asm”

• Run -> Reset



• 在最后一行指令上设置断点

• Run -> Go

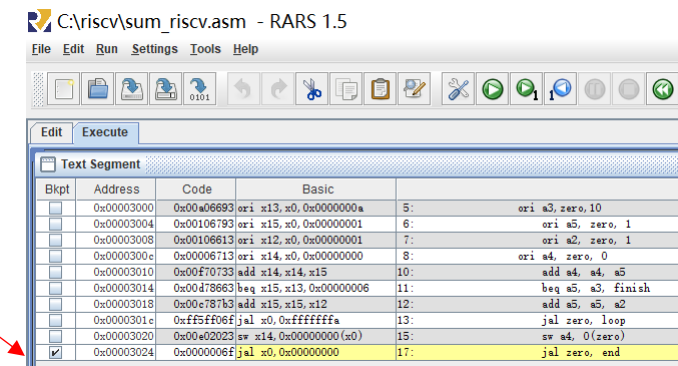


• 运行结束后，程序将停在断点处

• 此时可以看到程序运行的结果在0号存储单元中

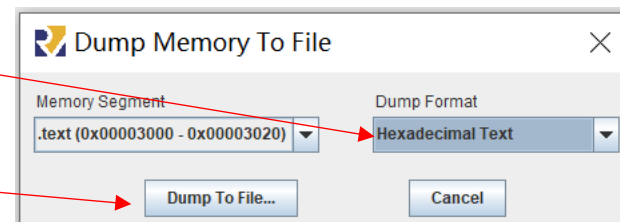
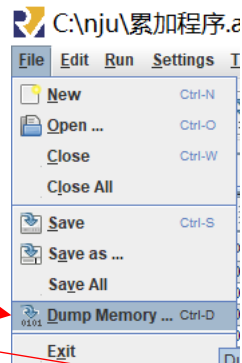
• 结果 = 37h = 55 = 1+2+3+...+10

Data Segment	
Address	Value (+0)
0x00000000	0x00000037



- 第五步：将汇编后的机器码，导出到一个文本文件中

- File -> Dump Memory To File
- 设置对话框中的“Dump Format”为“Hexadecimal Text”
- 点击“Dump To File”
- 输入文件名：sum_riscv.hex，保存



sum_riscv.hex - 记事本
文件(F) 编辑(E) 格式(O)

```
00a06693
00106793
00106613
00006713
00f70733
00d78663
00c787b3
ff5ff06f
00e02023
0000006f
```

Text Segment				
Bkpt	Address	Code	Basic	
☐	0x00003000	0x00a06693	ori x13, x0, 0x0000000a	5: ori a3, zero, 10
☐	0x00003004	0x00106793	ori x15, x0, 0x00000001	6: ori a5, zero, 1
☐	0x00003008	0x00106613	ori x12, x0, 0x00000001	7: ori a2, zero, 1
☐	0x0000300c	0x00006713	ori x14, x0, 0x00000000	8: ori a4, zero, 0
☐	0x00003010	0x00f70733	add x14, x14, x15	10: add a4, a4, a5
☐	0x00003014	0x00d78663	beq x15, x13, 0x00000006	11: beq a5, a3, finish
☐	0x00003018	0x00c787b3	add x15, x15, x12	12: add a5, a5, a2
☐	0x0000301c	0xff5ff06f	jal x0, 0xffffffffa	13: jal zero, loop
☐	0x00003020	0x00e02023	sw x14, 0x00000000(x0)	15: sw a4, 0(zero)
☐	0x00003024	0x0000006f	jal x0, 0x00000000	17: jal zero, end

• (3) 运行RISC-V计算费波那契数列的程序

- 斐波那契数列（Fibonacci sequence），又称黄金分割数列，指的是这样一个数列：0、1、1、2、3、5、8、13、21、34、55、……，在数学上，斐波那契数列通常以递推的方法定义： $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$ ($n \geq 2$, $n \in \mathbb{N}^*$)。
- 计算n+1个费波那契数列的程序如下（**fib_riscv.asm**，位于\ riscv目录中），n在程序中设置，计算好的费波那契数列存放在数据存储器地址为0、4、8、…、4*n的存储单元中。
- 在RARS 1.5汇编仿真器中运行该程序，并将该程序的机器码保存到**fib_riscv.hex**文件中。

```
fib_riscv.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
#RISC-V程序 计算费波那契数列 fib_riscv.asm
# n为数据个数 (可以改变)
# 计算好的n+1个数 (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...) 存放在地址为0, 4, 8, 12, ...4*n的数据存储器中

main:
    ori a2, zero, 10          # a2 = 10 = n
    ori a1, zero, 0            # a1 = 0
    sw a1, 0(zero)             # 0 -> (0)

    ori a1, zero, 1            # a1 = 1
    sw a1, 4(zero)             # 1 -> (4)

    sw a1, 8(zero)             # 1 -> (8)

    ori a3, zero, 2            # a3 = 2
    ori a4, zero, 1            # a4 = 1
    ori a5, zero, 1            # a5 = 1

loop:
    add a6, a4, a5             # a4+a5 -> a6
    ori a4, a5, 0              # a5 -> a4
    ori a5, a6, 0              # a6 -> a5
    add a3, a3, a1             # a3+1 -> a3    |a1=1
    add a7, a3, a3             # a3*2 -> a7
    add a0, a7, a7             # a7*2 -> a0
    sw a6, 0(a0)               # a6 -> 0(a0)
    beq a2, a3, end            # if a2=a3 goto end
    jal zero, loop             # goto loop

end:
    jal zero, end              # goto end
```

```
fib_riscv.hex -
文件(F) 编辑(E)
v2.0 raw
00a06613
00006593
00b02023
00106593
00b02223
00b02423
00206693
00106713
00106793
00f70833
0007e713
00086793
00b686b3
00d688b3
01188533
01052023
00d60463
fe1ff06f
0000006f
```

运算结果

0x00000000	0x00000000	0x00000001	0x00000001	0x00000002	0x00000003	0x00000005	0x00000008	0x0000000d
0x00000020	0x00000015	0x00000022	0x00000037	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

#RISC-V程序 计算费波那契数列 fib_riscv.asm

n为数据个数 (可以改变)

计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、.....) 存放在地址为0、4、8、12、...4*n的数据存储器中

main:

ori a2, zero, 10 # a2 = 10 = n

ori a1, zero, 0 # a1 = 0
sw a1, 0(zero) # 0 -> (0)ori a1, zero, 1 # a1 = 1
sw a1, 4(zero) # 1 -> (4)

sw a1, 8(zero) # 1 -> (8)

ori a3, zero, 2 # a3 = 2
ori a4, zero, 1 # a4 = 1
ori a5, zero, 1 # a5 = 1

loop:

add a6, a4, a5 # a4+a5 -> a6
ori a4, a5, 0 # a5 -> a4
ori a5, a6, 0 # a6 -> a5
add a3, a3, a1 # a3+1 -> a3 | a1=1
add a7, a3, a3 # a3*2 -> a7
add a0, a7, a7 # a7*2 -> a0
sw a6, 0(a0) # a6 -> 0(a0)
beq a2, a3, end # if a2=a3 goto end
jal zero, loop # goto loop

end:

jal zero, end # goto end

- (4) 要求

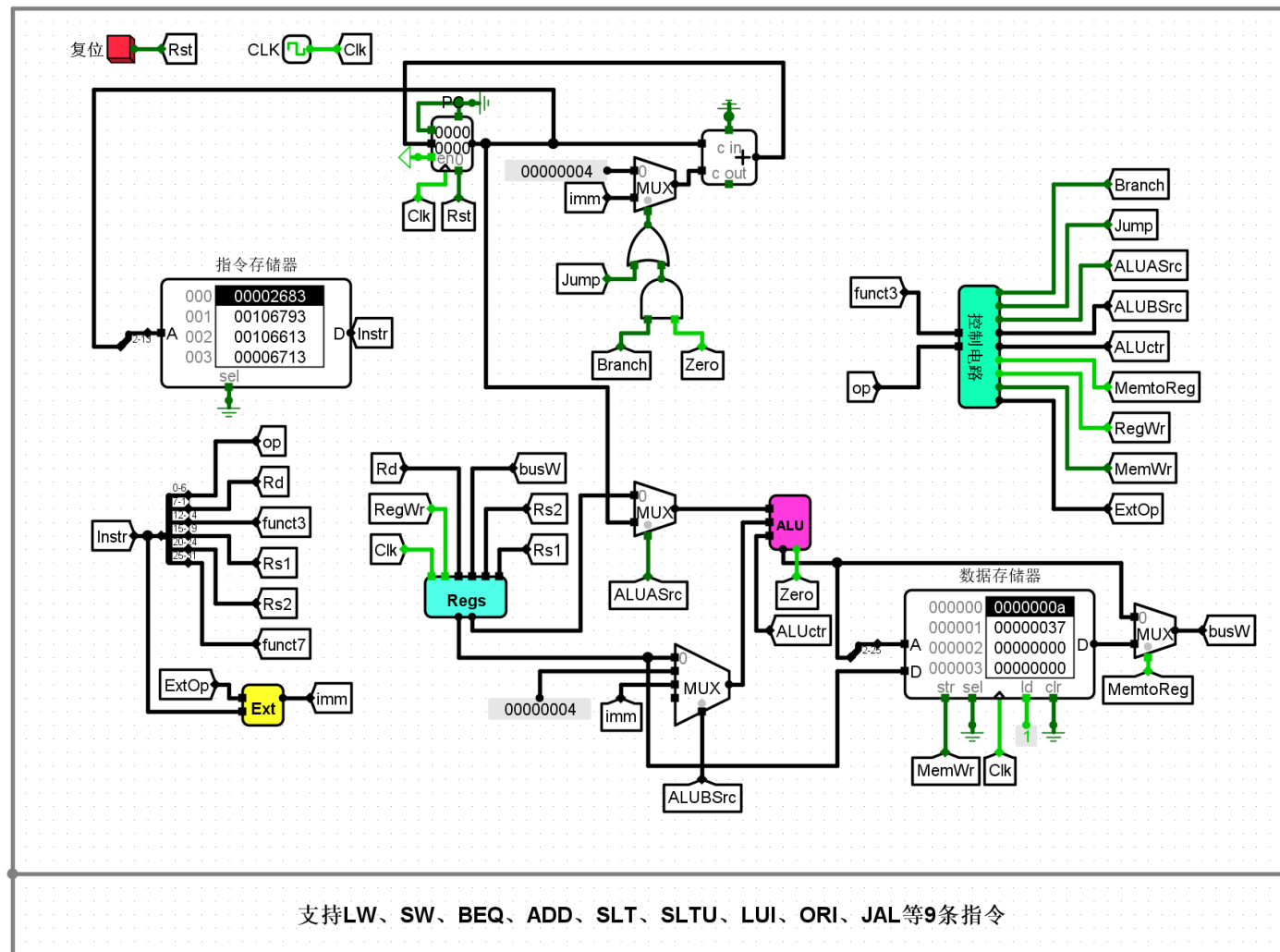
- 请同学们修改`sum_riscv.asm`程序，计算 $1+2+3+\cdots+100=5050=13\text{bah}$ （修改后的程序命名为：`sum100_riscv.asm`），然后在RARS 1.5汇编仿真器中运行，并将该程序的机器码保存到`sum100_riscv.hex`文件中。

4、在Logisim上运行RISC-V程序

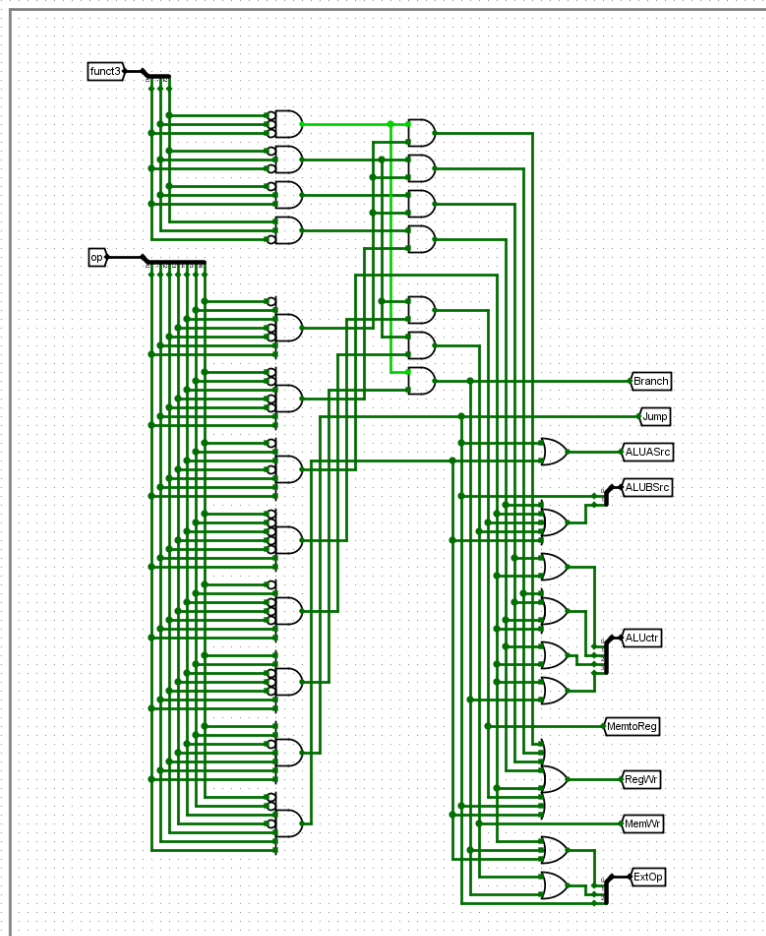
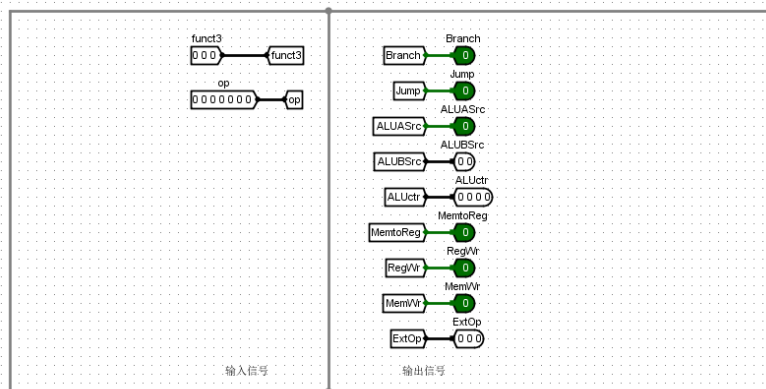
- (1) Logisim实现的单周期RISC-V处理器能够运行以下9条指令

- | | |
|----------------------------------|--|
| ① <code>add rd,rs1,rs2</code> | ; <code>rs1+rs2->rd</code> |
| ② <code>slt rd,rs1,rs2</code> | ; 带符号数比较指令 if <code>rs1<rs2</code> 1-> <code>rd</code> else 0-> <code>rd</code> |
| ③ <code>sltu rd,rs1,rs2</code> | ; 无符号数比较指令 if <code>rs1<rs2</code> 1-> <code>rd</code> else 0-> <code>rd</code> |
| ④ <code>ori rd,rs1,imm12</code> | ; <code>rs1</code> 或 <code>imm12</code> -> <code>rd</code> |
| ⑤ <code>lw rd,rs1,imm12</code> | ; <code>M[rs1+imm12]</code> -> <code>rd</code> |
| ⑥ <code>lui rd,imm20</code> | ; <code>imm20</code> -> <code>rd</code> |
| ⑦ <code>sw rs2,rs1,imm12</code> | ; <code>rs2</code> -> <code>M[rs1+imm12]</code> |
| ⑧ <code>beq rs1,rs2,imm12</code> | ; if <code>rs1=rs2</code> then <code>goto imm12</code> |
| ⑨ <code>jal rd,imm20</code> | ; <code>goto imm20</code> |

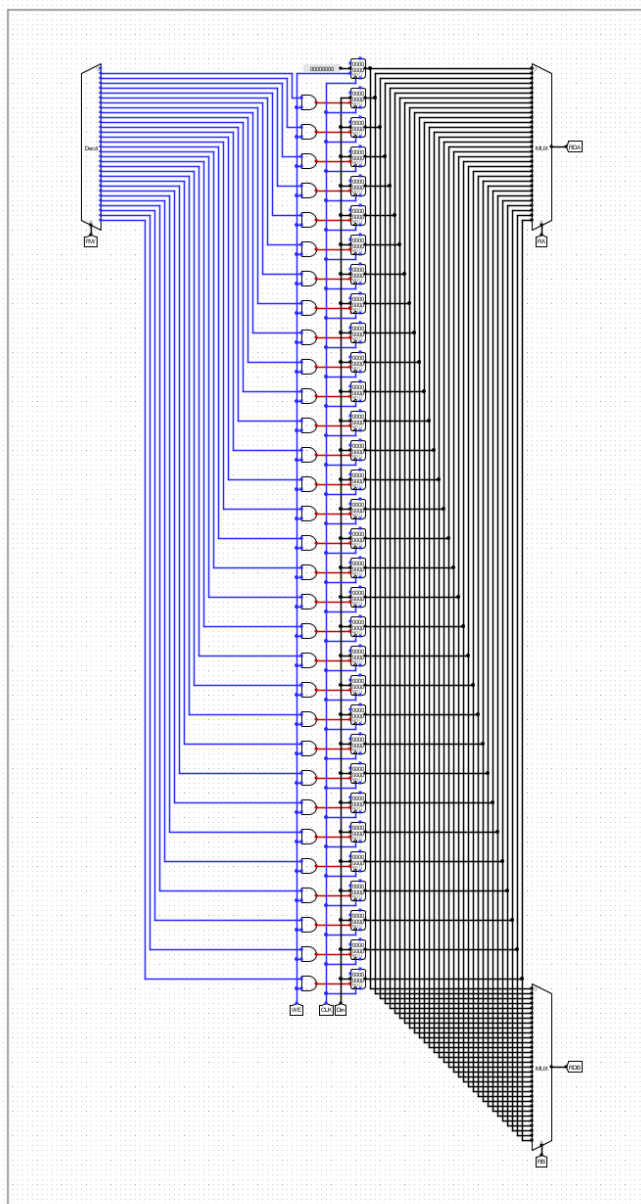
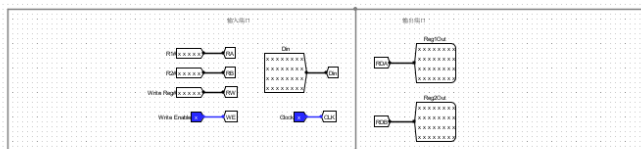
- (2) Logisim实现的单周期RISC-V处理器 (**单周期RISC-V处理器.circ**)



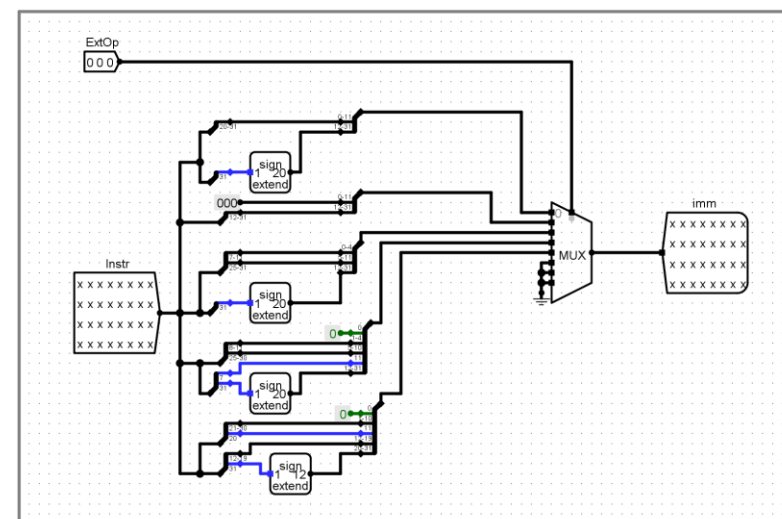
单周期RISC-V处理器数据通路



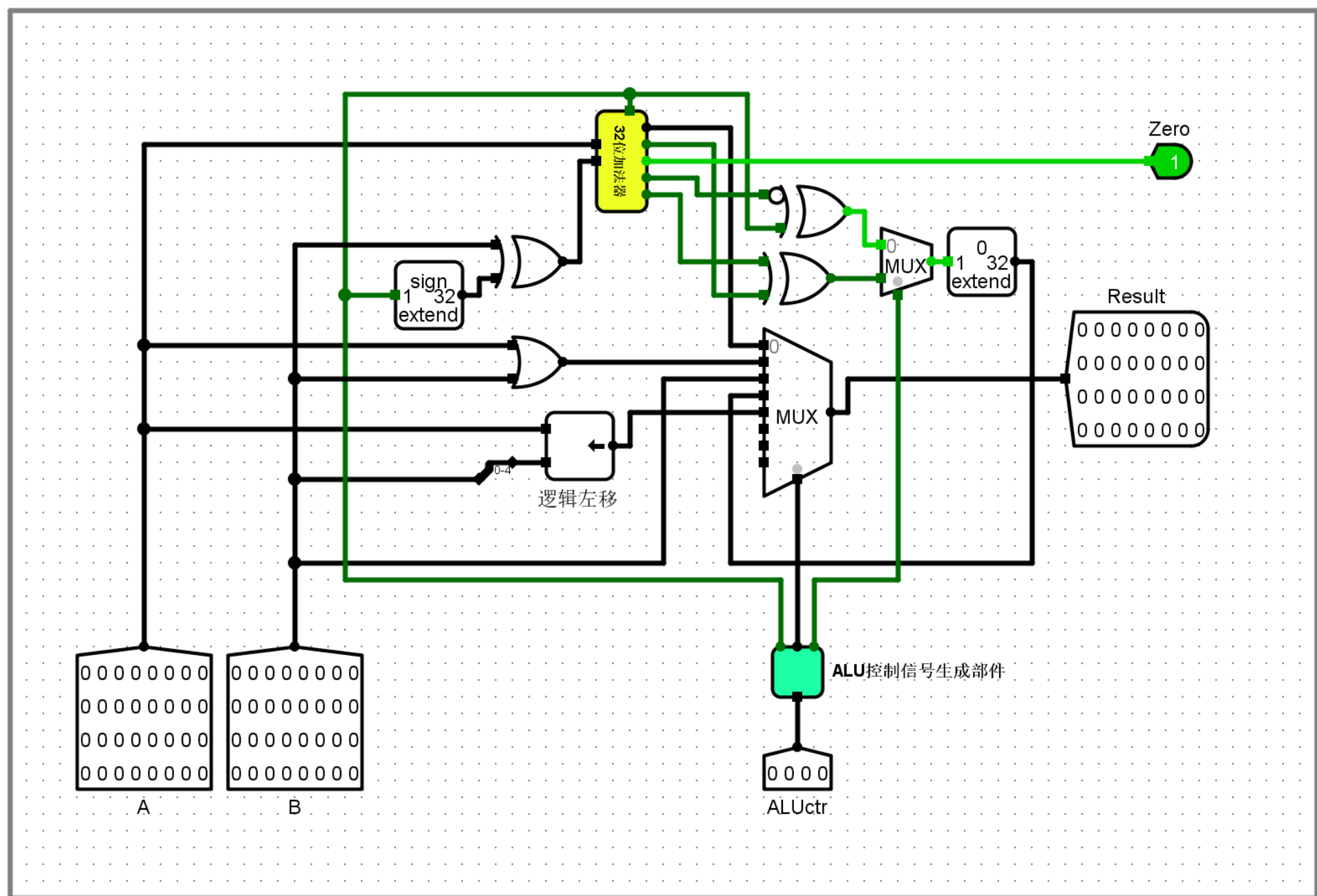
控制电路



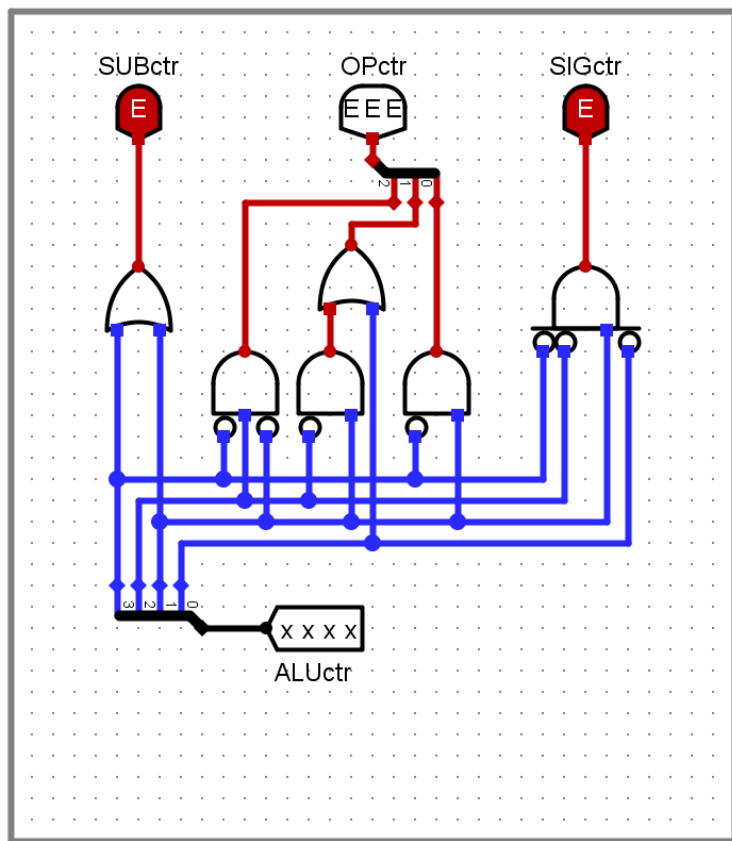
寄存器堆Regs



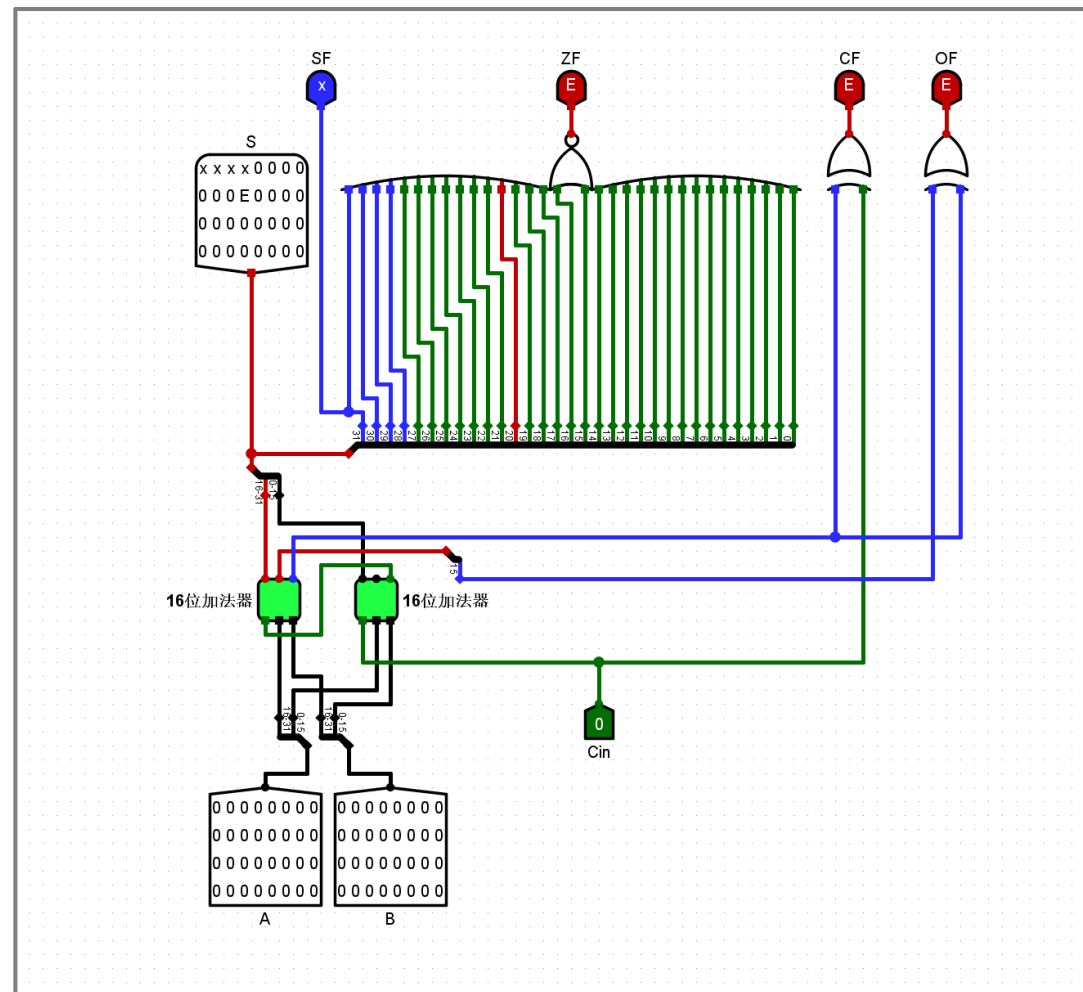
立即数扩展部件Ext



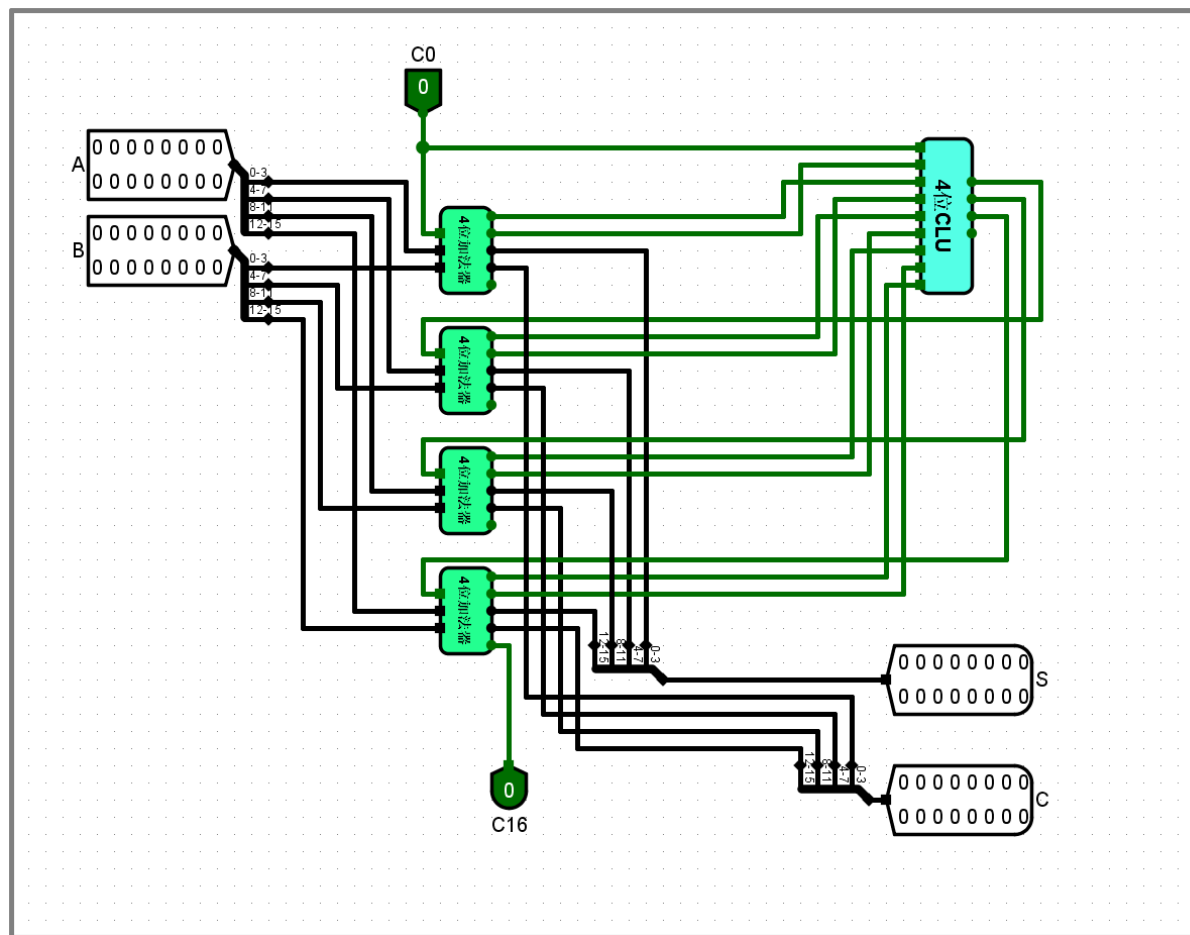
运算器ALU



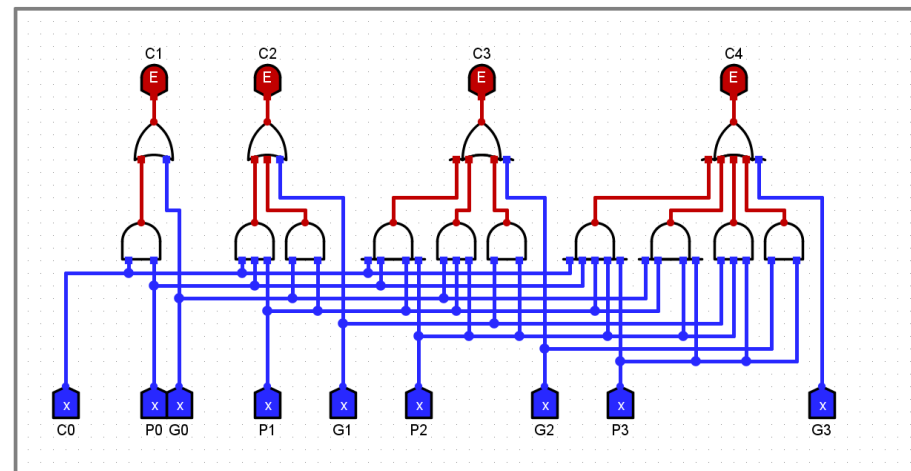
ALU控制信号生成部件



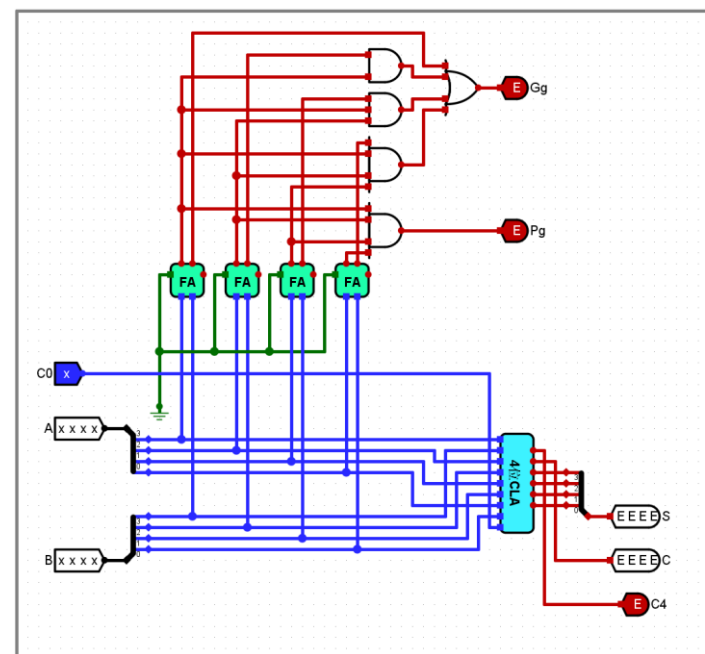
32位加法器



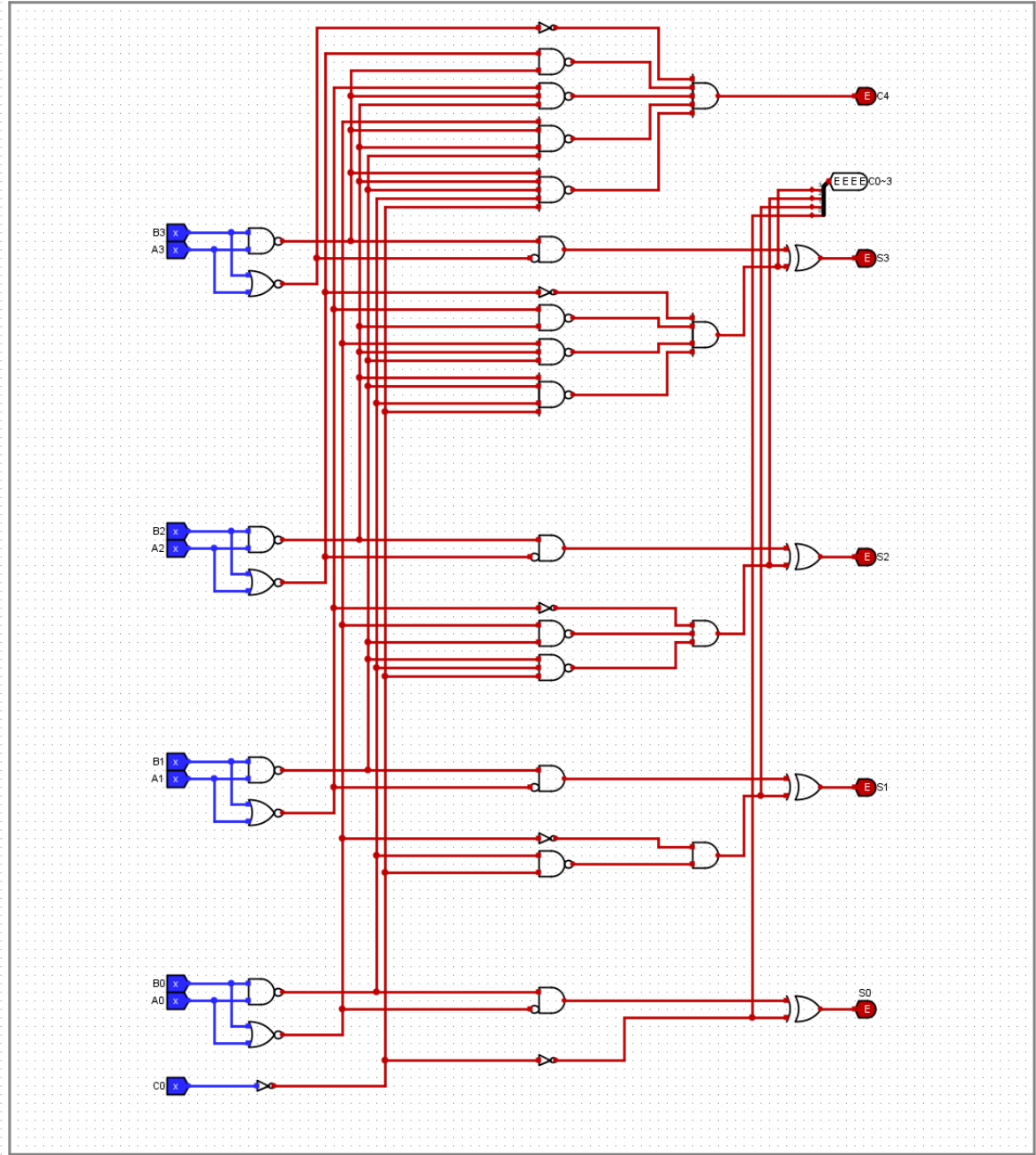
16位加法器



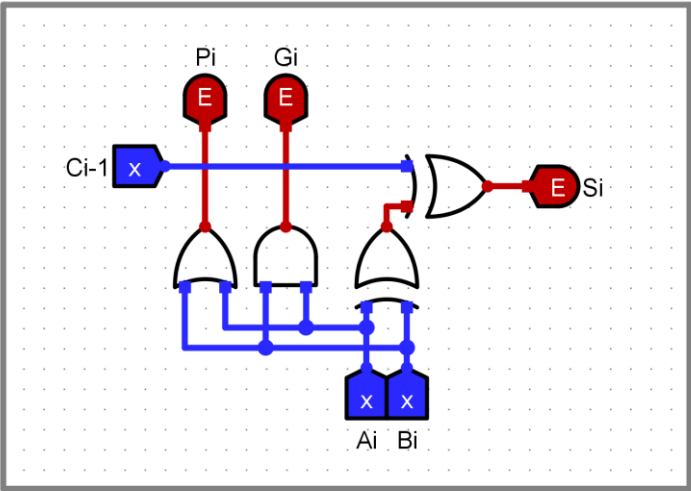
4位先行进位电路CLU



4位加法器



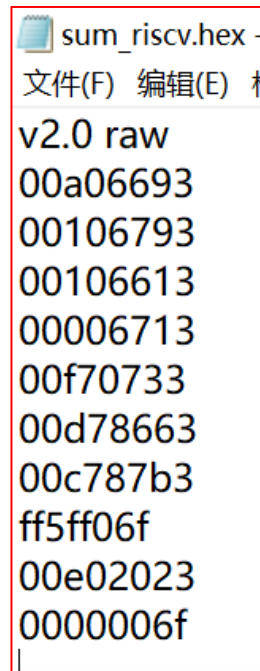
4位快速加法器 **CLA**



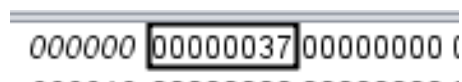
1位全加器**FA**

- (3) 在Logisim实现的单周期RISC-V处理器上运行**求累加和**的程序

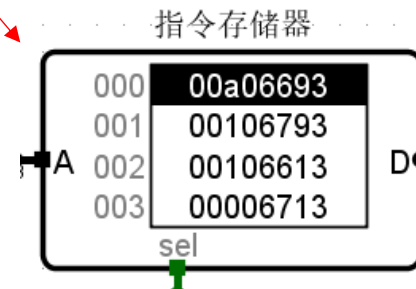
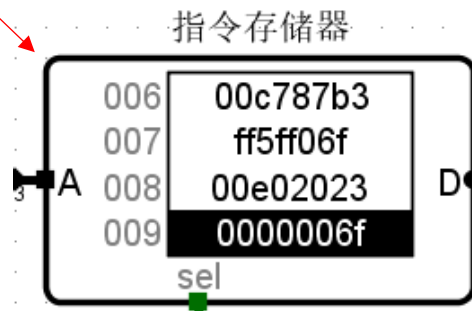
- 第一步：修改前面导出的机器码文件sum_riscv.hex，增加：**v2.0 raw**
- 第二步：在Logisim中打开设计文件“**单周期RISC-V处理器.circ**”（位于\rvscv目录中）
- 第三步：按**Ctrl+R**
- 第四步：在单周期RISC-V处理器数据通路的指令存储器中装入机器码文件**sum_riscv.hex**
- 第五步：设置时钟频率=1KHz，按**Ctrl+K**启动时钟，程序开始执行
- 第六步：指令存储器停止滚动后，表示程序执行结束
- 第七步：再按**Ctrl+K**停止时钟
- 第八步：此时观察数据存储器第0号单元的值（累加和，正确值为**37h=55**）



sum_riscv.hex
文件(F) 编辑(E) 格式(O)
v2.0 raw
00a06693
00106793
00106613
00006713
00f70733
00d78663
00c787b3
ff5ff06f
00e02023
0000006f



00000000 00000037 00000000 (



- (4) 在Logisim实现的单周期RISC-V处理器上运行计算费波那契数列的程序

- 第一步：修改前面导出的机器码文件fib_riscv.hex，增加：v2.0 raw

- 第二步：在Logisim中按Ctrl+R

- 第三步：在指令存储器中装入机器码文件fib_riscv.hex

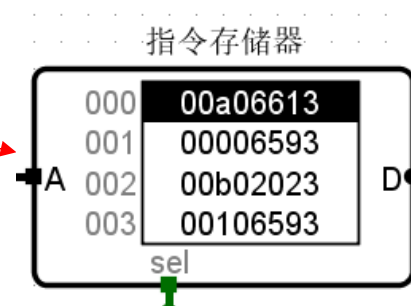
- 第四步：设置时钟频率=1KHz，按Ctrl+K启动时钟，程序开始执行

- 第五步：指令存储器停止滚动后，表示程序执行结束

- 第六步：再按Ctrl+K停止时钟

- 第七步：此时观察数据存储器中的内容（0号单元开始）

```
fib_riscv.hex
文件(F) 编辑(E)
v2.0 raw
00a06613
00006593
00b02023
00106593
00b02223
00b02423
00206693
00106713
00106793
00f70833
0007e713
00086793
00b686b3
00d688b3
01188533
01052023
00d60463
fe1ff06f
0000006f
```



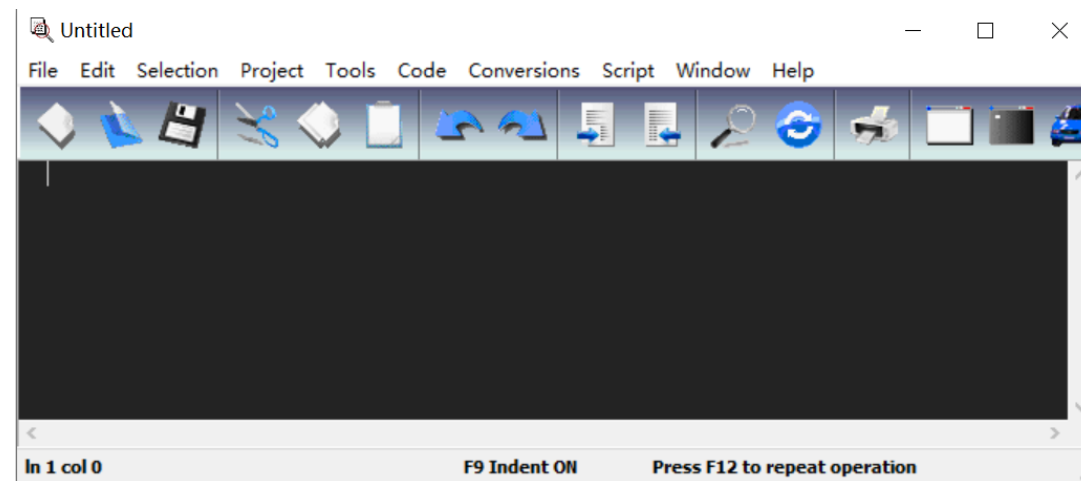
000000 00000000 00000001 00000001 00000002 00000003 00000005 00000008 0000000d 00000015 00000022 00000037 00000000 00000000 00000000 00000000

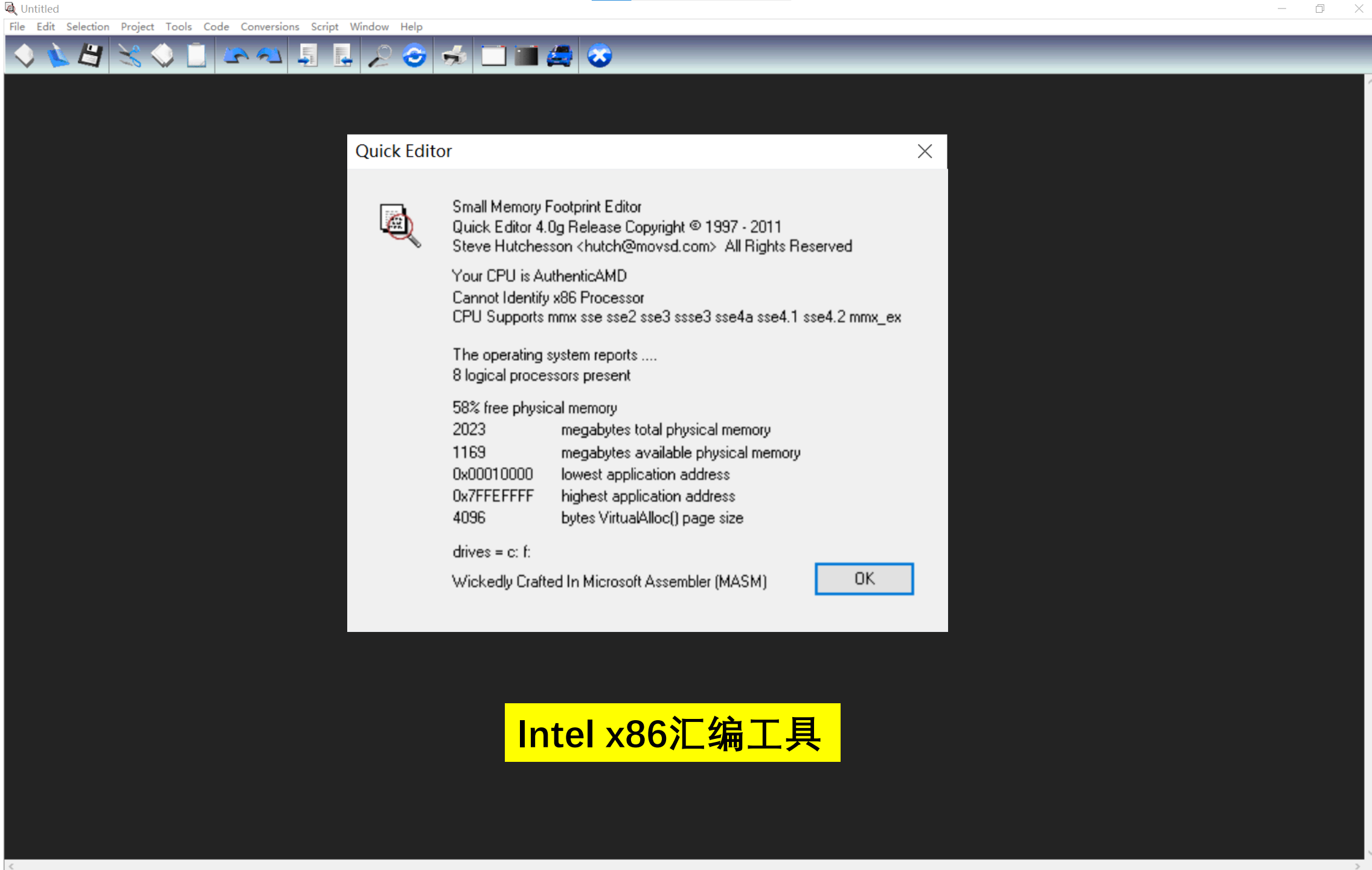
- (5) 要求

- 请同学们在Logisim实现的单周期RISC-V处理器上，运行计算 $1+2+3+\cdots+100=5050=13bah$ 的程序（`sum100_riscv.hex`），观看数据存储器0号单元的值是不是**13bah**？

5、Intel x86汇编语言程序的运行

- (1) Intel x86汇编工具
 - 点击运行\masm32目录中的“**qeditor.exe**”。





Intel x86汇编工具

- (2) 运行Intel x86汇编语言程序

- 显示“Hello World!”的Intel x86汇编语言程序如下 (**hello1_x86.asm**, 位于\ x86目录中) :

代码段

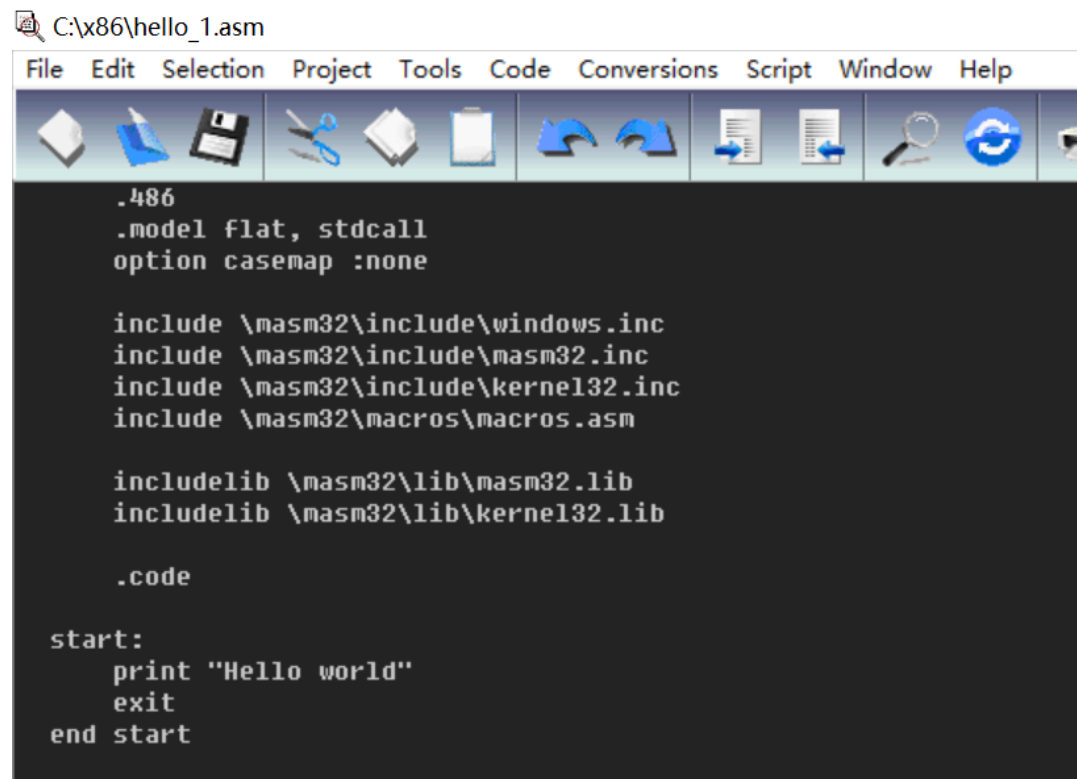
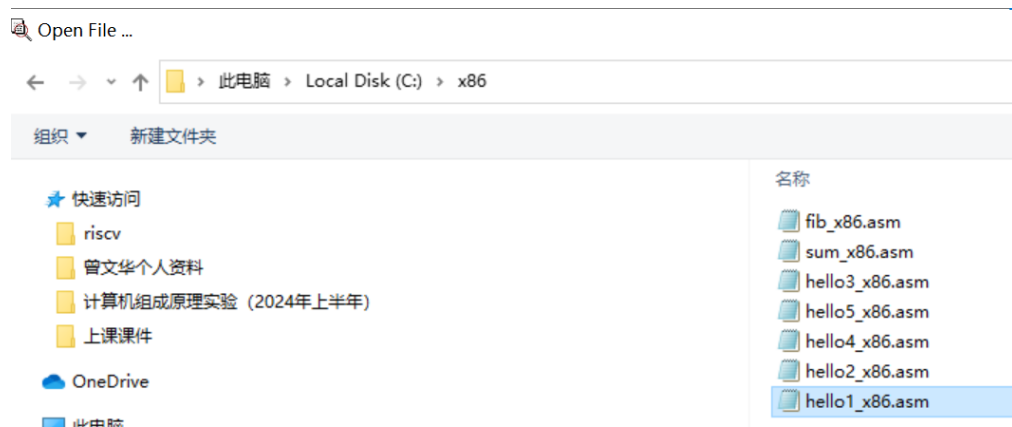
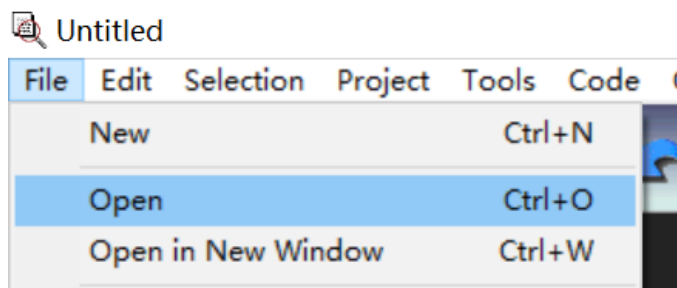
```
hello1_x86.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

.486
.model flat, stdcall
option casemap :none
include \masm32\include\windows.inc
include \masm32\include\masm32.inc
include \masm32\include\kernel32.inc
include \masm32\macros\macros.asm
includelib \masm32\lib\masm32.lib
includelib \masm32\lib\kernel32.lib

.code
start:
    print "Hello World!"
    exit
end start
```

显示字符串

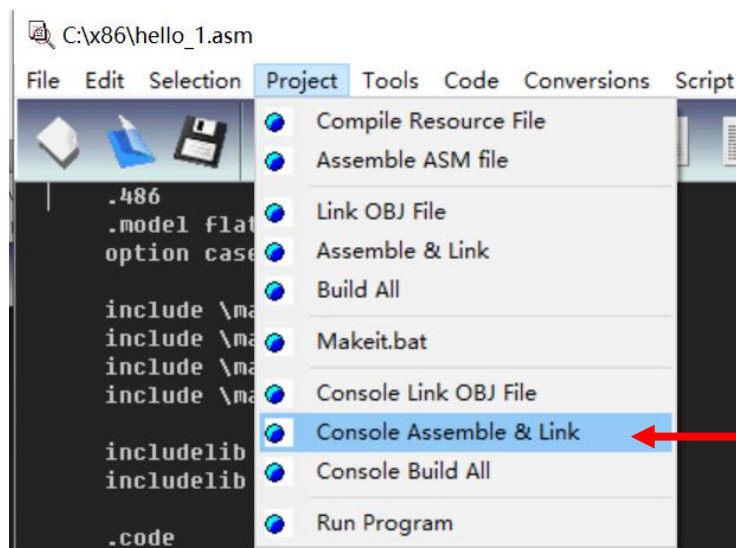
- 第一步：在masm32汇编工具中打开源程序“**hello1_x86.asm**”
 - 点击菜单：File -> Open



- 第二步：汇编

- 点击菜单：Project -> Console Assemble & Link

- 得到：hello1_x86.obj、hello1_x86.exe



此电脑 > Local Disk (C:) > x86

☐ 名称

hello1_x86.exe

hello1_x86.obj

```
C:\WINDOWS\system32\cmd.exe
Microsoft (R) Macro Assembler Version 6.14.8444
Copyright (C) Microsoft Corp 1981-1997. All rights reserved.

Assembling: C:\x86\hello1_x86.asm

*****
ASCII build
*****

Microsoft (R) Incremental Linker Version 5.12.8078
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.

驱动器 C 中的卷是 Local Disk
卷的序列号是 9A35-E55C

C:\x86 的目录
2023/04/18 08:22          335 hello1_x86.asm
2023/04/18 08:26       2,560 hello1_x86.exe
2023/04/18 08:26        567 hello1_x86.obj
                3 个文件          3,462 字节
                0 个目录 121,836,699,648 可用字节
请按任意键继续. . .
```

- 第三步：运行“**hello1_x86.exe**”

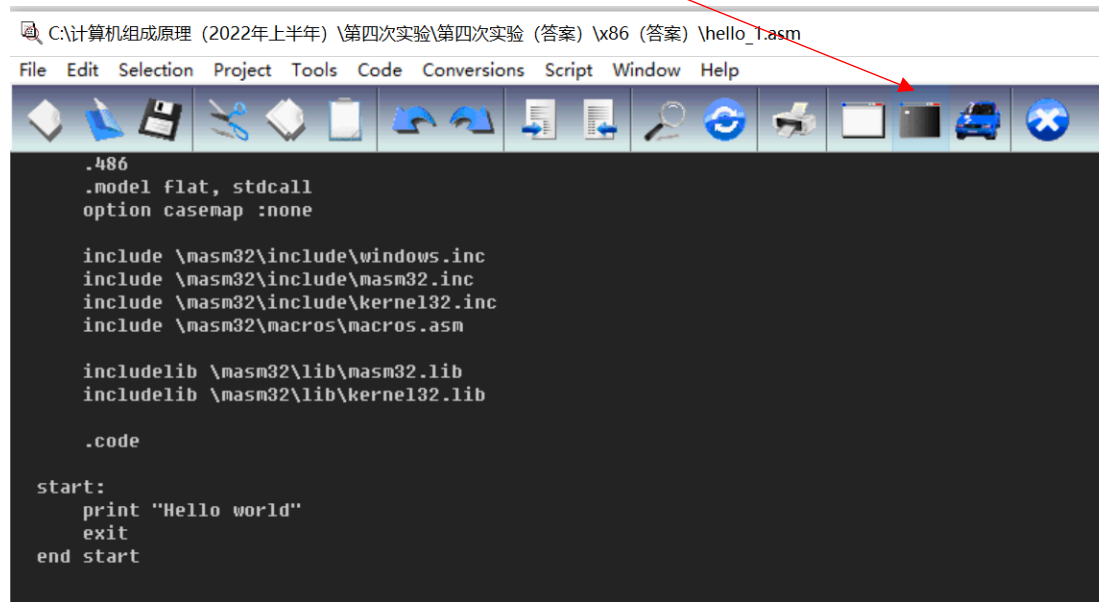
- 点击图标打开“命令提示符”

- 在“命令提示符”中输入“**hello1_x86.exe**”，即可执行程序

C:\Windows\System32\cmd.exe

Microsoft Windows [版本 10.0.19044.2846]
(c) Microsoft Corporation。保留所有权利。

C:\x86>hello1_x86.exe
Hello World!
C:\x86>_



```
C:\计算机组成原理 (2022年上半年) \第四次实验\第四次实验 (答案) \x86 (答案) \hello_1.asm
File Edit Selection Project Tools Code Conversions Script Window Help
[Icons]
.486
.model flat, stdcall
option casemap :none

include \masm32\include\windows.inc
include \masm32\include\masm32.inc
include \masm32\include\kernel32.inc
include \masm32\macros\macros.asm

includelib \masm32\lib\masm32.lib
includelib \masm32\lib\kernel32.lib

.code

start:
    print "Hello world"
    exit
end start
```

- 其它版本的“Hello World”程序

```
hello_2.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
IF 0
    Build this template with "CONSOLE ASSEMBLE AND LINK"
ENDIF

__UNICODE__ equ 1

include \masm32\include\masm32rt.inc

.code

start:
    call main
    inkey
    exit

main proc
    cls
    print "Hello World in UNICODE",13,10
    ret
main endp

end start
```

第2个Hello World程序

```
C:\x86>hello2_x86.exe
```

```
C:\Windows\System32\cmd.exe - hello_2.exe
```

```
Hello World in UNICODE
Press any key to continue ...
```

hello_3.asm - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
.486                                ; create 32 bit code
.model flat, stdcall                ; 32 bit memory model
option casemap :none                ; case sensitive

include \masm32\include\windows.inc ; always first
include \masm32\macros\macros.asm   ; MASM support macros
include \masm32\include\masm32.inc
include \masm32\include\gdi32.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc

includelib \masm32\lib\masm32.lib
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.code                                ; Tell MASM where the code starts

start:                              ; The CODE entry point to the program
    print chr$("Hey, this actually works.",13,10)
    exit
end start                            ; Tell MASM where the program ends
```

第3个Hello World程序

```
C:\x86>hello3_x86.exe
Hey, this actually works.
```

```
C:\x86>_
```

```
hello_4.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
|486
.model flat, stdcall
option casemap:none

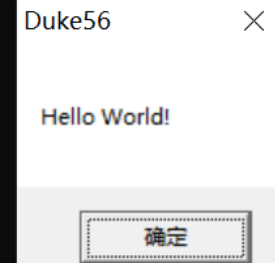
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib

.data
    szCaption db 'Duke56',0
    szText    db 'Hello World!',0

.code
start:
    invoke MessageBox,\
        NULL,\
        offset szText,\
        offset szCaption,\
        MB_OK
    invoke ExitProcess, NULL
end start
```

第4个Hello World程序

C:\x86>hello4_x86.exe




```
hello_5.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
486
.model flat, stdcall
option casemap: none

include \masm32\include\msvcrt.inc
include \masm32\include\kernel32.inc

includelib \masm32\lib\msvcrt.lib
includelib \masm32\lib\kernel32.lib

.data
    szText db 'Hello World!', 0

.code
start:
    invoke crt_printf, addr szText
    invoke ExitProcess, 0
end start
```

第5个Hello World程序

```
C:\x86>hello5_x86.exe
Hello World!
C:\x86>
```

- (3) 运行求累加和的Intel x86汇编语言程序

- 求累加和的Intel x86汇编语言程序如下（sum_x86.asm，位于\x86目录中）：

```
sum_x86.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
; x86 (32位) 汇编语言程序      sum_x86.asm      求累加和      sum = 1+2+...+n

.486
.model flat, stdcall
option casemap:none
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\msvcrt.lib
includelib \masm32\lib\masm32.lib
include \masm32\include\kernel32.inc
include \masm32\include\user32.inc
include \masm32\include\gdi32.inc
include \masm32\include\windows.inc
include \masm32\include\msvcrt.inc
include \masm32\include\masm32.inc
include \masm32\macros\macros.asm

.data
    n dd ?
    sum dd ?
    a dd 1
    b dd 0

.code
main:
    mov eax, sval(input("Input n= "))    ; 输入n, 送eax
    mov n,eax                            ; eax 保存到 n存储单元中
    mov ecx,n                            ; ecx=n
    mov eax,a                             ; eax=1
    mov ebx,b                             ; ebx=0
sum_loop:
    add ebx,eax
    add eax,a
    cmp eax,ecx                            ; 比较eax和ecx
    jg sum_stop                            ; 大于则转sum_stop
    jmp sum_loop
sum_stop:
    mov sum,ebx                            ; 累加和结果 (ebx) 保存到sum存储单元中
    print chr$("Sum= ")
    print str$(sum)
    exit
end main
```

.data

n dd ?

sum dd ?

a dd 1

b dd 0

.code

main:

mov eax, sval(input("Input n= "))

mov n,eax

mov ecx,n

mov eax,a

mov ebx,b

sum_loop:

add ebx,eax

add eax,a

cmp eax,ecx

jg sum_stop

jmp sum_loop

sum_stop:

mov sum,ebx

print chr\$("Sum= ")

print str\$(sum)

exit

end main

| ; 输入n, 送eax

; eax 保存到 n存储单元中

; ecx=n

; eax=1

; ebx=0

; 比较eax和ecx

; 大于则转sum_stop

; 累加和结果 (ebx) 保存到sum存储单元中

- 第一步：在masm32汇编工具中打开源程序“**sum_x86.asm**”

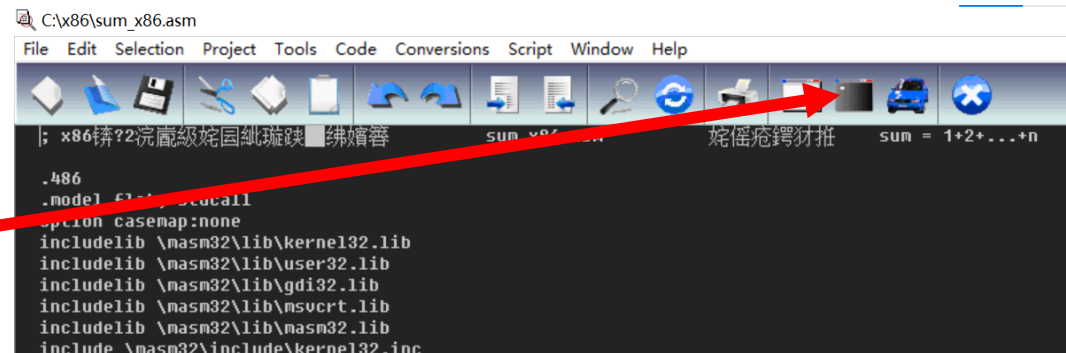
- 点击菜单：File -> Open

- 第二步：汇编

- 点击菜单：Project -> Console Assemble & Link
- 得到：**sum_x86.obj**、**sum_x86.exe**

- 第三步：运行“**sum_x86.exe**”

- 点击图标打开“命令提示符”
- 在“命令提示符”中输入“**sum_x86.exe**”，即可执行程序
- 输入n值（如10），即可得到累加和值Sum（如55）



C:\x86>sum_x86.exe
Input n= 10
Sum= 55
C:\x86>sum_x86.exe
Input n= 100
Sum= 5050
C:\x86>

- (4) 运行计算**费波那契数列**的Intel x86汇编语言程序
- 计算费波那契数列的Intel x86汇编语言程序如下 (**fib_x86.asm**, 位于\x86目录中) :

```
fib_x86.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
.486
.model flat, stdcall

option casemap:none

includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\user32.lib
includelib \masm32\lib\gdi32.lib
includelib \masm32\lib\msvcrt.lib
includelib \masm32\lib\masm32.lib

include \masm32\include\kernel32.inc
include \masm32\include\user32.inc
include \masm32\include\gdi32.inc
include \masm32\include\windows.inc
include \masm32\include\msvcrt.inc
include \masm32\include\masm32.inc
include \masm32\macros\macros.asm

.data
    n dd 0
    a dd 0
    b dd 1
    i dd 1
    t dd 0
```

```
.code
start:
    mov eax, sval(input("Enter a number : "))
    mov n,eax

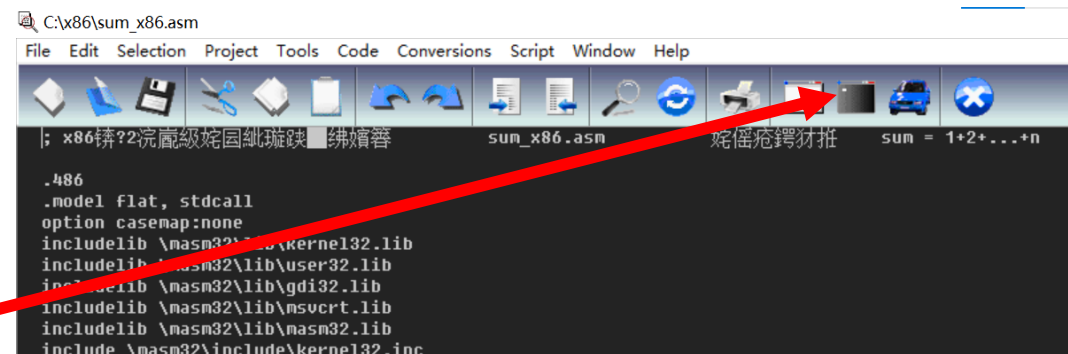
    print chr$("Fibonacci_number")
    print str$(0)
    print chr$(" is ")
    print str$(a)
    print chr$(" ",13,10)

    print chr$("Fibonacci_number")
    print str$(1)
    print chr$(" is ")
    print str$(b)
    print chr$(" ",13,10)

    mov ecx,n
    mov i,ecx
    dec i
    .while i
        mov eax,b
        mov t,eax
        mov eax,b
        add eax,a
        mov b,eax
        print chr$("Fibonacci_number")
        mov ecx,n
        sub ecx,i
        inc ecx
        print str$(ecx)
        print chr$(" is ")
        print str$(b)
        print chr$(" ",13,10)
        mov eax,t
        mov a,eax
        dec i
    .endw
    ret
end start
```

- 第一步：在masm32汇编工具中打开源程序“**fib_x86.asm**”
 - 点击菜单：File -> Open

- 第二步：汇编
 - 点击菜单：Project -> Console Assemble & Link
 - 得到：**fib_x86.obj**、**fib_x86.exe**

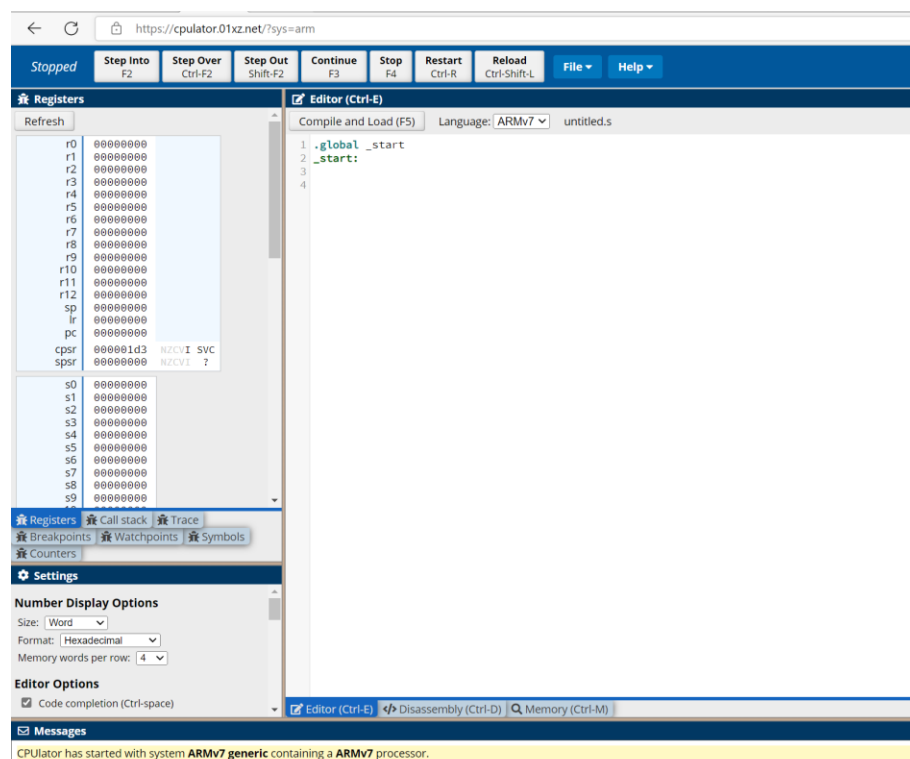


- 第三步：运行“**fib_x86.exe**”
 - 点击图标打开“命令提示符”
 - 在“命令提示符”中输入“**fib_x86.exe**”，即可执行程序
 - 输入数据个数（如10），即可得到费波那契数列：

```
C:\x86>fib_x86.exe
Enter a number : 10
Fibonacci_number0 is 0
Fibonacci_number1 is 1
Fibonacci_number2 is 1
Fibonacci_number3 is 2
Fibonacci_number4 is 3
Fibonacci_number5 is 5
Fibonacci_number6 is 8
Fibonacci_number7 is 13
Fibonacci_number8 is 21
Fibonacci_number9 is 34
Fibonacci_number10 is 55
```

6、ARMv7汇编语言程序的运行

- (1) ARMv7汇编工具
 - 在浏览器中打开ARMv7汇编工具：<https://cpulator.01xz.net/?sys=arm>



CPulator ARMv7 System Simulat

+

←

→

↺

https://cpulator.01xz.net/?sys=arm-de1soc

Stopped

Step Into
F2

Step Over
Ctrl-F2

Step Out
Shift-F2

Continue
F3

Stop
F4

Restart
Ctrl-R

Reload
Ctrl-Shift-L

File ▾

Help ▾

🔍 Registers

Refresh

r0	00000000	
r1	00000000	
r2	00000000	
r3	00000000	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000000	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

s0	00000000	
s1	00000000	
s2	00000000	
s3	00000000	
s4	00000000	
s5	00000000	
s6	00000000	
s7	00000000	
s8	00000000	
s9	00000000	
s10	00000000	

🔍 Registers 🔍 Call stack 🔍 Trace
🔍 Breakpoints 🔍 Watchpoints 🔍 Symbols
🔍 Counters

⚙ Settings

Number Display Options

Size: Word ▾

Format: Hexadecimal ▾

Memory words per row: 4 ▾

Editor Options

☒ Code completion (Ctrl-space)

🔍 Editor (Ctrl-E) </> Disassembly (Ctrl-D) 🔍 Memory (Ctrl-M)

🔍 Editor (Ctrl-E)

Compile and Load (F5) Language: ARMv7 ▾ untitled.s

```
1 .global _start
2 _start:
3
4
```

ARMv7汇编工具

📧 Messages

CPulator has started with system ARMV7 DE1-SoC containing a ARMv7 processor.

Clear

- (2) 在ARMv7汇编工具中运行求累加和程序

- 求累加和的ARMv7汇编语言程序如下（**sum_armv7.s**，位于\armv7目录中）：

```
sum_armv7.s - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
#ARMv7 汇编语言程序      求累加和      sum=1+2+3+.....+n      sum_armv7.s
# 累加和个数n放在[0x100]单元中 (100=0x64)
# 累加和结果sum放在[0x104]单元中 (5050=0x13ba)

.global _start
_start:
.org 0
        ldr r4,num          // r4=0x100
        ldr r5,result       // r5=0x014
        mov r0,#100
        str r0,[r4]         // 将100保存到[0x100]单元中
        ldr r3,[r4]         // 从[0x100]单元中读取数据，送r3
        mov r0,#1
        mov r1,#0

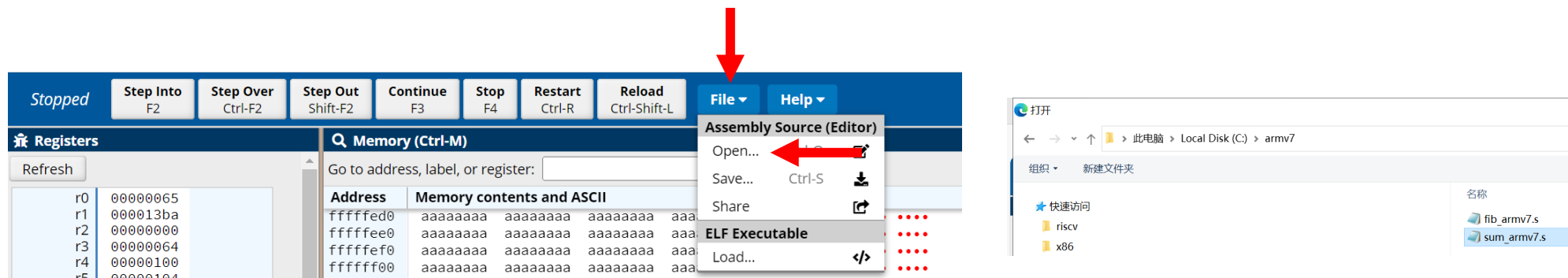
sum:
        add r1,r0
        add r0,#1
        cmp r0,r3
        bgt end             // 如果r0 > r3 则转 end
        b sum              // 转 sum

end:
        str r1,[r5]         // 将r1 保存到 [0x104]单元中

stop:
        b stop              // 转 stop

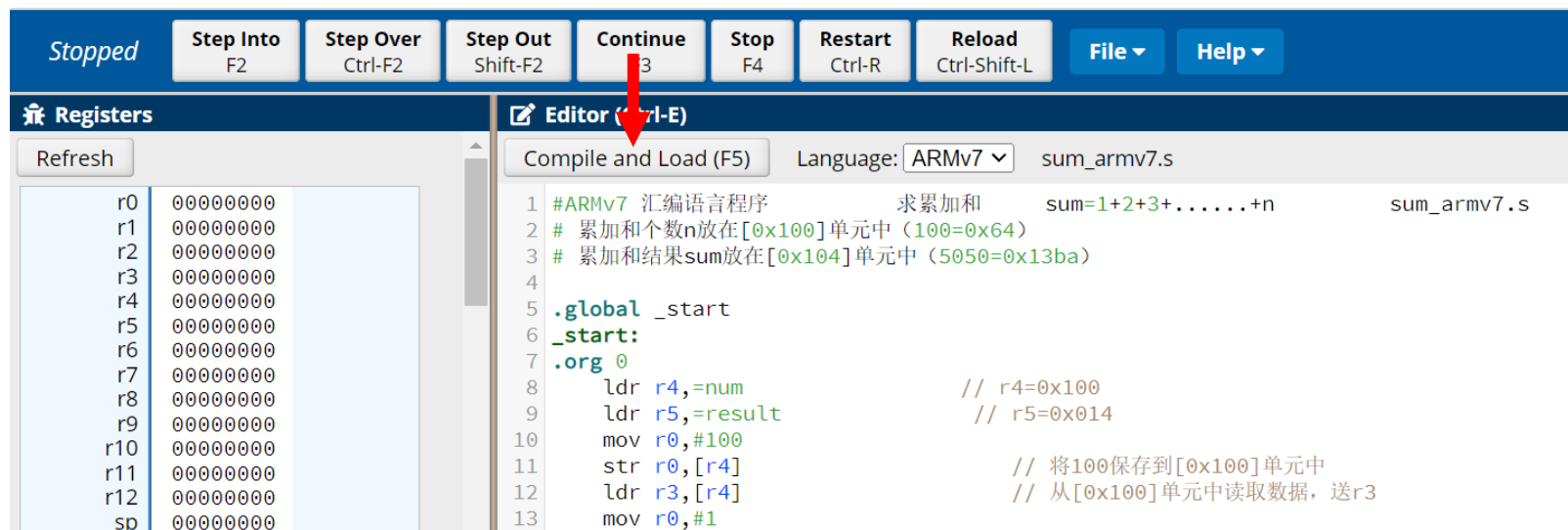
.org 0x100
num:
        .word 0
result:
        .word 0
```

- 第一步：打开源程序 (sum_armv7.s)



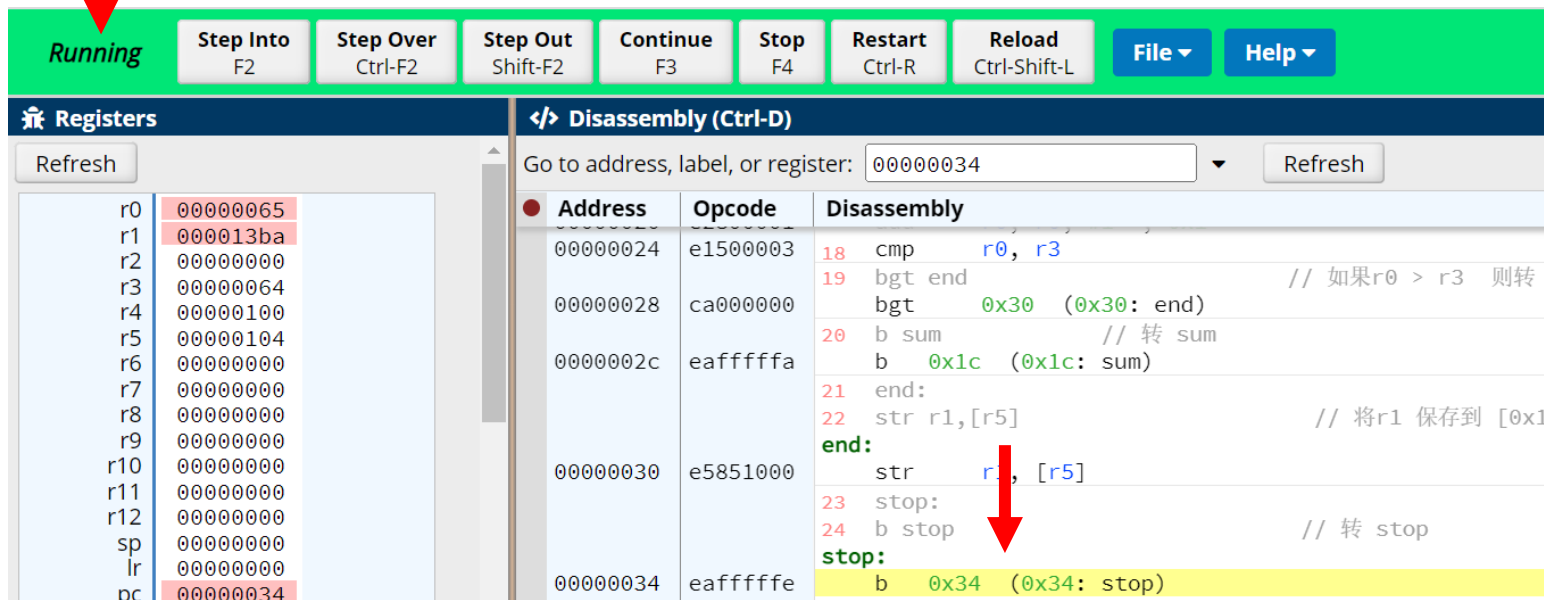
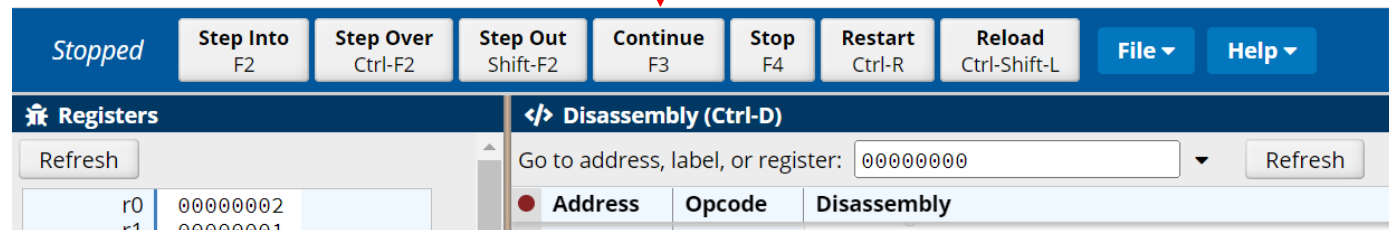
[点击这里](#)

- ## • 第二步：汇编



- 第三步：连续执行程序

点击这里



程序运行结束

Running

Step Into
F2

Step Over
Ctrl-F2

Step Out
Shift-F2

Continue
F3

Stop
F4

Restart
Ctrl-R

Reload
Ctrl-Shift-L

File ▾

Help ▾

Registers

Refresh

r0	00000065	
r1	000013ba	
r2	00000000	
r3	00000064	
r4	00000100	
r5	00000104	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000034	
cpsr	200001d3	NZCVI SVC
spsr	00000000	NZCVI ?
s0	00000000	
s1	00000000	
s2	00000000	

程序运行结果

Disassembly (Ctrl-D)

Go to address, label, or register: 00000034

Refresh

Address	Opcode	Disassembly
000000e4	00000000	andeq r0, r0, r0
000000e8	00000000	andeq r0, r0, r0
000000ec	00000000	andeq r0, r0, r0
000000f0	00000000	andeq r0, r0, r0
000000f4	00000000	andeq r0, r0, r0
000000f8	00000000	andeq r0, r0, r0
000000fc	00000000	andeq r0, r0, r0
num:		
00000100	00000064	andeq r0, r0, r4, RRX
result:		
00000104	000013ba	strheq r1, [r0], -r10
8		ldr r4,=num // r4=0x100
00000108	00000100	andeq r0, r0, r0, LSL #2
9		ldr r5,=result // r5=0x014
0000010c	00000104	andeq r0, r0, r4, LSL #2
_end:		
00000110	aaaaaaaa	bge 0xfeaaabc0
00000114	aaaaaaaa	bge 0xfeaaabc4
00000118	aaaaaaaa	bge 0xfeaaabc8

Editor (Ctrl-E)

Disassembly (Ctrl-D)

Memory (Ctrl-M)

CPulator ARMv7 System Simulat

+

←

→

↺

https://cpulator.01xz.net/?sys=arm-de1soc

Running

Step Into
F2

Step Over
Ctrl-F2

Step Out
Shift-F2

Continue
F3

Stop
F4

Restart
Ctrl-R

Reload
Ctrl-Shift-L

File

Help

Registers

Refresh

r00000065

r100013ba

r20000000

r300000064

r400000100

r50000104

r600000000

r700000000

r800000000

r900000000

r1000000000

r1100000000

r1200000000

sp00000000

lrfeaaaab4

pc00000034

cpsr200001d3NZCVI SVC

spsr000001d3NZCVI SVC

s000000000

s1000000000

s200000000

s300000000

s400000000

s500000000

s600000000

s700000000

s800000000

s900000000

Registers

Call stack

Trace

Breakpoints

Watchpoints

Symbols

Counters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Editor Options

☒ Code completion (Ctrl-space)

Memory (Ctrl-M)

Go to address, label, or register:

Refresh

Address	Memory contents and ASCII				
00000000	e59f4100	e59f5100	e3a00064	e5840000	•A••••Q••••d••••
00000010	e5943000	e3a00001	e3a01000	e0811000	•0••••••••••••
00000020	e2800001	e1500003	ca000000	ea000000	••••••P••••••
00000030	e5851000	ea000000	00000000	00000000	••••••••••••
00000040	00000000	00000000	00000000	00000000	••••••••••••
00000050	00000000	00000000	00000000	00000000	••••••~••••••
00000060	00000000	00000000	00000000	00000000	••••••~••••••
00000070	00000000	00000000	00000000	00000000	••••••~••••••
00000080	00000000	00000000	00000000	00000000	••••••~••••••
00000090	00000000	00000000	00000000	00000000	••••~••••••••
000000a0	00000000	00000000	00000000	00000000	••••~••••••••
000000b0	00000000	00000000	00000000	00000000	••••~••••••~•••
000000c0	00000000	00000000	00000000	00000000	••••~••••~•••~••
000000d0	00000000	00000000	00000000	00000000	••••~••••~•••~••
000000e0	00000000	00000000	00000000	00000000	••••~••••~•••~••
000000f0	00000000	00000000	00000000	00000000	••••~••••~•••~••
00000100	00000064	000013ba	00000100	00000104	d•••~••••~•••~••
00000110	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000120	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000130	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000140	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000150	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000160	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000170	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000180	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000190	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
000001a0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
000001b0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
000001c0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
000001d0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
000001e0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
000001f0	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000200	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaaaa	••••~••••~•••~••
00000210	aaaaaaaa	aa:	aa:	aa:	••••~••••~•••~••
00000220	aaaaaaaa	aa:	aa:	aa:	••••~••••~•••~••
00000230	aaaaaaaa	aa:	aa:	aa:	••••~••••~•••~••
00000240	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaa	••••~••••~•••~••
00000250	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaa	••••~••••~•••~••
00000260	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaa	••••~••••~•••~••
00000270	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaa	••••~••••~•••~••
00000280	aaaaaaaa	aaaaaaaa	aaaaaaaa	aaaaaa	••••~••••~•••~••

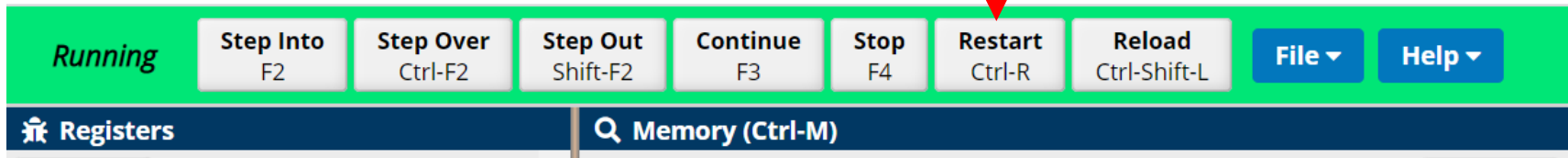
程序运行结果

点击这里，查看主存的内容

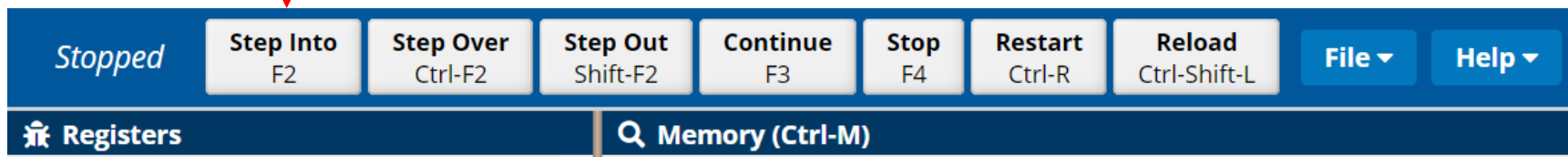
Editor (Ctrl-E) </> Disassembly (Ctrl-D) Memory (Ctrl-M)

- 第四步：单步执行程序

首先点击这里



然后点击这里；点击1次，执行1条指令



Stopped

Step Into
F2Step Over
Ctrl-F2Step Out
Shift-F2Continue
F3Stop
F4Restart
Ctrl-RReload
Ctrl-Shift-L

File ▾

Help ▾

Registers

Refresh

r0	00000000	
r1	00000000	
r2	00000000	
r3	00000000	
r4	00000100	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000004	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

执行第1条指令

Registers

Call stack

Trace

Breakpoints

Watchpoints

Symbols

Counters

Settings

Number Display Options

Size: Word ▾

Format: Hexadecimal ▾

Memory words per row: 4 ▾

Editor Options

</> Disassembly (Ctrl-D)

Go to address, label, or register: 00000000 ▾

Refresh

Address	Opcode	Disassembly
		3 # 累加和结果sum放在[0x104]单元中 (5050=0x13ba)
		5 .global _start
		6 _start:
		7 .org 0
		8 ldr r4,num // r4=0x100
		_start:
00000000	e59f5100	ldr r4,[pc,#256] ; 0x108
		9 ldr r5,result // r5=0x014
00000004	e59f5100	ldr r5,[pc,#256] ; 0x10c
00000008	e3a00064	10 mov r0,#100 ; 0x64
		11 str r0,[r4] // 将100保存到[0x100]单元中
0000000c	e5840000	str r0,[r4]
		12 ldr r3,[r4] // 从[0x100]单元中读取数据,送r3
00000010	e5943000	ldr r3,[r4]
00000014	e3a00001	13 mov r0,#1 ; 0x1
00000018	e3a01000	14 mov r1,#0 ; 0x0
		15 sum:
		16 add r1,r0
		sum:
0000001c	e0811000	add r1,r1,r0
		17 add r0,#1
00000020	e2800001	add r0,r0,#1 ; 0x1
00000024	e1500003	18 cmp r0,r3
		19 bgt end // 如果r0 > r3 则转 end
00000028	ca000000	bgt 0x30 (0x30: end)
		20 b sum // 转 sum
0000002c	ea000000	b 0x1c (0x1c: sum)
		21 end:
		22 str r1,[r5] // 将r1 保存到 [0x104]单元中
		end:
00000030	e5851000	str r1,[r5]
		23 stop:
		24 b stop // 转 stop
		stop:
		b 0x34 (0x34: stop)
00000034	ea000000	andeq r0,r0,r0
00000038	00000000	andeq r0,r0,r0
0000003c	00000000	andeq r0,r0,r0
00000040	00000000	andeq r0,r0,r0
00000044	00000000	andeq r0,r0,r0

Stopped

Step Into
F2

Step Over
Ctrl-F2

Step Out
Shift-F2

Continue
F3

Stop
F4

Restart
Ctrl-R

Reload
Ctrl-Shift-L

File

Help

Registers

Refresh

r0	00000000	
r1	00000000	
r2	00000000	
r3	00000000	
r4	00000100	
r5	00000104	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000008	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

s0	00000000	
s1	00000000	
s2	00000000	
s3	00000000	
s4	00000000	
s5	00000000	
s6	00000000	
s7	00000000	
s8	00000000	
s9	00000000	

RegistersCall stackTraceBreakpointsWatchpointsSymbolsCounters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Editor Options

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000

Refresh

Address	Opcode	Disassembly
		3 # 累加和结果sum放在[0x104]单元中 (5050=0x13ba)
		5 .global _start
		6 _start:
		7 .org 0
		8 ldr r4,num // r4=0x100
		_start:
00000000	e59f4100	ldr r4,[pc,#256] ; 0x108
		9 ldr r5,result // r5=0x014
00000000	e59f5100	ldr r5,[pc,#256] ; 0x10c
00000008	e3a00064	10 mov r0,#100 ; 0x64
		11 str r0,[r4] // 将100保存到[0x100]单元中
0000000c	e5840000	str r0,[r4]
		12 ldr r3,[r4] // 从[0x100]单元中读取数据, 送r3
00000010	e5943000	ldr r3,[r4]
00000014	e3a00001	13 mov r0,#1 ; 0x1
00000018	e3a01000	14 mov r1,#0 ; 0x0
		15 sum:
		16 add r1,r0
		sum:
0000001c	e0811000	add r1,r1,r0
		17 add r0,#1
00000020	e2800001	add r0,r0,#1 ; 0x1
00000024	e1500003	18 cmp r0,r3
		19 bgt end // 如果r0 > r3 则转 end
00000028	ca000000	bgt 0x30 (0x30: end)
		20 b sum // 转 sum
0000002c	ea000000	b 0x1c (0x1c: sum)
		21 end:
		22 str r1,[r5] // 将r1 保存到 [0x104]单元中
		end:
00000030	e5851000	str r1,[r5]
		23 stop:
		24 b stop // 转 stop
		stop:
		b 0x34 (0x34: stop)
00000034	ea000000	andeq r0,r0,r0
00000038	00000000	
0000003c	00000000	andeq r0,r0,r0
00000040	00000000	andeq r0,r0,r0
00000044	00000000	andeq r0,r0,r0

Stopped

Step Into
F2

Step Over
Ctrl-F2

Step Out
Shift-F2

Continue
F3

Stop
F4

Restart
Ctrl-R

Reload
Ctrl-Shift-L

File

Help

Registers

Refresh

r0	00000064
r1	00000000
r2	00000000
r3	00000000
r4	00000100
r5	00000104
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	0000000c
cpsr	000001d3
spsr	00000000

s0

s1

s2

s3

s4

s5

s6

s7

s8

Registers

Call stack

Trace

Breakpoints

Watchpoints

Symbols

Counters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Editor Options

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000

Refresh

Address	Opcode	Disassembly
		3 # 累加和结果sum放在[0x104]单元中 (5050=0x13ba)
		5 .global _start
		6 _start:
		7 .org 0
		8 ldr r4,num // r4=0x100
00000000	e59f4100	_start: ldr r4,[pc,#256] ; 0x108
00000004	e59f5100	9 ldr r5,result // r5=0x014
00000008	e2a00064	10 mov r0,#100 ; 0x64
0000000c	e5840000	11 str r0,[r4] // 将100保存到[0x100]单元中
00000010	e5943000	12 ldr r3,[r4] // 从[0x100]单元中读取数据, 送r3
00000014	e3a00001	13 mov r0,#1 ; 0x1
00000018	e3a01000	14 mov r1,#0 ; 0x0
0000001c	e0811000	15 sum: 16 add r1,r0 sum: add r1,r1,r0
00000020	e2800001	17 add r0,#1
00000024	e1500003	add r0,r0,#1 ; 0x1
00000028	ca000000	18 cmp r0,r3
0000002c	ea000000	19 bgt end // 如果r0 > r3 则转 end
00000030	e5851000	20 b sum // 转 sum
00000034	ea000000	b 0x1c (0x1c: sum)
00000038	ea000000	21 end:
0000003c	ea000000	22 str r1,[r5] // 将r1 保存到 [0x104]单元中
00000040	ea000000	end:
00000044	ea000000	str r1,[r5]
00000048	ea000000	23 stop:
0000004c	ea000000	24 b stop // 转 stop
00000050	ea000000	stop:
00000054	ea000000	b 0x34 (0x34: stop)
00000058	ea000000	andeq r0,r0,r0
0000005c	ea000000	andeq r0,r0,r0
00000060	ea000000	andeq r0,r0,r0
00000064	ea000000	andeq r0,r0,r0

- (3) 在ARMv7汇编工具中运行计算费波那契数列程序

- 计算费波那契数列的ARMv7汇编语言程序如下（fib_armv7.s，位于\armv7目录中）：

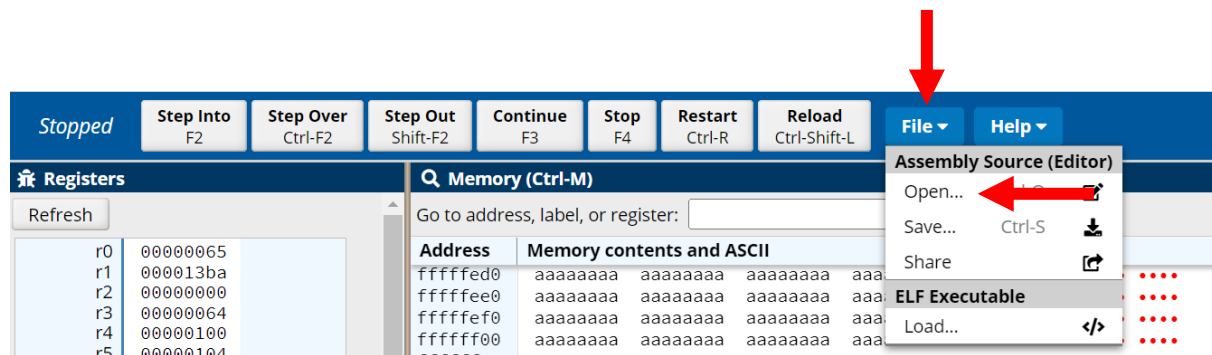
```
fib_armv7.s - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
# 计算费波那契数列      fib_armv7.s
# n=10  可以改变, 保存在0x100存储单元中
# 计算好的n+1个数 (0、1、1、2、3、5、8、13、21、34、55.....      0、1、1、2、3、5、8、0dh、15h、22h、37h.....  )
# 保存在0x200、0x204、0x208、...的存储单元中

.global _start
_start:
.org 0
    ldr r10,=num
    ldr r11,=result
    ldr r2,[r10]
    mov r1,#0
    add r12,r11,#0
    str r1,[r12]
    mov r1,#1
    add r12,r11,#4
    str r1,[r12]
    add r12,r11,#8
    str r1,[r12]
    mov r3,#2
    mov r4,#1
    mov r5,#1
loop:
    add r6,r4,r5
    mov r4,r5
    mov r5,r6
    add r3,r3,#1
    add r7,r3,r3
    add r0,r7,r7
    add r0,r0,r11
    str r6,[r0]
    cmp r2,r3
    beq stop
    b loop
stop:
    b stop

.org 0x100
num:
    .word 10

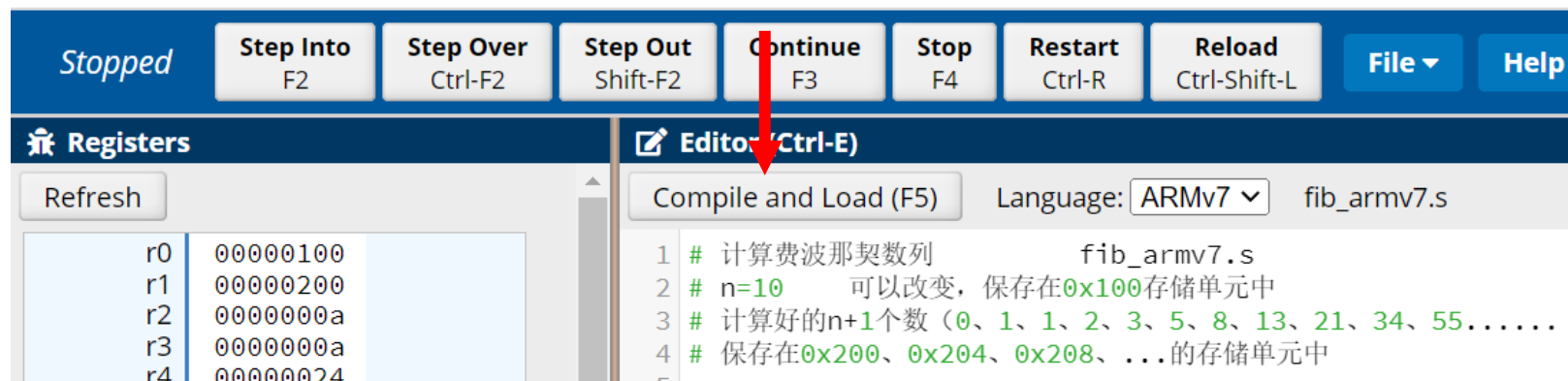
.org 0x200
result:
    .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
```

- 第一步：打开源程序（fib_armv7.s）



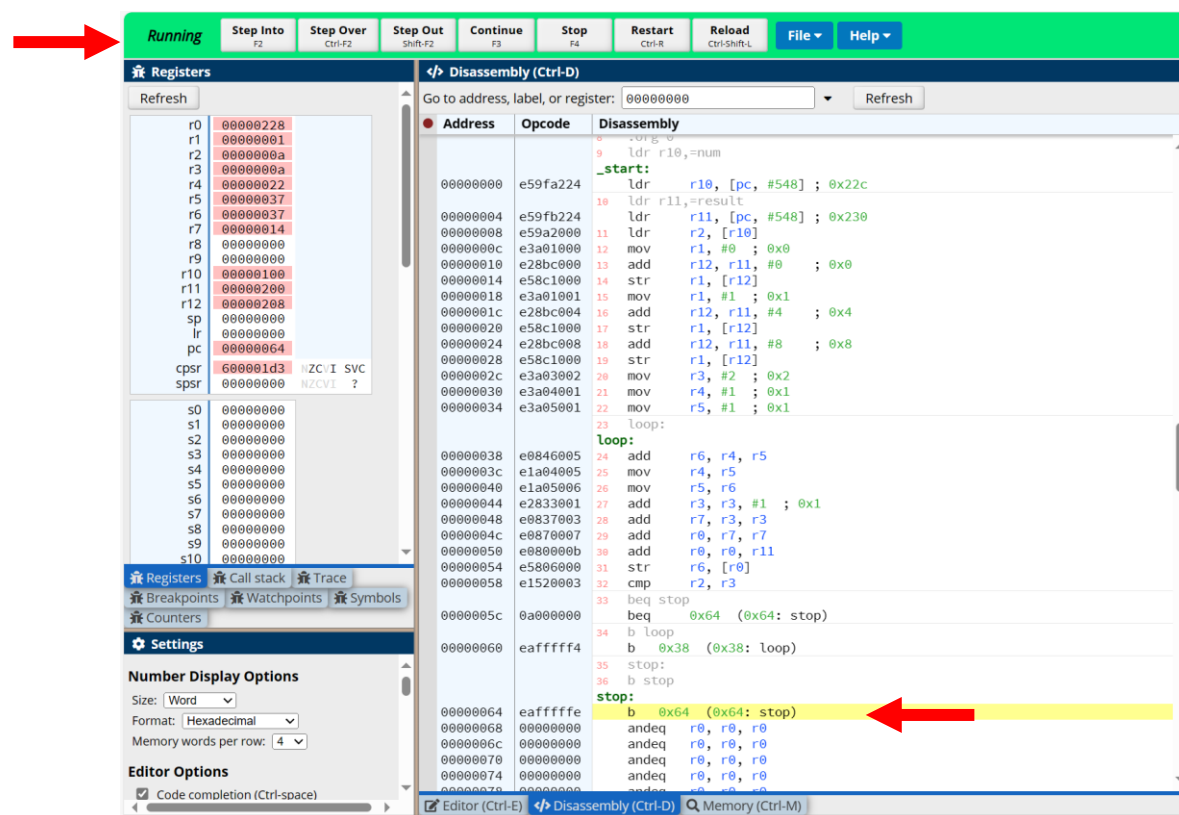
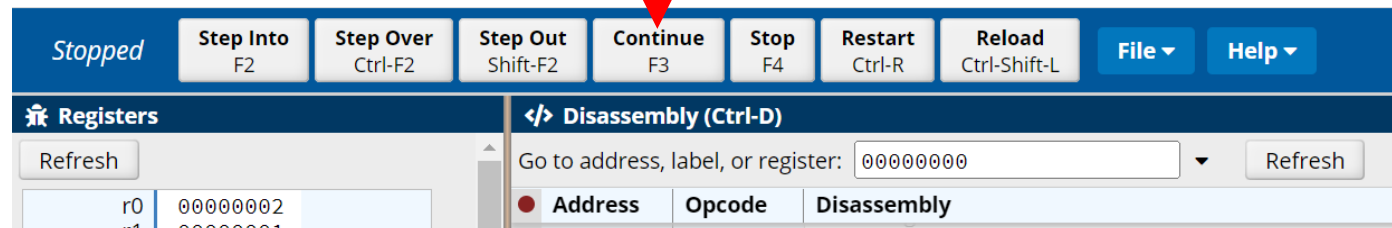
点击这里

- 第二步：汇编



- 第三步：连续执行程序

点击这里



程序运行结束

Running

Step Into
F2

Step Over
Ctrl-F2

Step Out
Shift-F2

Continue
F3

Stop
F4

Restart
Ctrl-R

Reload
Ctrl-Shift-L

File

Help

Registers

Refresh

r0	00000228	
r1	00000001	
r2	0000000a	
r3	0000000a	
r4	00000022	
r5	00000037	
r6	00000037	
r7	00000014	
r8	00000000	
r9	00000000	
r10	00000100	
r11	00000200	
r12	00000208	
sp	00000000	
lr	00000000	
pc	00000064	
cpsr	600001d3	NZCVI SVC
spsr	00000000	NZCVI ?

RegistersCall stackTraceBreakpointsWatchpointsSymbolsCounters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Editor Options

Code completion (Ctrl-space)

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000

Refresh

Address	Opcode	Disassembly
000001cc	00000000	andeq r0, r0, r0
000001d0	00000000	andeq r0, r0, r0
000001d4	00000000	andeq r0, r0, r0
000001d8	00000000	andeq r0, r0, r0
000001dc	00000000	andeq r0, r0, r0
000001e0	00000000	andeq r0, r0, r0
000001e4	00000000	andeq r0, r0, r0
000001e8	00000000	andeq r0, r0, r0
000001ec	00000000	andeq r0, r0, r0
000001f0	00000000	andeq r0, r0, r0
000001f4	00000000	andeq r0, r0, r0
000001f8	00000000	andeq r0, r0, r0
000001fc	00000000	andeq r0, r0, r0
00000200	00000000	andeq r0, r0, r0
00000204	00000001	andeq r0, r0, r1
00000208	00000001	andeq r0, r0, r1
0000020c	00000002	andeq r0, r0, r2
00000210	00000003	andeq r0, r0, r3
00000214	00000005	andeq r0, r0, r5
00000218	00000008	andeq r0, r0, r8
0000021c	0000000d	andeq r0, r0, sp
00000220	00000015	andeq r0, r0, r5, LSL r0
00000224	00000022	andeq r0, r0, r2, LSR #32
00000228	00000037	andeq r0, r0, r7, LSR r0
0000022c	00000100	ldr r10,=num
00000230	00000200	andeq r0, r0, r0, LSL #2
00000234	00000000	ldr r11,=result
00000238	aaaaaaaa	bge 0xfeaaace8
0000023c	aaaaaaaa	bge 0xfeaaacfc
00000240	aaaaaaaa	bge 0xfeaaad00
00000244	aaaaaaaa	bge 0xfeaaad04
00000248	aaaaaaaa	bge 0xfeaaad08
0000024c	aaaaaaaa	bge 0xfeaaad0c
00000250	aaaaaaaa	
00000254	aaaaaaaa	
00000258	aaaaaaaa	
0000025c	aaaaaaaa	

RegistersCall stackTraceBreakpointsWatchpointsSymbolsCounters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Editor Options

Code completion (Ctrl-space)

Editor (Ctrl-E) Disassembly (Ctrl-D) Memory (Ctrl-M)

程序运行结果

Running

Step Into
F2

Step Over
Ctrl-F2

Step Out
Shift-F2

Continue
F3

Stop
F4

Restart
Ctrl-R

Reload
Ctrl-Shift-L

File

Registers

Refresh

r0	00000228	
r1	00000001	
r2	0000000a	
r3	0000000a	
r4	00000022	
r5	00000037	
r6	00000037	
r7	00000014	
r8	00000000	
r9	00000000	
r10	00000100	
r11	00000200	
r12	00000208	
sp	00000000	
lr	00000000	
pc	00000064	
cpsr	600001d3	NZCVI SVC
spsr	00000000	NZCVI ?

RegistersCall stackTraceBreakpointsWatchpointsSymbolsCounters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Editor Options

Code completion (Ctrl-space)

Memory (Ctrl-M)

Go to address, label, or register:

Refresh

Address	Memory contents and ASCII
00000000	e59fa224 e59fb224 e59a2000 e3a01000
00000010	e28bc000 e58c1000 e3a01001 e28bc004
00000020	e58c1000 e28bc008 e58c1000 e3a03002
00000030	e3a04001 e3a05001 e0846005 e1a04005
00000040	e1a05006 e2833001 e0837003 e0870007
00000050	e080000b e5806000 e1520003 0a000000
00000060	ea000000 ea000000 00000000 00000000
00000070	00000000 00000000 00000000 00000000
00000080	00000000 00000000 00000000 00000000
00000090	00000000 00000000 00000000 00000000
000000a0	00000000 00000000 00000000 00000000
000000b0	00000000 00000000 00000000 00000000
000000c0	00000000 00000000 00000000 00000000
000000d0	00000000 00000000 00000000 00000000
000000e0	00000000 00000000 00000000 00000000
000000f0	00000000 00000000 00000000 00000000
00000100	0000000a 00000000 00000000 00000000
00000110	00000000 00000000 00000000 00000000
00000120	00000000 00000000 00000000 00000000
00000130	00000000 00000000 00000000 00000000
00000140	00000000 00000000 00000000 00000000
00000150	00000000 00000000 00000000 00000000
00000160	00000000 00000000 00000000 00000000
00000170	00000000 00000000 00000000 00000000
00000180	00000000 00000000 00000000 00000000
00000190	00000000 00000000 00000000 00000000
000001a0	00000000 00000000 00000000 00000000
000001b0	00000000 00000000 00000000 00000000
000001c0	00000000 00000000 00000000 00000000
000001d0	00000000 00000000 00000000 00000000
000001e0	00000000 00000000 00000000 00000000
000001f0	00000000 00000000 00000000 00000000
00000200	00000000 00000001 00000001 00000002
00000210	00000003 00000005 00000008 0000000d
00000220	00000015 00000022 00000037 00000100
00000230	00000200 00000000 aaaaaaaa aaaaaaaa
00000240	aaaaaaaa aaaaaaaa aaaaaaaa aaaaaaaa
00000250	aaaaaaaa aaaaaaaa aaaaaaaa aaaaaaaa
00000260	aaaaaaaa aaaaaaaa aaaaaaaa aaaaaaaa
00000270	aaaaaaaa aaaaaaaa aaaaaaaa aaaaaaaa
00000280	aaaaaaaa aaaaaaaa aaaaaaaa aaaaaaaa

RegistersCall stackTraceBreakpointsWatchpointsSymbolsCounters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Editor Options

Code completion (Ctrl-space)

Editor (Ctrl-E) Disassembly (Ctrl-D) Memory (Ctrl-M)

二、设计实验

1、MIPS汇编语言程序设计实验

- 请使用单周期MIPS处理器的24条指令，编写**排序程序（设计实验）**
 - 假设有10个数据，这10个数据的顺序是随机的，例如：8、1、5、2、7、9、6、4、3、10；这10个数据是通过指令存放在数据存储器的第0、4、8、…、36号地址的存储单元中。
 - 要求：（1）对这10个数据按照**从大到小**的顺序（降序）进行排序，排序后的数据仍然放在数据存储器的第0、4、8、…、36号地址的存储单元中。（2）对这10个数据按照**从小到大**的顺序（升序）进行排序，排序后的数据仍然放在数据存储器的第0、4、8、…、36号地址的存储单元中。请分别编写程序。
 - 要求：编写好的程序先在MIPS汇编器（**MARS 4.5**）上运行通过；然后再到Logisim实现的单周期MIPS处理器（**单周期MIPS处理器.circ**）上运行通过，观察程序运行的总周期数、无条件分支数、条件分支成功数分别是多少，并进行分析。

核心指令（8条）			
序号	MIPS指令	RTL描述	功能说明
1	add \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] + R[rt]$	寄存器加法指令，不考虑溢出
2	slt rd,\$rs,\$rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于量1指令，有符号数比较
3	addi \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	立即数加法指令，不考虑溢出
4	lw \$rt,imm(\$rs)	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$	取数指令
5	sw \$rt,imm(\$rs)	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$	存数指令
6	beq \$rs,\$rt,imm	$(R[rs] == R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(imm) < 2$	条件分支指令：如果 $rs == rt$ ，则跳转
7	bne \$rs,\$rt,imm	$(R[rs] != R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(imm) < 2$	条件分支指令：如果 $rs != rt$ ，则跳转
8	syscall	系统调用指令，用于停机	系统调用指令，用于停机

24条指令

基础指令（16条）			
序号	MIPS指令	RTL描述	功能说明
1	addiu \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] + R[rt]$	无符号寄存器加法指令
2	addiu \$rd,\$rs,imm	$R[rd] \leftarrow R[rs] + \text{SignExt}(imm)$	无符号立即数加法指令
3	andi \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] \& R[rt]$	逻辑与指令
4	andi \$rd,\$rs,imm	$R[rd] \leftarrow R[rs] \& (\text{16个0, imm})$	立即数逻辑与指令
5	ori \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] \vee R[rt]$	逻辑或指令
6	ori \$rd,\$rs,imm	$R[rd] \leftarrow R[rs] \vee (\text{16个0, imm})$	立即数逻辑或指令
7	xori \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] \oplus R[rt]$	逻辑异或指令
8	xori \$rd,\$rs,imm	$R[rd] \leftarrow R[rs] \oplus (\text{16个0, imm})$	立即数逻辑异或指令
9	slr \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] \ll R[rt]$	逻辑左移指令
10	slr \$rd,\$rs,imm	$R[rd] \leftarrow R[rs] \ll (\text{16个0, imm})$	立即数逻辑左移指令
11	sllr \$rd,\$rs,\$rt	$R[rd] \leftarrow (R[rs] \ll R[rt]) \> 1:0$	逻辑左移带符号指令
12	sllr \$rd,\$rs,imm	$R[rd] \leftarrow (R[rs] \ll (\text{16个0, imm})) \> 1:0$	小于量1指令，无符号数比较
13	sllt \$rd,\$rs,\$rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于量1指令，有符号数比较
14	j address	$PC \leftarrow \{PC[4:1], \text{address} < 2\}$	无条件分支指令
15	jal address	$R[31] \leftarrow PC[4] \quad PC \leftarrow \{PC[4:1], \text{address} < 2\}$	子程序调用指令
16	jz \$rs	$PC \leftarrow R[rs]$	条件跳转指令

核心指令（8条）

序号	MIPS指令	RTL描述	功能说明
1	add \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] + R[rt]$	寄存器加法指令，不考虑溢出
2	slt rd,rs,rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于置1指令，有符号数比较
3	addi rt,rs,imm	$R[rt] \leftarrow R[rs] + \text{SignExt}(\text{imm})$	立即数加法指令，不考虑溢出
4	lw rt,imm(rs)	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(\text{imm})]$	取数指令
5	sw rt,imm(rs)	$M[R[rs] + \text{SignExt}(\text{imm})] \leftarrow R[rt]$	存数指令
6	beq rs,rt,imm	$\text{if}(R[rs] == R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(\text{imm}) \ll 2$	条件分支指令：如果rs == rt，则跳转
7	bne rs,rt,imm	$\text{if}(R[rs] != R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(\text{imm}) \ll 2$	条件分支指令：如果rs != rt，则跳转
8	syscall	系统调用指令，用于停机	系统调用指令，用于停机

基础指令（16条）

序号	MIPS指令	RTL描述	功能说明
1	addu \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] + R[rt]$	无符号寄存器加法指令
2	addiu \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	无符号立即数加法指令
3	and \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] \& R[rt]$	逻辑与指令
4	andi \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] \& \{16\text{个}0, imm\}$	立即数逻辑与指令
5	sll \$rd,\$rt,imm	$R[rd] \leftarrow R[rt] \ll \text{shamt}$	逻辑左移
6	srl \$rd,\$rt,imm	$R[rd] \leftarrow R[rt] \gg \text{shamt}$	逻辑左移
7	sra \$rd,\$rt,imm	$R[rd] \leftarrow R[rt] \gg \text{shamt}$	逻辑左移
8	sub \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] - R[rt]$	寄存器减法指令
9	or \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] R[rt]$	逻辑或指令
10	ori \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] \{16\text{个}0, imm\}$	立即数逻辑或指令
11	nor \$rd,\$rs,\$rt	$R[rd] \leftarrow \neg(R[rs] R[rt])$	逻辑或非指令
12	sltu \$rd,\$rs,\$rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于置1指令，无符号数比较
13	slti \$rt,\$rs,imm	$R[rd] \leftarrow (R[rs] < \text{SignExt}(imm)) ? 1:0$	小于置1指令，立即数比较
14	j address	$PC \leftarrow \{(PC+4)_{31:28}, address<<2\}$	无条件分支指令
15	jal address	$R[31] \leftarrow PC+4 \quad PC \leftarrow \{(PC+4)_{31:28}, address<<2\}$	子程序调用指令
16	jr \$rs	$PC \leftarrow R[rs]$	寄存器跳转指令

2、RISC-V汇编语言程序设计实验

- 请使用RISC-V处理器的9条指令，编写**排序程序（设计实验）**
 - 假设有10个数据，这10个数据的顺序是随机的，例如：8、1、5、2、7、9、6、4、3、10；这10个数据是通过指令存放在数据存储器的第0、4、8、…、36号地址的存储单元中。
 - 要求：（1）对这10个数据按照**从大到小**的顺序（降序）进行排序，排序后的数据仍然放在数据存储器的第0、4、8、…、36号地址的存储单元中。（2）对这10个数据按照**从小到大**的顺序（升序）进行排序，排序后的数据仍然放在数据存储器的第0、4、8、…、36号地址的存储单元中。请分别编写程序。
 - 要求：编写好的程序先在RISC-V汇编器（**RARS 1.5**）上运行通过；然后再到Logisim实现的单周期RISC-V处理器（**单周期RISC-V处理器.circ**）上运行通过。

• RISC-V核心指令集RV32I的9条指令（我们设计的RISC-V单周期处理器仅支持该9条指令）：

- | | |
|---------------------|--|
| ① add rd,rs1,rs2 | ; rs1+rs2->rd |
| ② slt rd,rs1,rs2 | ; 带符号数比较指令 if rs1<rs2 1->rd else 0->rd |
| ③ sltu rd,rs1,rs2 | ; 无符号数比较指令 if rs1<rs2 1->rd else 0->rd |
| ④ ori rd,rs1,imm12 | ; rs1 或 imm12 -> rd |
| ⑤ lw rd,rs1,imm12 | ; M[rs1+imm12] -> rd |
| ⑥ lui rd,imm20 | ; imm20 -> rd |
| ⑦ sw rs2,rs1,imm12 | ; rs2 -> M[rs1+imm12] |
| ⑧ beq rs1,rs2,imm12 | ; if rs1=rs2 then goto imm12 |
| ⑨ jal rd,imm20 | ; goto imm20 |

- RISC-V核心指令集RV32I的9条指令（我们设计的RISC-V单周期处理器仅支持该9条指令）：

- ① `add rd,rs1,rs2` ; `rs1+rs2->rd`
- ② `slt rd,rs1,rs2` ; 带符号数比较指令 if `rs1<rs2` 1->`rd` else 0->`rd`
- ③ `sltu rd,rs1,rs2` ; 无符号数比较指令 if `rs1<rs2` 1->`rd` else 0->`rd`
- ④ `ori rd,rs1,imm12` ; `rs1` 或 `imm12 -> rd`
- ⑤ `lw rd rs1,imm12` ; `M[rs1+imm12] -> rd`
- ⑥ `lui rd,imm20` ; `imm20 -> rd`
- ⑦ `sw rs2,rs1,imm12` ; `rs2 -> M[rs1+imm12]`
- ⑧ `beq rs1,rs2,imm12` ; if `rs1=rs2` then `goto imm12`
- ⑨ `jal rd,imm20` ; `goto imm20`

3、Intel x86汇编语言程序设计实验

- 请使用Intel x86的指令，编写**排序程序（设计实验）**
 - 假设有10个数据，这10个数据的顺序是随机的，例如：8、1、5、2、7、9、6、4、3、10。
 - 要求：（1）对这10个数据按照**从大到小**的顺序（降序）进行排序。（2）对这10个数据按照**从小到大**的顺序（升序）进行排序。请分别编写程序。
- 在masm32汇编工具上运行上述2个程序。

三、挑战性实验

ARMv7汇编语言程序设计实验

- 请使用ARMv7的指令，编写**排序程序（挑战性实验）**
 - 假设有10个数据，这10个数据的顺序是随机的，例如：8、1、5、2、7、9、6、4、3、10。
 - 要求：（1）对这10个数据按照**从大到小**的顺序（降序）进行排序。（2）对这10个数据按照**从小到大**的顺序（升序）进行排序。请分别编写程序。
- 在ARMv7汇编工具上运行上述2个程序。

需要同学们完成的设计实验的内容

- ① MIPS汇编语言程序设计（排序程序）（设计实验）——文件名：sort1_mips.asm（降序排序）、sort2_mips.asm（升序排序）
 - ② RISC-V汇编语言程序设计（排序程序）（设计实验）——文件名：sort1_riscv.asm（降序排序）、sort2_riscv.asm（升序排序）
 - ③ Intel x86汇编语言程序设计（排序程序）（设计实验）——文件名：sort1_x86.asm（降序排序）、sort2_x86.asm（升序排序）
 - ④ ARMv7汇编语言程序设计（排序程序）（挑战性实验）——文件名：sort1_armv7.s（降序排序）、sort2_armv7.s（升序排序）
- 完成挑战性实验，本次实验成绩加6分

实验报告文件提交要求

- 1、请按照FTP上的实验报告模板（**厦门大学计算机组成原理实验报告样本（第4次实验）.docx**），撰写实验报告（Word版本），实验报告命名为：**学号+姓名+第4次实验报告**
- 2、请将实验报告和6个（或8个）源程序文件打包为1个压缩文件，压缩文件命名为：**学号+姓名+第4次实验.zip**
- 3、将压缩文件上传到**平台**上，第4次实验报告提交**截止时间参实验报告模板**